

# **A9N Microkernel Manual**

v0.1.15

Rekka 'horizon' IGUMI  
2026-8-18



# Document Status

本書は、A9N Microkernel の利用者、User-level System の開発者、HAL の移植者を対象とする Manual である。OS, Process, Virtual Memory, System Call, Interrupt, C, C++, Rust の基礎知識を前提とし、A9N 固有の知識は前提としない。

| 項目              | 説明                                                                                                                              |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------|
| Manual Version  | 0.1.15.                                                                                                                         |
| 実装 Architecture | x86_64. Architecture に依存しない Interface を先に説明し、固有の Register, Page Table, Boot, I/O を「x86_64 ABI」にまとめる。                            |
| Source of Truth | SPENCER が Submodule として取得する A9N の公開 Header と実装を基準とする。User-level Rust Interface は a9n_abi と a9n_types を基準とする。                    |
| 互換性             | Stable ABI の保証期間と Backward Compatibility の期間は定義されていない。Component の組合せは SPENCER と User Payload の Cargo.lock が固定する Version を基準とする。 |

数値, Type, Operation, Data Layout は公開 Interface 定義を優先する。State Transition と Error 変換は実装を参照する。Test と既存 Markdown は補助資料であり、公開 Interface または実装と競合する説明を仕様として扱わない。



# Table of Contents

|                                                                |    |
|----------------------------------------------------------------|----|
| <b>Document Status</b> .....                                   | 2  |
| <b>Part I Start Here</b> .....                                 | 10 |
| <b>1 Introduction</b> .....                                    | 12 |
| 1.1 Purpose and Audience .....                                 | 12 |
| 1.2 System Boundary .....                                      | 12 |
| 1.3 Policy/Mechanism Separation .....                          | 13 |
| 1.4 Manual Structure .....                                     | 13 |
| <b>2 Getting Started</b> .....                                 | 16 |
| 2.1 Goal .....                                                 | 16 |
| 2.2 Execution Path .....                                       | 16 |
| 2.3 Host Requirements .....                                    | 17 |
| 2.4 Source Checkout .....                                      | 17 |
| 2.5 Build .....                                                | 18 |
| 2.6 Run .....                                                  | 18 |
| 2.7 Run on x86_64 Hardware .....                               | 19 |
| 2.8 Modify the User Payload .....                              | 19 |
| 2.9 Troubleshooting .....                                      | 20 |
| <b>Part II Architecture-independent Kernel Interface</b> ..... | 22 |
| <b>3 Capability</b> .....                                      | 24 |
| 3.1 Purpose and Authority .....                                | 24 |
| 3.2 Object-Capability Model and ACL .....                      | 24 |
| 3.3 Capability Rights .....                                    | 25 |
| 3.4 Slot-local Identifier .....                                | 26 |
| 3.5 Capability Descriptor .....                                | 27 |
| 3.5.1 Node Index の算出 .....                                     | 27 |
| 3.5.2 Addressing の例 .....                                      | 28 |
| 3.6 Copy, Move, Mint, Remove, Revoke .....                     | 29 |

|                                                     |           |
|-----------------------------------------------------|-----------|
| 3.7 Common Result .....                             | 30        |
| 3.8 Concurrency and Lifetime .....                  | 30        |
| 3.9 Reserved and Unavailable Types .....            | 31        |
| <b>4 Kernel Call .....</b>                          | <b>32</b> |
| 4.1 Kernel Call Types .....                         | 32        |
| 4.2 Virtual Message Register .....                  | 32        |
| 4.3 Capability Call .....                           | 33        |
| 4.4 IPC Buffer .....                                | 34        |
| 4.5 Fault IPC .....                                 | 34        |
| 4.6 Interface Limitations .....                     | 35        |
| <b>Part III Kernel Object Reference .....</b>       | <b>36</b> |
| <b>5 Capability Node .....</b>                      | <b>38</b> |
| 5.1 Object Model .....                              | 38        |
| 5.2 Operation Summary .....                         | 38        |
| 5.3 COPY .....                                      | 39        |
| 5.4 MOVE .....                                      | 40        |
| 5.5 MINT and DEMOTE .....                           | 40        |
| 5.6 REVOKE and REMOVE .....                         | 40        |
| 5.7 Index and Error Semantics .....                 | 40        |
| 5.8 Concurrency .....                               | 41        |
| <b>6 Generic .....</b>                              | <b>42</b> |
| 6.1 Object Model .....                              | 42        |
| 6.2 CONVERT .....                                   | 42        |
| 6.3 Type-specific Parameters .....                  | 43        |
| 6.4 Device Generic .....                            | 44        |
| 6.5 Allocation and Revoke .....                     | 44        |
| 6.6 Error Summary .....                             | 45        |
| <b>7 Address Space, Page Table, and Frame .....</b> | <b>46</b> |
| 7.1 Object Relationship .....                       | 46        |
| 7.2 Mapping Attribute .....                         | 46        |
| 7.3 Address Space Operations .....                  | 47        |
| 7.4 MAP .....                                       | 48        |

|                                                    |           |
|----------------------------------------------------|-----------|
| 7.5 UNMAP and TLB .....                            | 48        |
| 7.6 GET_UNSET_DEPTH .....                          | 48        |
| 7.7 Page Table Capability .....                    | 49        |
| 7.8 Frame Capability .....                         | 49        |
| 7.9 Concurrency .....                              | 49        |
| <b>8 Process Control Block and Scheduler .....</b> | <b>50</b> |
| 8.1 Object Model .....                             | 50        |
| 8.2 Operation Summary .....                        | 50        |
| 8.3 CONFIGURE .....                                | 51        |
| 8.4 Register Access .....                          | 52        |
| 8.5 Process States .....                           | 53        |
| 8.6 Benno Scheduling .....                         | 53        |
| 8.7 Quantum and Preemption .....                   | 54        |
| 8.8 Direct Schedule .....                          | 55        |
| 8.9 Scheduling Limitations .....                   | 55        |
| 8.10 Revoke and Lifetime .....                     | 55        |
| <b>9 IPC Port .....</b>                            | <b>56</b> |
| 9.1 Object Model .....                             | 56        |
| 9.2 Message Layout .....                           | 56        |
| 9.3 Operation Summary .....                        | 57        |
| 9.4 SEND and RECEIVE .....                         | 58        |
| 9.5 CALL and Reply Authority .....                 | 58        |
| 9.6 IPC Fastpath .....                             | 59        |
| 9.7 Capability Transfer .....                      | 60        |
| 9.8 IDENTIFY .....                                 | 61        |
| 9.9 Concurrency and Revoke .....                   | 61        |
| <b>10 Notification Port .....</b>                  | <b>62</b> |
| 10.1 Object Model .....                            | 62        |
| 10.2 Operation Summary .....                       | 62        |
| 10.3 IDENTIFY .....                                | 63        |
| 10.4 Delivery and Wakeup .....                     | 63        |
| 10.5 Binding to a Process .....                    | 64        |

|                                                |    |
|------------------------------------------------|----|
| 10.6 Interrupt Delivery .....                  | 64 |
| 10.7 Revoke and Concurrency .....              | 64 |
| <b>11 Interrupt</b> .....                      | 66 |
| 11.1 Interrupt Object Model .....              | 66 |
| 11.2 Interrupt Region .....                    | 66 |
| 11.3 Interrupt Port .....                      | 67 |
| 11.4 Revoke and Concurrency .....              | 68 |
| <b>12 I/O Port</b> .....                       | 70 |
| 12.1 Object Model .....                        | 70 |
| 12.2 Operation Summary .....                   | 70 |
| 12.3 Address Range .....                       | 71 |
| 12.4 HAL Boundary and Lifetime .....           | 71 |
| <b>Part IV System Construction</b> .....       | 72 |
| <b>13 Boot and Init Protocol</b> .....         | 74 |
| 13.1 Scope .....                               | 74 |
| 13.2 Boot Flow .....                           | 74 |
| 13.3 Word 単位の Layout .....                     | 75 |
| 13.4 boot_info .....                           | 75 |
| 13.4.1 memory_info .....                       | 76 |
| 13.4.2 memory_map_entry .....                  | 76 |
| 13.4.3 init_image_info .....                   | 76 |
| 13.5 init_info .....                           | 77 |
| 13.5.1 generic_descriptor .....                | 77 |
| 13.5.2 init_info Layout .....                  | 77 |
| 13.6 Initial Capability Space .....            | 78 |
| 13.7 Address Space and Hardware Context .....  | 78 |
| 13.8 Generic Descriptor Generation .....       | 79 |
| 13.9 Init Startup Requirements .....           | 79 |
| 13.10 Boot Failure Model .....                 | 80 |
| <b>14 User-level System Architecture</b> ..... | 82 |
| 14.1 Scope .....                               | 82 |
| 14.2 Development Model .....                   | 82 |



|                                                     |            |
|-----------------------------------------------------|------------|
| 14.3 Implementation Units .....                     | 83         |
| 14.4 Software Roles .....                           | 83         |
| 14.5 Resource Lifetime and Concurrency .....        | 84         |
| <b>15 Building Init and Services .....</b>          | <b>86</b>  |
| 15.1 Scope and Preconditions .....                  | 86         |
| 15.2 Init Image Requirements .....                  | 86         |
| 15.3 Kernel Call Adapter .....                      | 87         |
| 15.4 Capability Inventory and Error Handling .....  | 87         |
| 15.5 Object Construction and Failure Recovery ..... | 88         |
| 15.6 Bootstrap to a Service Process .....           | 89         |
| 15.7 IPC Service Implementation .....               | 90         |
| 15.8 Notification and Fault Resolver .....          | 91         |
| 15.9 Service Shutdown .....                         | 92         |
| 15.10 Verification .....                            | 92         |
| <b>Part V Architecture-specific Interface .....</b> | <b>94</b>  |
| <b>16 x86_64 ABI .....</b>                          | <b>96</b>  |
| 16.1 Scope .....                                    | 96         |
| 16.2 Kernel Call Register .....                     | 96         |
| 16.3 Hardware Context .....                         | 97         |
| 16.4 Page Table .....                               | 98         |
| 16.5 I/O Port HAL .....                             | 99         |
| 16.6 Boot and Init .....                            | 99         |
| 16.7 Interrupt .....                                | 100        |
| 16.8 Initial Capability Descriptors .....           | 100        |
| 16.9 ABI Conformance Example .....                  | 100        |
| <b>17 HAL Porting Guide .....</b>                   | <b>102</b> |
| 17.1 Scope and Starting State .....                 | 102        |
| 17.2 Source Layout .....                            | 102        |
| 17.3 Architecture Constants .....                   | 103        |
| 17.4 Required HAL Interfaces .....                  | 103        |
| 17.5 Kernel Call Port .....                         | 104        |
| 17.6 Memory Port .....                              | 104        |

|                                                         |     |
|---------------------------------------------------------|-----|
| <u>17.7 Interrupt and Timer Port</u> .....              | 105 |
| <u>17.8 Boot Protocol Port</u> .....                    | 105 |
| <u>17.9 Build and Verification</u> .....                | 105 |
| <b><u>Part VI Ecosystem</u></b> .....                   | 108 |
| <b><u>18 Ecosystem</u></b> .....                        | 110 |
| <u>18.1 Scope</u> .....                                 | 110 |
| <u>18.2 Component Relationship</u> .....                | 110 |
| <u>18.3 a9n_types</u> .....                             | 110 |
| <u>18.4 a9n_abi</u> .....                               | 111 |
| <u>18.5 Nun</u> .....                                   | 111 |
| <u>18.6 A9NLoader-rs</u> .....                          | 111 |
| <u>18.7 SPENCER</u> .....                               | 112 |
| <u>18.8 Layer Selection</u> .....                       | 112 |
| <b><u>Back Matter Glossary and References</u></b> ..... | 114 |
| <b><u>Glossary</u></b> .....                            | 116 |
| <u>Architecture-independent Terms</u> .....             | 116 |
| <u>Kernel Objects and Runtime</u> .....                 | 119 |
| <u>Architecture-dependent Terms</u> .....               | 121 |
| <b><u>Acknowledgement</u></b> .....                     | 122 |
| <b><u>References</u></b> .....                          | 124 |

## Part I Start Here

本 Part は、A9N Microkernel の対象範囲と、標準構成を起動する最短の手順を示す。



## 1

## Introduction

### ■ 1.1 Purpose and Audience

A9N Microkernel[1] は、第 3 世代の Capability-based Microkernel である。A9N は、Capability に基づく権限管理、Process の実行、仮想メモリ、IPC、Notification、割り込み配送の機構を Kernel へ置き、Resource の配布と System Service の Policy を User Mode へ分離する。

本書は、A9N を初めて扱う OS 開発者が、標準構成の起動、共通 Kernel Interface の理解、Kernel Object の操作、Init と Service の実装、Architecture 固有 ABI の確認、HAL の移植を行える状態を目標とする。A9N の Source Repository は、GitHub の [horizon2038/A9N](https://github.com/horizon2038/A9N) で公開されている。

### ■ 1.2 System Boundary

A9N は、Kernel、HAL (Hardware Abstraction Layer)、User-level System の 3 領域に分かれる。Kernel は Architecture に依存しない Object Model と実行機構を実装する。HAL は CPU、Memory、Interrupt、Timer、I/O を Kernel Interface へ接続する。User-level System は Init を起点として Capability を配布し、Memory Manager、Driver、File System、System Service の Policy を実装する。

| 層                 | 主な Interface                       | 責務                                                                                        |
|-------------------|------------------------------------|-------------------------------------------------------------------------------------------|
| User-level System | Capability Call, IPC, Notification | Init, Resource Manager, Driver, Service, Application を構成する。                               |
| Kernel            | Kernel Object, Scheduler           | Capability の Authority, Process, Address Space, IPC, Notification, Fault を管理する。           |
| HAL               | HAL Interface                      | Architecture 固有の Context, Memory Mapping, Interrupt, Timer, I/O, Kernel Call Entry を実装する。 |

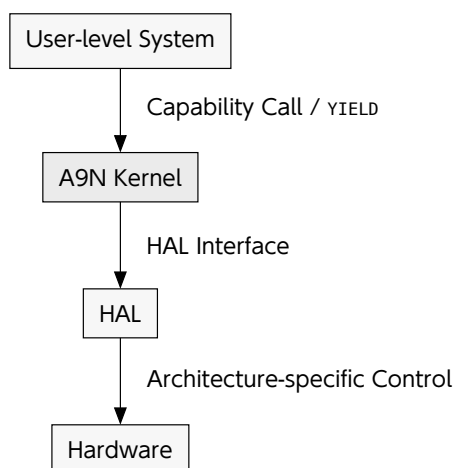


Figure 1: A9N の System Boundary

A9N が通常の User-level Software へ提供する Kernel Call は、**Capability Call** と **YIELD** の 2 種類である。Capability Call は、**Capability Descriptor** で指定した Capability を介して Kernel Object の Operation を実行する。YIELD は実行権を Scheduler へ返す。個別の Kernel Object Operation を独立した System Call として公開しない。

## ■ 1.3 Policy/Mechanism Separation

機構と方針の分離は、Resource の操作と保護に必要な Mechanism を Kernel へ置き、Resource の配分や Service の構成を決める Policy を User-level Software へ委ねる設計原則である。Hydra は、Scheduling, Paging, Protection の Mechanism を Kernel へ置き、外部の Policy が Mechanism を操作する構成としてこの原則を示した [2]。

A9N では、Capability による Authority の検査、Kernel Object の Operation、Process の実行、Memory Mapping、IPC、Notification、割り込み配送を Mechanism として提供する。Init と User-level System は、Capability をどの Process へ配布するか、Memory と CPU 時間をどの Service へ割り当てるか、Driver と System Service をどのように構成するかを Policy として決定する。Kernel に残る Type 検査、Rights 検査、Scheduling 規則、Resource 競合の処理は、保護境界と実行可能性を維持するための共通規則である。

## ■ 1.4 Manual Structure

Part I は、標準構成を起動して User Payload の実行を確認する。Part II は、Capability と Kernel Call の共通規約を定義する。Part III は、Capability Node, Generic, Memory, Process, IPC, Notification, Interrupt, I/O を Kernel Object ごとに記載する。Part IV は、Boot 後の Init と User-level System の構成を説明する。Part V は、x86\_64 固有 ABI と HAL 移植手順を扱う。Part VI は、A9N と組み合わせる Software Component を説明する。

最初に A9N を起動する読者は「Getting Started」へ進む。Kernel Interface を実装する読者は Part II から Part IV までを順に読む。既存 Runtime を使わずに Entry または Kernel Call Adapter を実装する読者は、共通 Kernel Interface を確認した後に「x86\_64 ABI」を読む。新しい Architecture を実装する読者は、既存の具体例として「x86\_64 ABI」を確認してから「HAL Porting Guide」へ進む。





# 2

## Getting Started

### ■ 2.1 Goal

本章では、SPENCER が用意する標準構成を Build し、A9N Microkernel 上で User Payload が動作するところまでを確認する。標準構成は、A9NLoader-rs を Bootloader、Nun を User-level Runtime、SPENCER の core Package を User Payload として用いる。Nun は a9n\_abi を介して Kernel Interface を利用し、a9n\_abi は共有型を定義する a9n\_types へ依存する。

Version 0.1.15 の SPENCER CLI は、x86\_64 Architecture に対して `--platform qemu` を受理する。Getting Started では各 Interface の内部を変更せず、標準構成の Boot を先に確認する。生成した x86\_64 Disk Image は、QEMU だけでなく UEFI 実機でも起動できる。Capability を用いた Service の構成は「Building Init and Services」、Register 配置や独自 Entry の実装は「x86\_64 ABI」に記載する。

### ■ 2.2 Execution Path

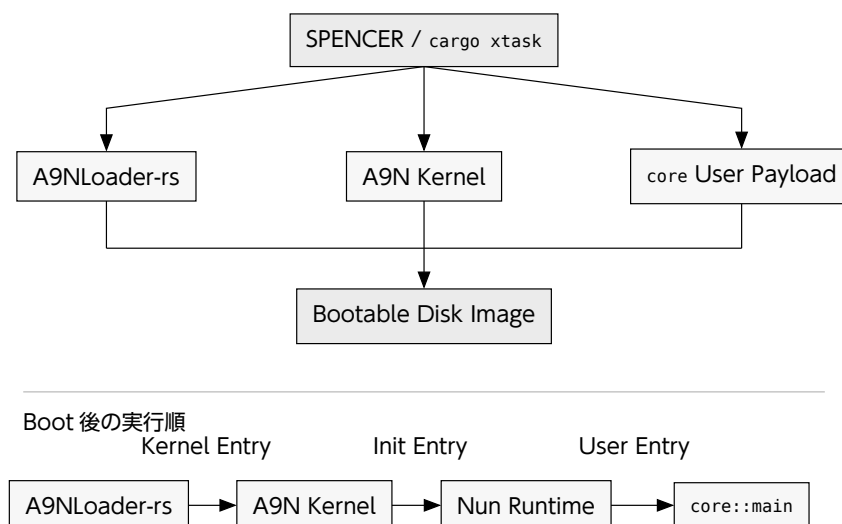


Figure 2: 標準構成の Build 対象と Boot 後の実行順

SPENCER は A9N Kernel, A9NLoader-rs, User Payload を個別に Build し、三つの Executable を一つの Disk Image へ格納する。Boot 時には A9NLoader-rs が Kernel と Init Image を読み込み、A9N Kernel へ制御を渡す。Kernel は core を Init として構成する。core の Entry は Nun が初期化し、IPC Buffer を Runtime へ対応付けた後に main を呼ぶ。

a9n\_abi と a9n\_types は独立した Executable ではない。Cargo Dependency として Nun と User Payload へ Link され、Entry, Kernel Call Wrapper, InitInfo, IpcBuffer 等を構成する。

## ■ 2.3 Host Requirements

Build には Git, rustup, CMake 3.29 以上, LLVM 18 以上, NASM, QEMU を用いる。LLVM には Clang, Clang++, LLD, llvm-config が必要である。UEFI Firmware は A9NLoader-rs Submodule の tools Directory に含まれる。Rust Toolchain は SPENCER の rust-toolchain.toml が指定するため、別の Version を手動で選択しない。初回 Build では、Rust Toolchain と Cargo Package を取得するために Network 接続を用いる。Command 例は POSIX Shell を対象とし、SPENCER と Build Artifact の保存先には書き込み権限が必要である。

| Tool                | 用途                                                    |
|---------------------|-------------------------------------------------------|
| Git                 | SPENCER と Submodule を取得する。                            |
| rustup / Cargo      | 指定された Rust Toolchain で Nun と User Payload を Build する。 |
| CMake / LLVM / NASM | A9N Kernel と x86_64 HAL を Build する。                   |
| QEMU                | 生成した UEFI Disk Image を実行する。                           |

## ■ 2.4 Source Checkout

取得する Repository は SPENCER だけである。A9N, Nun, A9NLoader-rs は SPENCER の Git Submodule に含まれるため、A9N の別 Clone は不要である。GitHub の SSH Credential を用いない環境では、Clone Command の URL 置換規則によって Submodule URL を HTTPS へ置き換える。

```
cd /path/to/workspace
git -c url."https://github.com/".insteadOf=git@github.com: \
  clone --recurse-submodules \
  https://github.com/horizon2038/spencer.git spencer
cd spencer
git submodule status
```

/path/to/workspace は SPENCER を配置する Directory である。git submodule status の行頭にある - は、行に示された Submodule をまだ取得していない状態を表す。未取得の Submodule がある場合は、SPENCER Directory で git submodule update を実行する。

```
git submodule update --init --recursive
```

cargo には rustup が管理する Executable を用いる。Package Manager が導入した別の Cargo が先に選択される環境では、rustup の Executable Directory を Shell

の検索順で先に置く。command -v cargo は Cargo の Path を表示する。rustup show active-toolchain は選択された Toolchain を表示する。cargo xtask --help は SPENCER の Command を表示する。

```
command -v cargo
rustup show active-toolchain
cargo xtask --help
```

## ■ 2.5 Build

SPENCER Directory で次を実行する。--os-manifest を省略すると core/Cargo.toml, --os-target-json を省略すると Nun/arch/x86\_64-unknown-a9n.json, --os-binary を省略すると core が選択される。

```
cargo xtask build \
  --arch x86-64 \
  --platform qemu \
  --release
```

cargo xtask build は、CMake による A9N Kernel の Build, Cargo による A9NLoader-rs の Build, Cargo による core の Build, Disk Image の生成を順に実行する。core は Nun へ依存し、Nun は a9n\_abi へ、a9n\_abi は Cargo Package a9n-types へ依存する。Build Log の nun, a9n\_abi, a9n-types は、Cargo による依存関係の解決と Compile を示す。Cargo Package 名は a9n-types, Rust の crate 名は a9n\_types である。

| Artifact   | Path                                                                      |
|------------|---------------------------------------------------------------------------|
| Kernel     | out/x86_64-qemu-release/a9n/kernel.elf                                    |
| Init       | out/x86_64-qemu-release/nun_os_target_dir/x86_64-unknown-a9n/release/core |
| Loader     | out/x86_64-qemu-release/a9nloader/a9nloader-rs.efi                        |
| Disk Image | out/x86_64-qemu-release/spencer.img                                       |

Disk Image 内では、A9NLoader-rs を/EFI/BOOT/BOOTX64.EFI, A9N Kernel を/kernel/kernel.elf, core を/kernel/init.elf として配置する。Host 上の Artifact 名と Disk Image 内の File 名を区別する必要がある。

cargo xtask build の再実行は、out/x86\_64-qemu-release 以下の Artifact を更新し、spencer.img を再生成する。Source と Submodule の Revision は変更しない。

## ■ 2.6 Run

Build と同じ SPENCER Directory で次を実行する。run は Build Pipeline を実行した後、生成した Disk Image を QEMU で起動する。

```
cargo xtask run \
  --arch x86-64 \
  --platform qemu \
  --release
```

Boot が完了すると、Serial 出力の末尾に以下の 5 行が現れる。Version 文字列は Build した Kernel によって異なる。QEMU は Ctrl-A に続けて x を入力して終了する。

```
Nun - an operating system framework based on the A9N Microkernel
Configuring <init> ...
Configuring Initial IPC buffer to thread local storage...
Hello, world!
version: <kernel version>
```

Serial 出力には、各 Component の境界が実行順に現れる。A9NLoader-rs の出力に続いて A9N Kernel の初期化 Log が現れ、Nun の Logo と IPC Buffer 初期化を経て、core::main の Hello, world! へ到達する。Hello, world! より前の Boot Log は、Loader, Kernel, Runtime, User Payload の到達段階を示す。

## ■ 2.7 Run on x86\_64 Hardware

out/x86\_64-qemu-release/spencer.img は、x86\_64 UEFI Firmware が Removable Media として読み込める Raw Disk Image である。Disk Image は/EFI/BOOT/BOOTX64.EFI に A9NLoader-rs を保持する。USB メモリー等の Block Device 全体へ Disk Image を Byte 単位で書き込むことで、生成物を x86\_64 実機から起動できる。File System 上へ spencer.img を通常の File としてコピーする操作では、Boot Media を構成できない。

### WARNING

Raw Disk Image の書き込みは、指定した Block Device の既存 Partition と Data を上書きする。書き込み前に、対象が USB メモリー等の交換可能 Device 全体であることを Device 名、容量、接続状態から確認する必要がある。対象 Device を一意に識別できない場合は、書き込みを実行してはならない。

Host OS が提供する Disk Image Writer または Raw Device Write Tool へ、Image として out/x86\_64-qemu-release/spencer.img、Destination として USB メモリー等の Block Device 全体を指定する。書き込み完了後に Host OS の Flush と Device の取り出し処理を実行し、x86\_64 実機の UEFI Boot Menu から USB Device を選択する。Legacy BIOS Boot は対象外である。Secure Boot を有効にした Firmware は、署名されていない A9NLoader-rs の実行を拒否し得るため、Firmware が未署名の UEFI Application を許可する設定を用いる必要がある。

実機上の動作範囲は、A9N の x86\_64 HAL が対応する CPU、Firmware 情報、Interrupt Controller、Timer、I/O Device に依存する。USB Device から A9NLoader-rs を開始できても、未対応 Hardware を用いる Service や Driver の動作は保証されない。QEMU と実機は同じ Disk Image を利用できるが、Hardware 依存の初期化結果は一致するとは限らない。

## ■ 2.8 Modify the User Payload

標準の User Payload は core/src/main.rs にある。nun::entry! は Nun の Entry Stub を生成し、初期化後に InitInfo への参照を main へ渡す。nun::println! は Nun が提供する出力 Macro である。最小の変更は、表示する文字列を置き換えて再度 cargo xtask run を実行することで確認できる。

```

#![no_std]
#![no_main]

nun::entry!(main);

fn main(init_info: &nun::InitInfo) {
    nun::println!("A9N user payload");
    nun::println!(
        "kernel version: {}.{}.{}",
        init_info.kernel_major_version,
        init_info.kernel_minor_version,
        init_info.kernel_patch_version,
    );

    loop {}
}

```

最初の User Payload では、Nun が再公開する `InitInfo` と Macro だけを利用する。Kernel Call Wrapper を直接扱う場合は `a9n_abi`、共有型だけを扱う Library は `a9n_types` を用いる。各層の選択基準と責務は「Ecosystem」に記載する。

## ■ 2.9 Troubleshooting

| 現象                                     | 確認箇所                                                                                                     |
|----------------------------------------|----------------------------------------------------------------------------------------------------------|
| Cargo または LLVM の Dynamic Library Error | <code>command -v cargo</code> を確認し、rustup が管理する Cargo を選択する。                                             |
| Submodule 内の File が見つからない              | <code>git submodule status</code> を確認し、未取得なら <code>git submodule update --init --recursive</code> を実行する。 |
| QEMU が TCP Port 1234 を確保できない           | Host の 127.0.0.1:1234 を使用中の Process を停止してから再実行する。                                                        |
| core を Link できない                       | 既定の Nun Target JSON を用い、 <code>__init_info_start</code> と <code>__init_ipc_buffer_start</code> を保持する。    |



## **Part II Architecture-independent Kernel Interface**

本 Part は、すべての Architecture で共有する Capability Model と Kernel Call Interface を定義する。





# 3

## Capability

### ■ 3.1 Purpose and Authority

Capability は、Kernel Object に対する操作権限を表す、カーネル管理の参照である。Capability Slot は、Object 実体を指す component, capability\_type, Rights, 3 Word の Slot-local Data, Dependency 情報を持つ。ユーザ空間は Slot の Address を直接扱わず、Capability Descriptor で Slot を指定する。

Capability Call を実行するには、対象 Slot へ到達でき、Slot の Type が操作と一致し、必要な Rights が設定されていなければならない。Object 固有の操作では、追加の Descriptor や引数も検査する。

Capability は、Object の Memory Address をユーザ空間へ公開しない。  
FRAME::GET\_ADDRESS は宣言されているが、対象 Revision の FRAME Capability Call は  
DEBUG\_UNIMPLEMENTED を返す。

### ■ 3.2 Object-Capability Model and ACL

Object-Capability Model では、Object を指定する参照と Authority を、カーネルが管理する Capability として一体化する [3]。A9N では、Process の Root Capability Node から Capability Descriptor によって到達可能な Capability Slot が、Object Operation を呼び出す Authority の根拠となる。Rights を要求する Operation では、Capability Slot の Rights も検査する。Capability Descriptor は Slot の探索位置を表す値であり、Descriptor 値だけを知っていても Authority は得られない。

A9N の Object-Capability Model は、KeyKOS, EROS, seL4 の設計事例から影響を受けている。KeyKOS は Object への Key を Authority として保持し、Key を介した Message で Object Operation を呼び出す [4]。EROS は、Process, Node, Page を Capability によって参照する Object として構成し、Commodity Processor 上で Capability-based Architecture を実装した [5]。seL4 は、Object への参照と Rights を持つ Capability を CNode へ格納し、Thread の Root CNode から到達できる CSpace を Authority の範囲とする [6]。A9N の Capability Slot, Capability Node, Root Capability Node, Capability Descriptor による探索は、同じ

Object-Capability Model に基づくが、Object Type と Operation, Descriptor Encoding, Rights の意味は A9N 固有である。

Access Control List (ACL) は、Object ごとに、呼出し主体の Identity と許可する Operation を対応付けた Entry を保持し、要求された Operation を Entry と照合する Access Control Model である [7]。ACL では、Object 側の Entry を更新して Authority を変更する。Object-Capability Model では、Capability の COPY, MOVE, MINT によって Authority を委譲する。

主要な違いは、ACL が Object 側の List と呼出し主体の Identity を判定根拠とするのに対し、Object-Capability Model が呼出し主体から到達可能な Capability を判定根拠とする点にある。A9N の Kernel Object は、Process Identity と Rights を対応付ける共通 ACL を持たない。

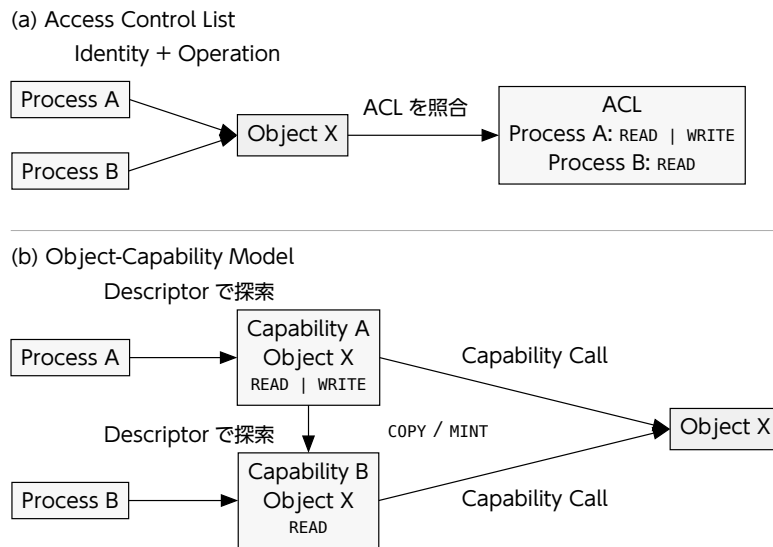


Figure 3: ACL と Object-Capability Model における Authority の所在

## 3.3 Capability Rights

| Right  | Value | Meaning                                                      |
|--------|-------|--------------------------------------------------------------|
| NONE   | 0x00  | 操作権限を持たない。                                                   |
| READ   | 0x01  | Object からの受信, 読出し, 待機に使用する。具体的な検査は Object Operation が定義する。   |
| WRITE  | 0x02  | Object への送信, 書込み, 変更に使用する。具体的な検査は Object Operation が定義する。    |
| COPY   | 0x04  | Node の COPY と MINT で Source Slot を複製できる。                     |
| MODIFY | 0x08  | IPC Port と Notification Port の Slot-local Identifier を変更できる。 |
| ALL    | 0x0f  | READ   WRITE   COPY   MODIFY を表す。                            |

Rights の意味は操作ごとに異なる。Rights Bit だけを見て、全 Object に共通する読出し・書込みの規則があると解釈してはならない。Node の MOVE は Source Slot の Rights を検査せず、Destination Node Slot に READ | WRITE を要求する。

MINT と DEMOTE に渡す `new_rights` は、Source Rights の Subset でなければならない。対象 Revision は未定義 Bit を拒否せず、Subset であるかだけを検査する。ユーザ空間は `0x0f` 以外の Bit を設定してはならない。

## ■ 3.4 Slot-local Identifier

Slot-local Identifier は、IPC Port または Notification Port を参照する Capability Slot の `data[0]` に保存する 1 Word 値である。Identifier は Port Object そのものの ID でも、Process や Thread の Global ID でもない。同じ Port Object を参照する複数の Capability Slot は、それぞれ異なる Identifier を保持できる。

IDENTIFY は、MR2 で指定した Identifier を呼出しに使用した Slot だけへ保存する。この操作には MODIFY Right が必要である。COPY と MINT は Slot-local Data を複製するため、作成された Slot は Source Slot と同じ Identifier を持つ。その後一方の Slot を IDENTIFY しても、他方の Slot は変化しない。Generic から作成した IPC Port と Notification Port の初期 Identifier は 0 である。

Identifier の配送は Kernel が行う。IPC Port では、Kernel が送信に使用された Slot の Identifier を取り出し、Message とは別に Receiver の MR3 へ書く。Notification Port では、Kernel が NOTIFY に使用された Slot の Identifier を Pending Flag へ Bitwise OR し、WAIT または POLL の MR2 へ返す。Notification Port を Process へ Bind した場合は、Pending Flag を IPC 形式の Result に含め、MR3 へ返す。呼出し側が Payload へ自己申告した値を Receiver が識別情報として信用する方式ではない。

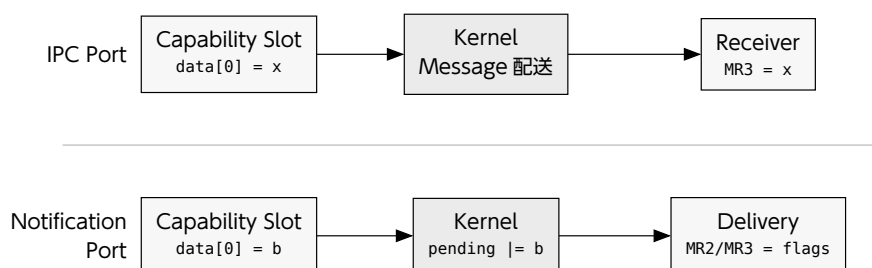


Figure 4: Slot-local Identifier の Kernel による配送

Kernel が保証するのは、呼出しに使用された Capability Slot に保存された値を、Object 固有の規則で配送することである。Identifier の一意性や意味は Kernel が決めない。User-level System は、IPC Client, Session, Capability の役割, Request の配送先を識別する Label として利用できる。Notification では、各 bit を Device, IRQ, Event 種別, Wakeup 理由へ割り当て、複数の Event Source を一つの Port へ多重化できる。

Identifier を信頼できる識別情報として使う場合、Authority を配布する側が値を設定し、MINT または DEMOTE によって受領側 Slot から MODIFY を除く必要がある。MODIFY を保持する主体は Identifier を変更できるため、その Slot から届く Identifier を変更不可能な識別情報として扱うことはできない。

## ■ 3.5 Capability Descriptor

Capability Descriptor は、Root Node を起点として Capability Slot を探索するための 1 Word Address である。上位 8 bit には Encoded Depth を置き、残りの bit を Descriptor Payload として用いる。Word 幅を  $W$ 、Encoded Depth を  $E$ 、Descriptor Payload を  $P$ 、復元後の Depth を  $D$  とすると、Descriptor  $d$  は次式で表せる。

$$\begin{aligned} d &= E \times 2^{W-8} + P, \\ D &= E + 8. \end{aligned} \quad (1)$$

Depth を直接格納する形式ではない。ユーザ空間は、目的の探索境界  $D$  から  $E = D - 8$  を求め、上位 8 bit へ格納する。 $E$  の範囲は  $0 \leq E \leq 255$ 、 $P$  の範囲は  $0 \leq P < 2^{W-8}$  である。一般化した符号化処理は次の通りである。

```
encoded_depth = depth - 8
raw_descriptor = (encoded_depth << (WORD_BITS - 8)) | descriptor_payload
```

`extract_depth()` は上位 8 bit を取り出した後で 8 を加える。上位 8 bit が 0 の場合も Depth は 8 となる。`traverse_slot()` は Root Node の Slot を選択した後で Depth を比較するため、Depth 8 では Capability を指定できない。Capability Call で指定できる最小 Depth は、 $8 + i_0 + r_0$  である。

### 3.5.1 Node Index の算出

探索は `traverse_slot(d, D, 8)` から始まる。第  $j$  段の Node へ入る前の `descriptor_used_bits` を  $u_j$ 、Node の `ignore_bits` を  $i_j$ 、`radix_bits` を  $r_j$  とする。初期値は  $u_0 = 8$  である。

Ignore Bits は読み飛ばす bit 数であり、Radix Bits は Node Index として読む bit 数である。`calculate_capability_index()` に合わせ、Mask を  $M_j$ 、右 Shift 量を  $S_j$ 、Node Index を  $I_j$  とすると、算出式は次の通りである。

$$\begin{aligned} M_j &= 2^{r_j} - 1, \\ S_j &= W - (u_j + i_j + r_j), \\ I_j &= \left\lfloor \frac{d}{2^{S_j}} \right\rfloor \bmod 2^{r_j}, \\ u_{j+1} &= u_j + i_j + r_j. \end{aligned} \quad (2)$$

剰余演算は、右 Shift 後の値へ  $M_j$  を Bitwise AND する処理と等価である。実装との対応は次の通りである。

```
mask_bits = (1 << radix_bits) - 1
shift_bits = WORD_BITS - (ignore_bits + radix_bits + descriptor_used_bits)
index = (descriptor >> shift_bits) & mask_bits
new_used_bits = descriptor_used_bits + ignore_bits + radix_bits
```

Node は `slot[index]` を選択した後、 $u_{j+1}$  と Depth を比較する。両者が等しければ選択した Slot を探索結果として返す。 $u_{j+1} < D$  の場合は、Slot が参照する Capability Component へ探索を引き継ぐ。探索継続時に空 Slot へ到達した場合は EMPTY、Depth

に達する前に Node 以外の Capability へ到達した場合は TERMINAL となる。Capability Call は、両 Error を INVALID\_DESCRIPTOR へ変換する。

Depth は、探索対象までに消費するすべての Ignore Bits と Radix Bits を含まなければならない。探索対象が第  $k$  段の Node に属する場合、必要な Depth は次式となる。

$$D = 8 + \sum_{j=0}^k (i_j + r_j). \quad (3)$$

対象 Revision の Init Root Node と Generic から作成する Node は、いずれも ignore\_bits = 0 である。ignore\_bits = 0 の Capability Space では、Descriptor Payload を上位側から各 Node の radix\_bits ずつ分割する。

### 3.5.2 Addressing の例

具体例として、Word 幅  $W = 64$  の Capability Space を考える。Root Node, Node<sub>1</sub>, Node<sub>2</sub> の ignore\_bits はすべて 0 とし、次の接続を用いる。

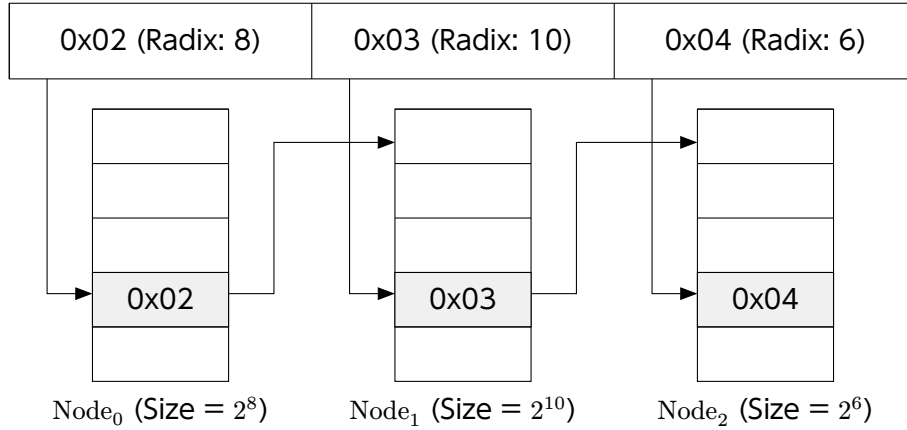


Figure 5: Capability 構成の例

Root Node は  $2^8 = 256$  Slot, Node<sub>1</sub> は  $2^{10} = 1024$  Slot, Node<sub>2</sub> は  $2^6 = 64$  Slot を持つ。Target Capability までに消費する bit 数は次の通りである。

$$D_{\text{target}} = 8 + 8 + 10 + 6 = 32. \quad (4)$$

先頭の 8 は Encoded Depth Field 自体の幅である。Descriptor へ格納する値は  $D_{\text{target}}$  ではなく、 $E = 32 - 8 = 24 = 0x18$  となる。Descriptor Payload は、各 Node Index を所定の bit 位置へ配置して作る。

$$\begin{aligned} P &= 2 \times 2^{48} + 3 \times 2^{38} + 4 \times 2^{32} \\ &= 0x0002_00c4_0000_0000, \\ d &= 24 \times 2^{56} + P \\ &= 0x1802_00c4_0000_0000. \end{aligned} \quad (5)$$

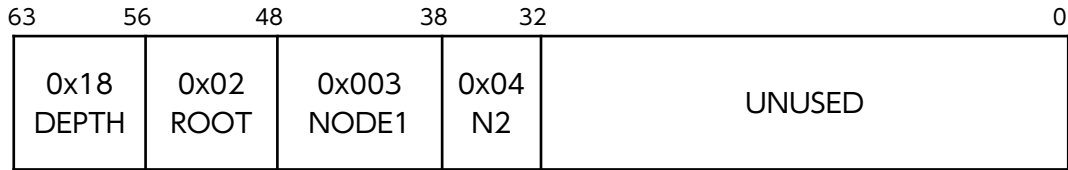


Figure 6: 0x1802\_00c4\_0000\_0000 の Bit 配置

各段の計算結果を次に示す。

| 段階 | Node      | $u_j$ | $s_j$ | Index | 判定             |
|----|-----------|-------|-------|-------|----------------|
| 0  | Root Node | 8     | 48    | 0x02  | $u_1 = 16 < D$ |
| 1  | Node 1    | 16    | 38    | 0x003 | $u_2 = 26 < D$ |
| 2  | Node 2    | 26    | 32    | 0x04  | $u_3 = 32 = D$ |

Root Node は bit 55..48 から 0x02 を読み、Node<sub>1</sub> へ進む。Node<sub>1</sub> は bit 47..38 から 0x003 を読み、Node<sub>2</sub> へ進む。Node<sub>2</sub> は bit 37..32 から 0x04 を読む。更新後の Used Bits が Depth 32 と一致するため、slot[0x04] に格納された Target Capability が探索結果となる。下位 32 bit は探索に使用しないため、ユーザ空間は 0 で埋める必要がある。

途中の Node 自身も指定できる。Root Node の slot[0x02] に格納された Node<sub>1</sub> を指定する場合、探索は Root Node だけで終わる。必要な Depth は  $D = 8 + 8 = 16$ 、Encoded Depth は  $E = 8$  であり、Descriptor は次の値となる。

```
raw_descriptor = 0x0802_0000_0000_0000
```

Depth を 32 のままにすると探索は Node<sub>1</sub> で止まらず、後続の Index を読み続ける。Depth は Capability の Type ではなく、探索を終了する Slot 境界を指定する値である。

#### WARNING

対象 Revision は、復元した Depth が Word 幅以下であることを検査しない。Word 幅を超える Depth や Node 境界と一致しない Depth は、Shift 量の Underflow を招く。ユーザ空間は  $8 < D \leq W$  を満たし、かつ  $D = 8 + \sum (i_j + r_j)$  となる Descriptor だけを生成する必要がある。retrieve\_slot(index) を直接利用する Node 操作も Index 上限を検査しないため、Operation 引数の Index は  $0 \leq \text{index} < 2^{\text{radix\_bits}}$  へ制限する必要がある。

## 3.6 Copy, Move, Mint, Remove, Revoke

COPY は、Source Slot と同じ Object, Rights, Slot-local Data を持つ Sibling Slot を作る。MOVE は、Object への参照, Rights, Slot-local Data, Dependency 上の位置を Destination Slot へ移し、Source Slot を NONE に戻す。MINT は COPY を行った後、Destination Rights を new\_rights へ制限する。

REMOVE は対象 Slot を空に戻す。同じ Object を参照する Sibling が残っている場合は、対象 Slot だけを Dependency 列から外す。最後の Sibling を削除する場合は、Object 固有の revoke() を呼んだ後で Slot を初期化する。

REVOKE が行う処理は Object ごとに異なる。Generic は Watermark を Base Address へ戻す。Process Control Block は Process を停止して内部参照を外す。Interrupt Port は IRQ Binding を解除する。Page Table, Frame, IPC Port, Notification Port, I/O Port には、限定的な処理や空の `revoke()` が残る。対象 Revision は、参照整合性を維持したまま Object 用 Memory を再利用する手順を定めていない。

## ■ 3.7 Common Result

Capability Call は、共通 Result を MR0 と MR1 へ返す。操作固有の Result は MR2 以降へ返す。

| 項目      | 説明                                               |
|---------|--------------------------------------------------|
| MR0 = 1 | 操作が成功した。MR1 の値は定義されない。                           |
| MR0 = 0 | 操作が失敗した。MR1 に <code>capability_error</code> を返す。 |

| Value | Error               | Meaning                                                   |
|-------|---------------------|-----------------------------------------------------------|
| 0     | ILLEGAL_OPERATION   | 操作が Type, State, Rights, または残容量に適合しない。                    |
| 1     | PERMISSION_DENIED   | 必要な Rights がない, または指定した権限が Source の権限を超える。                |
| 2     | INVALID_DESCRIPTOR  | Capability Descriptor による Slot 探索または Object Type 検査に失敗した。 |
| 3     | INVALID_DEPTH       | Error 値は定義されるが, 主要な Slot 探索経路では INVALID_DESCRIPTOR へ変換する。 |
| 4     | INVALID_ARGUMENT    | Count, Index, Range, Address, Size のいずれかが受理されない。          |
| 5     | FATAL               | HAL Error または Kernel 内部整合性 Error が発生した。                   |
| 6     | DEBUG_UNIMPLEMENTED | 宣言済みの操作または Type が実装されていない。                                |

FATAL からユーザ空間で回復する方法は定義されていない。  
`convert_kernel_to_capability_error()` は、すべての `kernel_error` を FATAL へ変換する。

## ■ 3.8 Concurrency and Lifetime

Capability Object と Dependency 列には、共通の Locking 規約がない。SMP 用の CPU Affinity Field と Lock Primitive は存在するが、Capability 操作全体の Thread Safety は保証されない。同じ Object や Node を複数 Core から操作する場合は、User-level の Policy で直列化する必要がある。

Capability Slot の Lifetime は、Slot を所有する Node, または Process 内部 Slot の Lifetime に従う。Capability Object 用の Memory は Generic から単調増加で割り当てられ、REMOVE を行っても個別には返却されない。Generic を Revoke すると Watermark が初期値に戻るため、派生 Capability が残ったまま再割当てを行うと Alias が生じ得る。

## ■ 3.9 Reserved and Unavailable Types

Virtualization Capability 群は、対象 Revision では利用できない。Type 値と一部の Object 作成経路は存在するが、Virtual Machine を構成して Guest を実行する公開 Interface は完成していない。

| Object                | Type | 作成経路               | 実装状態                                                       |
|-----------------------|------|--------------------|------------------------------------------------------------|
| Virtual CPU           | 13   | Generic の CONVERT. | execute() は Operation を Dispatch せず、ILLEGAL_OPERATION を返す。 |
| Virtual Address Space | 14   | なし。                | Type 値だけが予約され、Component 実装を持たない。                           |
| Virtual Page Table    | 15   | なし。                | Generic 変換は INVALID_ARGUMENT となり、Component 実装を持たない。        |

### WARNING

Virtualization Capability を隔離境界として使用してはならない。Guest Memory の隔離、Guest Entry、VM Exit、Virtual Interrupt、Revoke は公開 Interface として実装されていない。x86\_64 HAL に存在する VMX 補助実装の状態は「x86\_64 ABI」に記載する。



## 4

## Kernel Call

## ■ 4.1 Kernel Call Types

Kernel Call は、ユーザ空間からカーネルの機能呼び出すための共通インターフェースである。命令、レジスタ配置、呼出しで破壊されるレジスタはアーキテクチャごとに異なる。カーネルが解釈する Call Type と Message Register の意味は共通である。

A9N が通常の User-level Software へ提供する Kernel Call は、CAPABILITY\_CALL と YIELD の 2 つである。Capability Node, Memory, Process, IPC, Notification, Interrupt, I/O を含む Kernel Object の操作は、すべて CAPABILITY\_CALL へ集約される。Kernel Object ごとに独立した Kernel Call Number は存在しない。

| Call            | Number | Description                                                                           |
|-----------------|--------|---------------------------------------------------------------------------------------|
| CAPABILITY_CALL | -1     | MR0 の Capability Descriptor によって対象 Capability Slot を探索し、MR1 の Object Operation を実行する。 |
| YIELD           | -2     | 実行中 Process を Ready Queue へ戻し、Quantum を 10 へ設定して Scheduling を実行する。Capability を要求しない。  |

DEBUG = -3 も実装に残るが、Header で Deprecated とされる開発用 Call である。DEBUG は通常の Kernel Interface を構成する Call として数えない。MR0 の下位 8 bit を Kernel Logger へ出力するだけであり、Production の User-level Software は利用しない。

定義されていない Call Number は、INVALID\_KERNEL\_CALL Fault として Fault Dispatcher へ渡される。Resolver IPC Port を持たない Process は BLOCKED\_SUSPEND へ遷移する。

## ■ 4.2 Virtual Message Register

Virtual Message Register は、Kernel Call の入力と出力を運ぶ Process ごとの論理 Word 列である。各 Word は MR0, MR1, … の Index で識別する。Virtual Message Register 方式は、L4 系 Microkernel の IPC 設計に由来する [8]。Kernel と Kernel Object は、HAL の `get_message_register(process, index)` および

`configure_message_register(process, index, value)` を介して同じ Index 空間を読み書きする。

Architecture を  $A$ , Architecture ABI が Hardware Register へ割り当てる MR Index の集合を  $R_A$  とする。HAL が実装する保存先の写像  $\text{map}_{A(i)}$  は、次式で表せる。

$$\text{map}_{A(i)} = \begin{cases} \text{hardware-context.register}_{A(i)} & i \in R_A \\ \text{ipc-buffer.messages}[i] & i \notin R_A \end{cases} \quad (6)$$

Hardware Register へ割り当てられる Index 集合  $R_A$ , 各 Index に対応する Register, Kernel Call Entry で破壊される Register は Architecture ABI が定める。Kernel Object は保存先を区別せず, MR Index だけを指定する。IPC Port も Sender の  $\text{MR}_i$  から Receiver の  $\text{MR}_i$  へ同じ Index で Payload を Copy する。

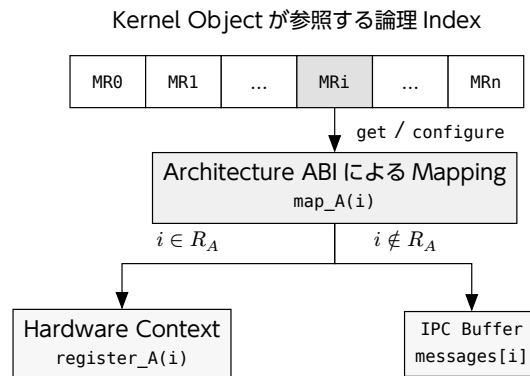


Figure 7: Virtual Message Register と保存先の対応

Virtual Message Register は型情報を持たない。Capability Descriptor, Operation Number, Bit Field, Address, Payload の意味は, Kernel Call または Capability Operation ごとの Message Layout が定める。呼出し側は, 利用する最大 MR Index を Architecture ABI と IPC Buffer の範囲に基づいて事前検査できる。事前検査の有無にかかわらず, Kernel Call の成否は返却された Result から判定する。

## ■ 4.3 Capability Call

入力は次の通りである。

| 項目    | 説明                            |
|-------|-------------------------------|
| MR0   | Raw Capability Descriptor.    |
| MR1   | 対象 Object の Operation Number. |
| MR2.. | Operation 固有の入力.              |

出力は次の通りである。

| 項目    | 説明                                   |
|-------|--------------------------------------|
| MR0   | 成功時は 1, 失敗時は 0.                      |
| MR1   | 失敗時の <code>capability_error</code> . |
| MR2.. | Operation 固有の出力.                     |

カーネルは、実行中 Process の Root Node を起点として、MR0 の Capability Descriptor によって対象 Capability Slot を探索する。空の Slot、探索途中の Leaf Object、Depth の不一致により対象 Slot へ到達できない場合、MR0 = 0 と MR1 = INVALID\_DESCRIPTOR を返す。Message Register とレジスタの対応は「x86\_64 ABI」に記載する。

## ■ 4.4 IPC Buffer

IPC Buffer は、Virtual Message Register の退避、IPC の Payload、Capability 転送の情報に用いる共有データ構造である。大きさは 1 Page 以内である。Hardware Register だけでは表せない Virtual Message Register を IPC Buffer へ割り当てるかどうかは HAL が定める。

messages の Word 数は、1 Page の Word 数から Capability Transfer 用の 18 Word を除いて算出する。16 Word は Source Descriptor 列、2 Word は Destination Node Descriptor と Destination Index である。

$$\text{MESSAGE\_BUFFER\_SIZE\_MAX} = \frac{\text{PAGE\_SIZE}}{\text{sizeof}(\text{word})} - 18 \quad (7)$$

| 項目                                | 説明                                                                             |
|-----------------------------------|--------------------------------------------------------------------------------|
| messages[MESSAGE_BUFFER_SIZE_MAX] | Virtual Message Register の Backing Store。MR Index との対応は Architecture ABI が定める。 |
| transfer_source_descriptors[16]   | 送信側 Root Node から移動元 Capability Slot を探索するための Descriptor を格納する。                 |
| transfer_destination_node         | 受信側 Root Node から Destination Node Slot を探索するための Capability Descriptor を格納する。   |
| transfer_destination_index        | 受信側 Destination Node 内の開始 Index を格納する。                                         |

PCB の CONFIGURE で IPC Buffer 用 Frame を設定すると、カーネルは Frame Capability を Process 内部の Slot へ Copy し、Frame をカーネルから参照できる Address へ変換して process.buffer へ設定する。Init には Boot 処理が同じ設定を行う。

### WARNING

Hardware Register へ割り当てられない Message Register を使う操作には、有効な IPC Buffer が必要である。IPC Buffer が設定されていない場合、HAL は NO\_SUCH\_ADDRESS を返す。HAL Error を kernel\_error へ変換する Capability Operation は、Result を FATAL として返す。Frame の大きさと配置はアーキテクチャごとの ABI に従い、カーネルへ設定する前に対象 Process の Address Space へ Map する必要がある。

## ■ 4.5 Fault IPC

Process Control Block へ Fault Resolver として IPC Port を設定すると、Kernel は Process Fault を Resolver へ CALL 相当で配送する。Fault を起こした

Process は Reply を受け取るまで BLOCKED\_FAULT または BLOCKED\_REPLY となる。  
Resolver Slot には READ | WRITE が必要である。

Fault Message の共通 Header は次の通りである。message\_info.source は FAULT = 1 となる。

| 項目  | 説明                              |
|-----|---------------------------------|
| MR0 | 1.                              |
| MR1 | 0.                              |
| MR2 | Kernel が生成した message_info.      |
| MR3 | Resolver Slot-local Identifier. |
| MR4 | fault_type.                     |

| Fault Type                          | Additional Message Registers                                                                                   |
|-------------------------------------|----------------------------------------------------------------------------------------------------------------|
| MEMORY,<br>MEMORY_INSTRUCTION_FETCH | MR5 = program counter, MR6 = fault address, MR7 = architecture fault code.                                     |
| INVALID_INSTRUCTION                 | MR5 = program counter, MR6 = architecture fault code.                                                          |
| INVALID_ARITHMETIC                  | MR5 = program counter, MR6 = architecture fault code.                                                          |
| INVALID_KERNEL_CALL                 | MR5 = program counter, MR6 = kernel call number, MR7.. = hardware context. Context の長さと同じはアーキテクチャごとの ABI が定める。 |
| ARCHITECTURE                        | MR5 = program counter, MR6 = architecture fault code.                                                          |

INVALID\_KERNEL\_CALL への Reply では、Resolver から返された Hardware Context が Fault を起こした Process へ適用される。Hardware Context の書換えは Privilege 境界に影響するため、Resolver はアーキテクチャごとの制約を満たす値だけを返す必要がある。

## ■ 4.6 Interface Limitations

Capability Call は、不正な Descriptor, Operation, Rights, Argument を Capability Error として返す。ユーザ空間の呼出し層は、Descriptor, Message Length, Transfer Count, Destination Index を Kernel Call の前に追加検査できる。事前検査は Error を早い段階で分類する手段であり、Kernel の検査を置き換える要件ではない。

Capability Call で成功時に定義される共通出力は MR0 だけである。MR1 は初期化されない。操作固有の Result を持たない Call では、古い Message Register 値が残り得る。

## Part III Kernel Object Reference

本 Part は、Architecture に依存しない Kernel Object を Type ごとに記載する。



## 5

## Capability Node

### ■ 5.1 Object Model

Capability Node は、 $2^{\text{radix\_bits}}$  個の Capability Slot を持つ Radix Tree である。子 Slot へ別の Node を格納すると、複数階層の Capability Space を構成できる。Generic の CONVERT で作成した Node では、`ignore_bits = 0` となる。

Node に対する操作では、Capability Call の対象となった Node を転送先とする。MR3 の Source Descriptor によって、呼出し元 Process の Root Node を起点として Source Slot を探索する。

### ■ 5.2 Operation Summary

| Method   | Message Register                                   | Description                                                                                                                             | Error                                                          |
|----------|----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------|
| COPY = 1 | MR2 = destination index<br>MR3 = source descriptor | Source Capability を Destination Slot へ Copy する。Target Node に READ   WRITE, Source Slot に COPY を要求する。Destination Slot は NONE でなければならない。  | ILLEGAL_OPERATION, PERMISSION_DENIED, INVALID_ARGUMENT, FATAL. |
| Method   | Message Register                                   | Description                                                                                                                             | Error                                                          |
| MOVE = 2 | MR2 = destination index<br>MR3 = source descriptor | Source Capability を Destination Slot へ Move する。Target Node に READ   WRITE を要求する。Source Rights は検査しない。Destination Slot は NONE でなければならない。 | ILLEGAL_OPERATION, INVALID_ARGUMENT, FATAL.                    |

| Method     | Message Register                                                       | Description                                                                                                            | Error                                                          |
|------------|------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------|
| MINT = 3   | MR2 = destination index<br>MR3 = source descriptor<br>MR4 = new rights | Source Capability を Copy し、Destination Rights を new_rights へ縮退する。Target Node に READ   WRITE, Source Slot に COPY を要求する。 | ILLEGAL_OPERATION, PERMISSION_DENIED, INVALID_ARGUMENT, FATAL. |
| Method     | Message Register                                                       | Description                                                                                                            | Error                                                          |
| DEMOTE = 4 | MR2 = target index<br>MR4 = new rights                                 | Target Slot の Rights を縮退する。Node Slot に WRITE, Target Slot に READ   WRITE を要求する。                                        | ILLEGAL_OPERATION, PERMISSION_DENIED, INVALID_ARGUMENT.        |
| Method     | Message Register                                                       | Description                                                                                                            | Error                                                          |
| REVOKE = 5 | MR2 = target index                                                     | Target Slot の派生 Slot を削除し、Object 固有の revoke() を呼ぶ。Target Slot に READ   WRITE を要求する。                                    | ILLEGAL_OPERATION, INVALID_ARGUMENT, Object 固有 Error.          |
| Method     | Message Register                                                       | Description                                                                                                            | Error                                                          |
| REMOVE = 6 | MR2 = target index                                                     | Target Slot を Dependency 列から外して初期化する。Node Slot に READ または WRITE, Target Slot に READ   WRITE を要求する。                     | PERMISSION_DENIED, INVALID_ARGUMENT, FATAL.                    |

$O$ : 同一 Object への参照,  $R' \subseteq R$

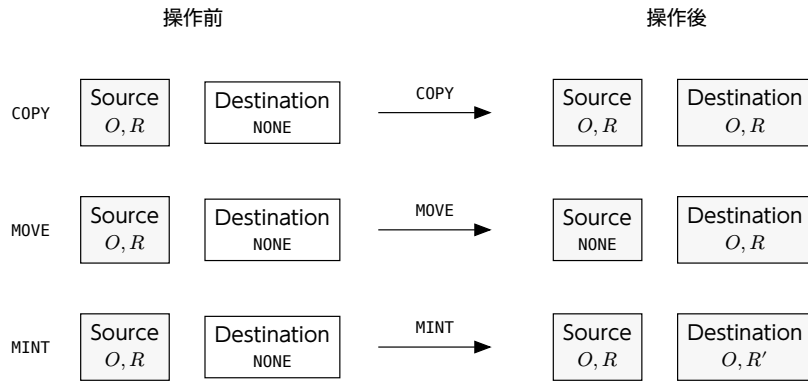


Figure 8: COPY, MOVE, MINT による Slot 内容の変化

## 5.3 COPY

COPY は、Source Slot の Type, Component, Rights, Slot-local Data を Destination Slot へ複製する。カーネルは Destination Slot を Source Slot と同じ Dependency Depth へ挿入する。Object 自体は複製しない。



COPY の成功後も Source Slot は有効である。Destination Slot の Rights は Source Rights と同一である。Rights 縮退を伴う複製には MINT を使用する。

## ■ 5.4 MOVE

MOVE は、Source Slot の内容と Dependency Link を Destination Slot へ移す。成功すると Source Slot の Type は NONE となる。Source Slot に COPY Right は必要ない。

Source と Destination に同じ Slot を指定した場合の動作は定義されていない。ユーザ空間は、異なる Slot を指定する必要がある。

## ■ 5.5 MINT and DEMOTE

MINT で Source Rights を超える Rights は作成できない。Subset であるかは次の式で検査する。

```
(source_rights & new_rights) == new_rights
```

DEMOTE は、既存 Slot の Rights を直接減らす。失われた Rights を同じ Slot へ戻す操作は存在しない。同じ Object を参照する別の Slot には影響しない。

## ■ 5.6 REVOKE and REMOVE

REMOVE は、対象 Slot に Sibling が残っている場合、Object 固有の Revoke を行わない。最後の Sibling を削除する場合だけ、カーネルが Object 固有の `revoke()` を呼ぶ。

REVOKE は、Target Slot に続く Dependency 列を走査し、Target Depth より深い Slot を順に削除する。削除する Slot が Object への最後の参照であれば、Slot の初期化前に Object 固有の `revoke()` を呼ぶ。Target Slot が呼出し対象と同じ Node Object を参照する場合は、無限再帰を避けるため Node 自身の `revoke()` を呼ぶ。

### CAUTION

Revoke と Remove は不可分ではない。Child の削除中に Error が起きても、削除済みの Slot は元に戻らない。Object ごとの `revoke()` には空実装が残るため、操作の成功だけを見て全 Resource が解放されたと判断してはならない。

## ■ 5.7 Index and Error Semantics

対象 Revision の `capability_node::retrieve_slot()` は、Index の範囲を検査しない。Header には `is_index_valid()` が存在するが、`retrieve_slot()` からは呼ばれない。Repository 内の境界 Test は実装と一致していない。

Destination Slot が使用中の場合、Low-level の Copy または Move は `kernel_error::INIT_FIRST` を返す。Node 操作は `convert_kernel_to_capability_error()` によって `kernel_error::INIT_FIRST` を FATAL へ変換する。ユーザ空間は、Destination Slot が NONE であることを Node の管理情報から保証する必要がある。

## ■ 5.8 Concurrency

Node の Slot 配列と Dependency Link に共通 Lock はない。同じ Node に対する COPY, MOVE, MINT, DEMOTE, REVOKE, REMOVE は、ユーザ空間で同期する必要がある。同じ Destination Index を使う操作は、必ず直列化する。

# Generic

## 6.1 Object Model

**Generic** は、カーネルオブジェクトの作成に使う物理メモリ領域を表す Capability である。Generic Slot は Base Address, Size Bits, Device Flag, Watermark を持つ。カーネルは、Generic の Watermark を必要な Alignment まで進め、後続領域からオブジェクト用メモリを割り当てる。一般的な Heap Allocator は使用しない。

| 項目               | 説明                                                        |
|------------------|-----------------------------------------------------------|
| data[0]          | Base Physical Address.                                    |
| data[1] bit 0..6 | Size Bits. Generic の範囲は $2^{\text{size\_bits}}$ Byte である. |
| data[1] bit 7    | Device Generic Flag.                                      |
| data[2]          | 次回割当ての基準となる Watermark.                                    |



Figure 9: data[1] の Bit 配置

Init へ渡される Generic には READ | WRITE が設定され、COPY は設定されない。  
 CONVERT は、呼出しに使った Generic Slot の Rights を検査しない。Generic  
 Capability を保持する Process は、対象領域から Kernel Object を作成できる。

## 6.2 CONVERT

Generic が受理する Operation Number は `CONVERT = 0` だけである。

| 項目  | 説明                                  |
|-----|-------------------------------------|
| MR0 | Generic Capability Descriptor.      |
| MR1 | 0.                                  |
| MR2 | 作成する <code>capability_type</code> . |

| 項目  | 説明                                                                               |
|-----|----------------------------------------------------------------------------------|
| MR3 | Type 固有の <code>specific_bits</code> .                                            |
| MR4 | 作成数. 1 以上でなければならない.                                                              |
| MR5 | 呼出し Process の Root Node から Destination Node Slot を探索するための Capability Descriptor. |
| MR6 | Destination 開始 Index.                                                            |

カーネルは、Object Size に合わせて Watermark を Alignment し、Destination Node の連続する Slot へ Object を作成する。Object を一つ作成するたびに Watermark を更新し、作成した Slot を Generic の Child として Dependency 列へ挿入する。

割当て単位を  $u = 2^{\text{memory\_size\_bits}}$ 、作成数を  $c$ 、割当て前の Watermark を  $w$  とする。割当て開始 Address は  $a = \text{align}(w, u)$ 、割当て後の Watermark は  $w' = a + cu$  となる。Alignment Gap を含む領域が Generic の終端を越える場合、`CONVERT` は Object を作成しない。

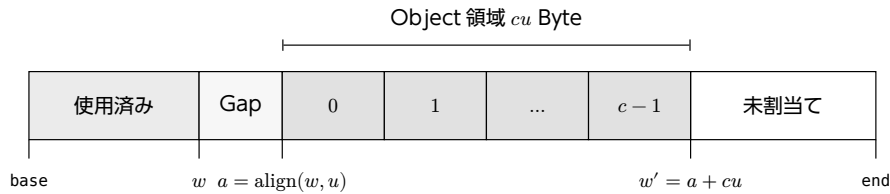


Figure 10: Watermark の Alignment と `CONVERT` による連続割当て

### CAUTION

`count > 1` の `CONVERT` は Atomic ではない。途中で Destination Slot の設定や Object の作成に失敗しても、作成済みの Object と進んだ Watermark は元に戻らない。ユーザ空間は作成済み Slot を調べ、必要に応じて Revoke または Remove を行う必要がある。

## 6.3 Type-specific Parameters

| Type                  | Value | specific_bits     | Result                                                                                                                               |
|-----------------------|-------|-------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| NODE                  | 2     | Node Radix.       | $2^{\text{specific\_bits}}$ Slot を持つ Node を作成する。 <code>try_make_capability_node()</code> は $1 \leq \text{specific\_bits} < W$ を要求する。 |
| GENERIC               | 3     | Child Size Bits.  | 親と同じ Device 属性を持つ Child Generic を作成する。                                                                                               |
| ADDRESS_SPACE         | 4     | 未使用.              | 1 Page を割り当て、HAL で Root Address Space を初期化する。                                                                                        |
| PAGE_TABLE            | 5     | Page Table Depth. | 1 Page を割り当て、Depth を Slot Data へ保存する。                                                                                                |
| FRAME                 | 6     | Frame Size Bits.  | HAL が受理する Size の Frame を作成する。受理する値はアーキテクチャごとの ABI が定める。                                                                              |
| PROCESS_CONTROL_BLOCK | 7     | 未使用.              | Process Control Block と User Hardware Context を初期化する。                                                                                |

| Type              | Value | specific_bits | Result                                                   |
|-------------------|-------|---------------|----------------------------------------------------------|
| IPC_PORT          | 8     | 未使用.          | Identifier 0 の IPC Port を作成する.                           |
| NOTIFICATION_PORT | 9     | 未使用.          | Identifier 0 の Notification Port を作成する.                  |
| VIRTUAL_CPU       | 13    | 未使用.          | Virtual CPU Object を作成する.<br>Capability Call は Stub である. |

Node 用 Memory の Size 計算は、`sizeof(capability_slot) * 2specific_bits` を含む。対象 Revision は乗算と加算の Overflow を検査しない。ユーザ空間は、計算結果が Word 幅に収まり、Alignment 後の領域全体が Generic の範囲内に収まる Radix だけを指定する必要がある。

| Type                  | Value | Failure                                                |
|-----------------------|-------|--------------------------------------------------------|
| NONE                  | 0     | INVALID_ARGUMENT.                                      |
| DEBUG                 | 1     | INVALID_ARGUMENT. Size 計算経路で拒否される.                     |
| INTERRUPT_REGION      | 10    | INVALID_ARGUMENT. Init Boot 経路だけが作成する.                 |
| INTERRUPT_PORT        | 11    | INVALID_ARGUMENT. Interrupt Region から作成する.             |
| IO_PORT               | 12    | INVALID_ARGUMENT. Init Boot 経路と I/O Port の MINT が作成する. |
| VIRTUAL_ADDRESS_SPACE | 14    | INVALID_ARGUMENT. Type 固有作成経路を持たない.                    |
| VIRTUAL_PAGE_TABLE    | 15    | INVALID_ARGUMENT. Size 計算経路で拒否される.                     |

## 6.4 Device Generic

Device Generic から作成できる Type は GENERIC と FRAME だけである。別の Type を指定すると、カーネルは INVALID\_ARGUMENT を返す。Child Generic は Device Flag を引き継ぐ。

Device Frame の Cache 属性や Memory Type は Generic Slot に含まれない。必要な属性をユーザ空間から指定できるかどうかは、Address Space の Mapping ABI が定める。

## 6.5 Allocation and Revoke

割当て単位は Type ごとに異なる。Node には、Node Object と Slot 配列を格納できる最小の 2 の冪を用いる。PCB, IPC Port, Notification Port, Virtual CPU には、各 Object Size を 2 の冪へ切り上げた値を用いる。Address Space と Page Table には基本 Page Size を用いる。Frame と Child Generic には specific\_bits を用いる。

残容量は、Alignment 後の Watermark に `unit_size * count` を加えた End Address で検査する。領域に収まらない場合は ILLEGAL\_OPERATION, `count = 0` の場合は INVALID\_ARGUMENT となる。

Generic の `revoke()` は Watermark を Base Address へ戻す。派生 Object の Memory は Clear せず、残っている Slot も無効化しない。Revoke 後に同じ領域を再割当てすると、古い Capability と新しい Object が同じ Memory を参照し得る。ユー

ザ空間は、全 Child と全 Sibling を無効にしてから Generic を Revoke する必要がある。

## ■ 6.6 Error Summary

| Error              | Condition                                                                           |
|--------------------|-------------------------------------------------------------------------------------|
| INVALID_DESCRIPTOR | Destination Node Slot の探索または NODE Type 検査に失敗した。                                     |
| INVALID_ARGUMENT   | count = 0, 未対応 Type, 不正な Frame Size, Device Generic からの不許可変換, 使用中 Destination Slot. |
| ILLEGAL_OPERATION  | Generic 残容量が不足した, または Destination Node が利用できない。                                     |
| FATAL              | HAL 初期化, Memory Alignment, Message Register の読出し, 内部 Slot 操作が失敗した。                  |

## 7

## Address Space, Page Table, and Frame

### 7.1 Object Relationship

Address Space は Root Page Table を表す Capability である。Page Table は中間 Page Table を、Frame は物理メモリ範囲を表す。Mapping と Unmapping は Address Space Capability Call から実行し、Page Table と Frame は操作の引数として指定する。

Generic の CONVERT は、Address Space と Page Table に基本 Page Size の領域を割り当て、Frame に指定 Size の物理メモリを割り当てる。Address Space の初期化、User Address の範囲、利用できる Frame Size は HAL が定める。

Address Space に対する操作は、Address Space Slot の Rights を検査しない。Address Space Capability を保持する Process は、対象 Address Space の Mapping を変更できる。

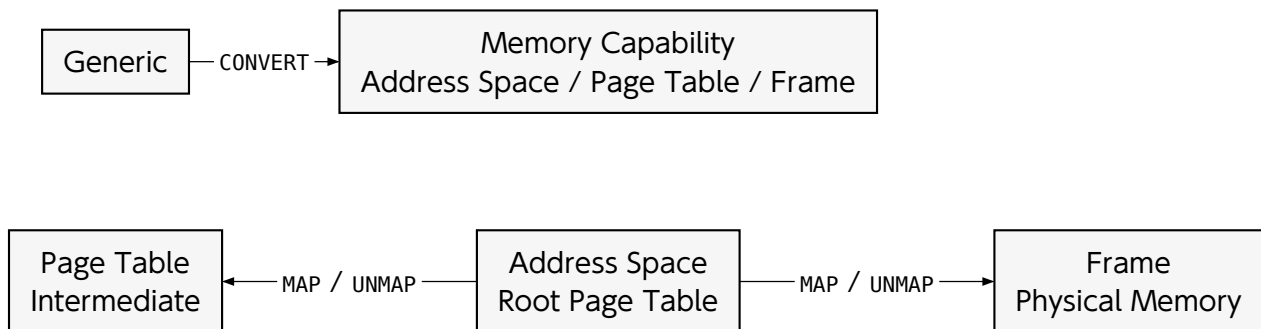


Figure 11: Memory Capability Object の作成と Mapping 操作

### 7.2 Mapping Attribute

MAP の MR4 は Memory Rights Bit Mask である。

| Right   | Value | Meaning                                      |
|---------|-------|----------------------------------------------|
| READ    | 0x1   | 読出しを許可する。Hardware Page Entry への変換は HAL が定める。 |
| WRITE   | 0x2   | 書込みを許可する。                                    |
| EXECUTE | 0x4   | 命令実行を許可する。                                   |
| ALL     | 0x7   | Read, Write, Execute を指定する。                  |

|          |   |   |         |       |
|----------|---|---|---------|-------|
| 7        | 3 | 2 | 1       | 0     |
| RESERVED |   |   | EXECUTE | WRITE |
|          |   |   |         | READ  |

Figure 12: MAP の Memory Rights Bit 配置

Cache 属性, Global 属性, Memory Type を指定する Field は共通インターフェースに存在しない。未定義の Attribute Bit は明示的に拒否されないが, Mapping へ反映されない。ユーザ空間は 0x0..0x7 だけを使用する必要がある。

## 7.3 Address Space Operations

| Method  | Message Register                                                                      | Description                                                           | Error                                                                                          |
|---------|---------------------------------------------------------------------------------------|-----------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| MAP = 1 | MR2 = Page Table or Frame descriptor<br>MR3 = user virtual address<br>MR4 = attribute | Page Table または Frame を Address Space へ Map する。対象 Entry は未使用でなければならない。 | INVALID_DESCRIPTOR,<br>INVALID_ARGUMENT,<br>ILLEGAL_OPERATION,<br>PERMISSION_DENIED,<br>FATAL. |

| Method    | Message Register                                                   | Description                                  | Error                                                                                          |
|-----------|--------------------------------------------------------------------|----------------------------------------------|------------------------------------------------------------------------------------------------|
| UNMAP = 2 | MR2 = Page Table or Frame descriptor<br>MR3 = user virtual address | Page Table または Frame に対応する Entry を Clear する。 | INVALID_DESCRIPTOR,<br>INVALID_ARGUMENT,<br>ILLEGAL_OPERATION,<br>PERMISSION_DENIED,<br>FATAL. |

| Method              | Message Register                                   | Description                                                          | Error                                                                     |
|---------------------|----------------------------------------------------|----------------------------------------------------------------------|---------------------------------------------------------------------------|
| GET_UNSET_DEPTH = 3 | MR2 = user virtual address<br>MR3 = leaf size bits | 指定 Leaf を Map するために最初に不足する Page Table Depth を MR2 へ返す。不足がなければ 0 を返す。 | INVALID_DESCRIPTOR,<br>ILLEGAL_OPERATION,<br>PERMISSION_DENIED,<br>FATAL. |



| Method                      | Message Register      | Description                                       | Error                         |
|-----------------------------|-----------------------|---------------------------------------------------|-------------------------------|
| CAN_MAP_FRAME_SIZE_BITS = 4 | MR2 = frame size bits | HAL が Frame Size を受取する場合<br>は成功を返す。追加 Result はない。 | INVALID_DESCRIPTOR,<br>FATAL. |

## ■ 7.4 MAP

Page Table Mapping は、対象 Page Table の Depth から格納先の Entry Depth を決める。受取する Depth と Address は HAL が検査する。

Frame Mapping は、Frame Size Bits から Leaf Depth を決める。必要な中間 Page Table が存在しない場合、カーネルは INVALID\_DESCRIPTOR を返す。Size Bits と Leaf Depth の対応はアーキテクチャごとの ABI が定める。

対象 Entry が Present の場合、カーネルは ILLEGAL\_OPERATION を返す。既存の Mapping を置き換える操作はない。Mapping を変更する場合は、UNMAP が成功した後で MAP を行う必要がある。

Virtual Address と Physical Address の Alignment は Address Space Call で明示検査されない。ユーザ空間は、HAL が定める Mapping Size に合わせて両方の Address を Alignment する必要がある。

## ■ 7.5 UNMAP and TLB

Page Table の UNMAP は、対象 Entry が Not Present でも成功を返す。Frame の UNMAP は、同じ状態で ALREADY\_MAPPED 内部 Error を返し、Capability Error は ILLEGAL\_OPERATION となる。Page Table と Frame では、再実行時の結果が異なる。

UNMAP は、対象 Entry の Physical Address と、MR2 の Descriptor によって探索した Slot が参照する Page Table または Frame の Physical Address を比較しない。対象 Capability は Unmap する Depth を決めるためだけに使う。同じ Depth または Size を持つ別 Object を参照する Capability Slot の Descriptor でも、指定 Virtual Address の Entry を Clear できる。Address Space Capability は、Address Space 内の全 Mapping を変更できる権限として扱う必要がある。

Mapping 対象 Address Space が実行中 Process の Root Address Space と同一である場合、HAL は対象 Virtual Address の TLB Entry を無効化する。別の Address Space に対する即時無効化の要否は HAL が定める。具体的な命令と Address Space 切替え時の処理はアーキテクチャごとの ABI に記載する。

## ■ 7.6 GET\_UNSET\_DEPTH

GET\_UNSET\_DEPTH は、Root から Leaf の直前まで Page Table をたどる。最初の Not Present Entry が必要とする Child Depth を返す。途中で Huge Page Entry がある場合は ILLEGAL\_OPERATION となる。

ユーザ空間は、返された Depth の Page Table を Generic から作成し、指定 Address へ MAP した後で GET\_UNSET\_DEPTH を繰り返す。Result 0 は、指定 Frame Size に必要な中間 Page Table がすべて存在することを表す。

## ■ 7.7 Page Table Capability

Page Table Capability の `execute()` を直接呼ぶと、すべての操作で `ILLEGAL_OPERATION` となる。Page Table の作成時、カーネルは Page Table Memory を Zero Clear する。Page Table Depth は Slot-local Data へ保存される。

Page Table の `revoke()` は空実装である。Page Table Capability を Remove しても、Address Space 内の Entry は残る。関連する Mapping を先に `UNMAP` する必要がある。

## ■ 7.8 Frame Capability

Frame Slot Data は Physical Address, Flags, Size Bits を保持する。Generic から作成する Frame の Flags は 0 である。Address Space Mapping Attribute は Frame Slot Flags ではなく MAP の MR4 から取得する。

Header は `GET_ADDRESS = 1` と MR2 Result を宣言する。Frame Capability の `execute()` は Operation Number を Decode せず、常に `DEBUG_UNIMPLEMENTED` を返すため、Physical Address を取得する公開 API としては使用できない。

Frame の `revoke()` も空実装である。Frame Capability を Remove しても、Mapping は残り、Frame Memory も Zero Clear されない。Frame を再割当てする前に、全 Address Space から Unmap し、必要な Data をユーザ空間で消去する必要がある。

### WARNING

Page Table や Frame Capability を Remove しても、Hardware Page Table Entry は残り得る。Mapping を残したまま Generic を Revoke したり Memory を再利用したりすると、古い Address Space から別 Object の Memory へ到達できる。Process 停止、Mapping の Unmap、Capability の Remove、Memory の再利用の順に破棄する必要がある。

## ■ 7.9 Concurrency

Page Table 更新に対する Address Space 単位の Lock と Remote TLB Shutdown は、対象 Revision に存在しない。同じ Address Space への Mapping 変更は、ユーザ空間で直列化する必要がある。Single-core 以外の実行では、別 Core の TLB 整合性が保証されない。

## 8

## Process Control Block and Scheduler

### ■ 8.1 Object Model

Process Control Block (PCB) は、ユーザ空間の実行文脈を表す Capability Object である。PCB は Hardware Context, Floating-point Context, Process Status, Priority, Quantum, CPU Affinity, Root Capability Node, Root Address Space, IPC Buffer, Notification Port, Fault Resolver Port, IPC Reply State を持つ。

Generic から PCB を作成すると、User Mode 用の Hardware Context と Floating-point Context が初期化される。Address Space, Root Node, IPC Buffer, Notification Port, Fault Resolver, Instruction Pointer, Stack Pointer, Thread-local Base, Priority, CPU Affinity は、CONFIGURE で設定する。Init の PCB だけは、Boot 処理が直接構成する。

PCB に対する操作は、PCB Slot の Rights を検査しない。PCB Capability を保持する Process は、対象 Process の Context, Address Space, Scheduling State を変更できる。PCB Capability は、Process Manager だけへ配布する必要がある。

### ■ 8.2 Operation Summary

| Method            | Message Register                               | Description                                                  | Error                                                                    |
|-------------------|------------------------------------------------|--------------------------------------------------------------|--------------------------------------------------------------------------|
| CONFIGURE = 1     | MR2 = configuration info<br>MR3..MR12 = fields | Bit Mask で選択した PCB Field を順番に設定する。複数 Field の更新は Atomic ではない。 | INVALID_DESCRIPTOR,<br>ILLEGAL_OPERATION,<br>INVALID_ARGUMENT,<br>FATAL. |
| Method            | Message Register                               | Description                                                  | Error                                                                    |
| READ_REGISTER = 2 | MR2 = register count                           | Hardware Context の先頭から register_count Word を MR3..へ返す。       | INVALID_ARGUMENT,<br>FATAL.                                              |

| Method             | Message Register                       | Description                                                                               | Error                       |
|--------------------|----------------------------------------|-------------------------------------------------------------------------------------------|-----------------------------|
| WRITE_REGISTER = 3 | MR2 = register count<br>MR3.. = values | Hardware Context の先頭から register_count Word を書き換える。                                        | INVALID_ARGUMENT,<br>FATAL. |
| Method             | Message Register                       | Description                                                                               | Error                       |
| RESUME = 4         | 追加引数なし。                                | Process を READY へ設定し、Quantum を 10 へ設定して Ready Queue へ追加する。                                | FATAL.                      |
| Method             | Message Register                       | Description                                                                               | Error                       |
| SUSPEND = 5        | 追加引数なし。                                | IPC Queue または Notification Queue から Process を外し、Ready Queue から削除して BLOCKED_SUSPEND へ設定する。 | FATAL.                      |

## 8.3 CONFIGURE

configuration\_info で Bit が 1 になっている Field だけを更新する。カーネルは Bit 0 から Bit 9 の順に処理する。途中で Error が発生しても、更新済みの Field は元に戻らない。

Descriptor を受け取る Field では、呼出し Process の Root Capability Node を起点として、Descriptor によって対象 Capability Slot を探索する。探索した Slot の Type を検査した後、Capability を PCB 内部 Slot へ Copy する。

| Bit | Field               | MR  | Description                                                                                                               |
|-----|---------------------|-----|---------------------------------------------------------------------------------------------------------------------------|
| 0   | address_space       | MR3 | Descriptor によって対象 Slot を探索し、ADDRESS_SPACE Capability を PCB 内部の Root Address Space Slot へ Copy する。                         |
| 1   | root_node           | MR4 | Descriptor によって対象 Slot を探索し、NODE Capability を PCB 内部の Root Slot へ Copy する。                                                |
| 2   | frame_ipc_buffer    | MR5 | Descriptor によって対象 Slot を探索し、FRAME Capability を PCB 内部 Slot へ Copy して、Frame Physical Address から Kernel 側 ipc_buffer*を設定する。 |
| 3   | notification_port   | MR6 | Descriptor によって対象 Slot を探索し、NOTIFICATION_PORT Capability を Process へ Bind して、PCB 内部 Slot へ Copy する。既存 Binding は解除する。      |
| 4   | ipc_port_resolver   | MR7 | Descriptor によって対象 Slot を探索し、IPC_PORT Capability を Fault Resolver として PCB 内部 Slot へ Copy する。                               |
| 5   | instruction_pointer | MR8 | HAL が User Address として受理する値を Instruction Pointer へ設定する。                                                                   |
| 6   | stack_pointer       | MR9 | HAL が User Address として受理する値を Stack Pointer へ設定する。                                                                         |

| Bit | Field             | MR   | Description                                                                                      |
|-----|-------------------|------|--------------------------------------------------------------------------------------------------|
| 7   | thread_local_base | MR10 | HAL が User Address として受理する値を Thread-local Base へ設定する。対応する Hardware Context はアーキテクチャごとの ABI が定める。 |
| 8   | priority          | MR11 | 0 以上 32 未満の Priority を設定する。大きい値ほど高 Priority である。                                                 |
| 9   | affinity          | MR12 | CPU Core Index を保存する。対象 Revision は Range 検査と Scheduler 適用を行わない。                                  |

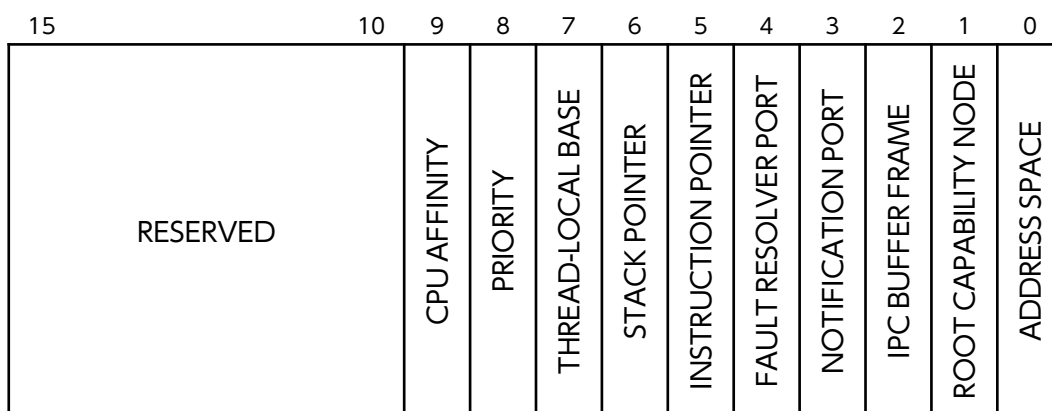


Figure 13: configuration\_info 下位 16 bit の配置

Bit 10 以降はカーネルが解釈しない。ユーザ空間は未使用 Bit を 0 にする必要がある。

CONFIGURE は、Instruction Pointer、Stack Pointer、Thread-local Base に対応する Page が Map 済みかを確認しない。HAL の Address 検査も、数値の範囲だけを確認する。受理される範囲はアーキテクチャごとの ABI に記載する。

#### WARNING

frame\_ipc\_buffer は Frame の物理 Address をカーネルから参照できる Address へ変換する。Frame Size、構造体の配置、ユーザ空間の Mapping は CONFIGURE で検査されない。IPC Buffer には専用 Frame を用い、ipc\_buffer 構造体を Frame 先頭へ配置する必要がある。

## 8.4 Register Access

READ\_REGISTER と WRITE\_REGISTER は、Hardware Context を Word 配列として扱う。register\_count の上限は HARDWARE\_CONTEXT\_SIZE である。配列の長さとは各 Index の意味はアーキテクチャごとの ABI が定める。

**CAUTION**

対象 Revision の `READ_REGISTER` は、Hardware Context の読出しと返却先 MR への書込みを Index 順に行う。実行中 Process が自身の PCB を読む場合、返却先 MR と後続の読出し対象が重なると、返却値は呼出し開始時点の Snapshot にならない。Process Manager は、停止中の別 Process を読出し対象とする必要がある。自己 PCB で Snapshot を維持できる範囲は、アーキテクチャごとの ABI に従う必要がある。重なりが発生した後に元の Context を復元する共通 Operation は存在しない。

`WRITE_REGISTER` は、各 Word が User Mode の Privilege 制約を満たすかを検証しない。不正な Hardware Context は、Context Restore 時の Fault、Privilege 境界の破壊、Kernel Crash を起こし得る。ユーザ空間の Process Manager は、アーキテクチャごとの ABI で書換えを許された Field だけを変更する必要がある。

## ■ 8.5 Process States

| State           | Meaning                                                  |
|-----------------|----------------------------------------------------------|
| UNUSED          | 初期または未使用状態。                                              |
| READY           | Scheduler が実行対象として扱う状態。実行中 Process も READY を使用する。        |
| BLOCKED_SEND    | IPC Port で Receiver を待つ Sender。                          |
| BLOCKED_RECEIVE | IPC Port で Sender または Bind 済み Notification を待つ Receiver。 |
| BLOCKED_REPLY   | CALL の Reply を待つ Client。                                 |
| BLOCKED_WAIT    | Notification Port の WAIT で通知を待つ Process。                 |
| BLOCKED_SUSPEND | PCB の SUSPEND または回復不能 Fault で停止した Process。               |
| BLOCKED_FAULT   | Fault Resolver への配送待ちとなった Process。                       |

**CAUTION**

RESUME は Idempotent ではない。Ready Queue へ所属済みの Process または実行中 Process へ RESUME を再実行すると、重複追加の失敗または Queue Link の破損を起こし得る。User-level Process Manager は PCB ごとの State と Queue 所属を管理し、構成済みかつ Ready Queue へ所属していない Process だけを RESUME する必要がある。

SUSPEND は対象 Process を Wait Queue と Ready Queue から外すが、対象 Process が呼出し元自身である場合に即時 Scheduling を実行しない。Self Suspend を実行した Process は BLOCKED\_SUSPEND へ設定された後も Kernel Call から復帰し得る。実行中 Process を停止する処理は、別の Process Manager から SUSPEND するか、IPC または Notification の Blocking Operation へ移す必要がある。

## ■ 8.6 Benno Scheduling

Benno Scheduling は、実行可能な Process だけを Ready Queue へ保持し、固定 Priority と Priority 内 Round-robin を組み合わせる Scheduling 手法である [8]。対象

Revision の Scheduler は Benno Scheduling を用い、Priority 0 から 31 に対応する 32 本の FIFO Ready Queue を持つ。大きい Priority 値ほど選択順位が高い。

Ready Queue には、`status == READY` の Process だけを格納する。実行中 Process も READY State を持つが、Ready Queue には所属しない。Block された Process は、IPC Port または Notification Port の Wait Queue へ移るか、いずれの Queue にも所属しない。`schedule()` は、空でない最高 Priority Queue の Head を取り出す。`add_process()` は、対象 Priority Queue の Tail へ Process を追加する。同じ Priority の Process は、Quantum 満了ごとに FIFO 順で選択される。

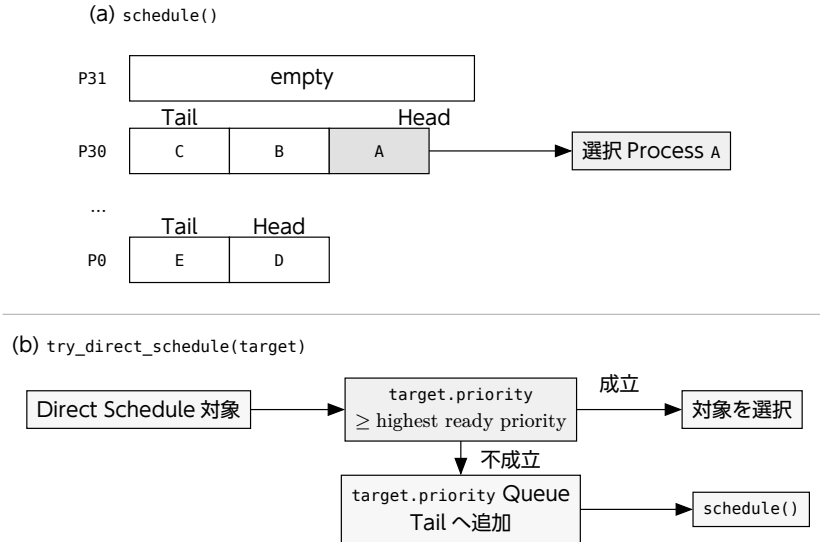


Figure 14: Priority 別 Ready Queue と Direct Schedule の選択規則

固定 Priority 方式は、選択可能な高 Priority Process が存在する間、低 Priority Process を選択しない。対象 Revision は Priority Aging と実行時間 Reservation を実装しないため、ユーザ空間の Process Manager は Priority 設定による Starvation を考慮する必要がある。

## ■ 8.7 Quantum and Preemption

Process Quantum の基準値 `QUANTUM_MAX` は 10 Tick である。RESUME, YIELD, Quantum 満了による再投入は、Quantum を 10 へ設定する。Timer Handler は実行中 Process の Quantum を Tick ごとに 1 減らす。Quantum が 0 以下になると、Process を Ready Queue の Tail へ戻して `try_schedule_and_switch()` を実行する。

Notification によって Process が Ready になった場合、`schedule_if_preempted_by()` は通知先 Priority が実行中 Process の Priority より大きいときだけ Scheduling を実行する。同 Priority の通知先は Ready Queue で待機する。IPC による Direct Schedule は、通常の Priority Preemption とは別の経路である。

YIELD は実行中 Process を READY のまま Ready Queue へ戻し、次の Process を選択する。YIELD は Block State を作らない。同 Priority Queue に別 Process が存在する場合、FIFO 順の次 Process が選択される。

## ■ 8.8 Direct Schedule

A9N の Direct Schedule は、L4 系 Microkernel の Direct Process Switch に由来する IPC Optimization である [8]。try\_direct\_schedule(target) は、IPC の通信相手を通常の Scheduling を介さずに選択する。対象 Process の Priority が Ready Queue 内の最高 Priority 以上なら、Scheduler は対象 Process を Ready Queue から外して選択する。Ready Queue に対象 Process より高い Priority の Process が存在する場合、Scheduler は対象 Process を Ready Queue へ追加し、schedule() で最高 Priority Process を選択する。

try\_direct\_schedule\_and\_switch() は、切替元 Process に残る Quantum を切替先 Process へ加算してから Context Switch を行う。切替先 Quantum は 10 を超え得る。IPC Port は、待機中 Receiver へ CALL を配送する経路と Fault を Resolver へ配送する経路で Direct Schedule を使用する。CALL と REPLY\_RECEIVE を組み合わせる IPC Fastpath は「IPC Port」に記載する。

## ■ 8.9 Scheduling Limitations

CPU Affinity は PCB へ保存されるが、対象 Revision の Scheduler は参照しない。Application Processor Entry も空実装である。Ready Queue と highest\_priority を Core ごとに分離する実装、Remote Preemption、Queue Lock は存在しない。Scheduling は Single-core でのみ有効である。

## ■ 8.10 Revoke and Lifetime

PCB を Revoke すると、Process を IPC および Notification の Wait Queue から外し、Suspend 状態へ移す。続いて、Root Node、Root Address Space、IPC Buffer Frame、Resolver Port、Notification Port の内部 Slot を削除し、Hardware Context と Floating-point Context を初期化し直す。

PCB Object 用 Memory は Generic へ個別返却されない。PCB Capability を Remove しても、同じ PCB を参照する Sibling Slot が残る場合、PCB Revoke は実行されない。Reply State を持つ Process には相手 Process への Pointer が残るため、ユーザ空間は IPC Session を終了してから PCB を破棄する必要がある。



## 9

## IPC Port

## ■ 9.1 Object Model

IPC Port は、Message と Capability を Process 間で転送する Capability Object である。Port は Sender Queue または Receiver Queue を FIFO で持つ。Port State は WAIT, READY\_TO\_SEND, READY\_TO\_RECEIVE のいずれかであり、Sender と Receiver が同時に Queue へ入ることはない。

IPC Port Capability Slot は、Slot-local Identifier を `data[0]` に持つ。同じ IPC Port Object を参照する Slot であっても、Identifier は個別に設定できる。Receiver は、送信に使われた Slot の Identifier を MR3 で受け取る。MR3 は Sender の Payload でなく、Kernel が Slot-local Data から設定する。

## ■ 9.2 Message Layout

全 IPC 操作は、MR2 を `message_info` として読み取る。ユーザ空間から呼び出す場合は、`source = NORMAL` を設定する。別の Source を指定すると、カーネルは `INVALID_ARGUMENT` を返す。

| Bits   | Field          | Description                                            |
|--------|----------------|--------------------------------------------------------|
| 0      | block          | Peer 不在時に Process を Block する場合は 1, Block せずに戻る場合は 0.   |
| 1..8   | message_length | MR4 から始まる Payload Word 数. 範囲は 0 から 255.                |
| 9..12  | transfer_count | 移動する Capability 数. 範囲は 0 から 15.                        |
| 13..14 | source         | NORMAL = 0, FAULT = 1, NOTIFICATION = 2, RESERVED = 3. |
| 15     | reserved       | Kernel は明示検査しない. ユーザ空間は 0 を設定する必要がある.                  |
| 16..63 | 未使用.           | Kernel は解釈しない. ユーザ空間は 0 を設定する必要がある.                    |

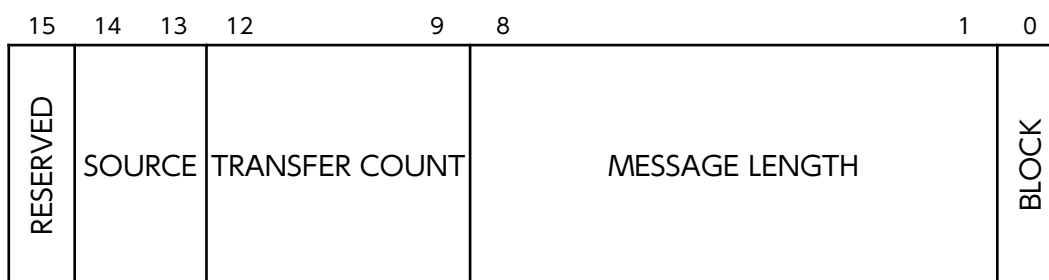


Figure 15: message\_info の Bit 配置

| 項目    | 説明                                                |
|-------|---------------------------------------------------|
| MR0   | IPC Port Capability Descriptor.                   |
| MR1   | Operation Number.                                 |
| MR2   | message_info. IDENTIFY では新しい Identifier と兼用する.    |
| MR3   | 受信時の Source Identifier. 送信時の値は Payload として転送されない. |
| MR4.. | message_length Word の Payload.                    |

Payload は, Sender と Receiver の同じ MR Index 間で転送する. message\_length = n の場合は, MR4 から MR(3+n) までを転送する. Payload がレジスタに割り当てられた MR を超える場合は, IPC Buffer が必要となる.

## 9.3 Operation Summary

| Method   | Message Register              | Description                                                                                                    | Error                                             |
|----------|-------------------------------|----------------------------------------------------------------------------------------------------------------|---------------------------------------------------|
| SEND = 1 | MR2 = info<br>MR4.. = payload | Message を Receiver へ転送する. IPC Port Slot に WRITE を要求する. Receiver 不在かつ block = 1 の場合, Sender は BLOCKED_SEND となる. | PERMISSION_DENIED,<br>INVALID_ARGUMENT,<br>FATAL. |

| Method      | Message Register | Description                                                                                                                               | Error                                             |
|-------------|------------------|-------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------|
| RECEIVE = 2 | MR2 = info       | Sender から Message を受信する. IPC Port Slot に READ を要求する. Sender 不在かつ block = 1 の場合, Receiver は BLOCKED_RECEIVE となる. 既存 Reply Authority を破棄する. | PERMISSION_DENIED,<br>INVALID_ARGUMENT,<br>FATAL. |

| Method   | Message Register              | Description                                                             | Error                                             |
|----------|-------------------------------|-------------------------------------------------------------------------|---------------------------------------------------|
| CALL = 3 | MR2 = info<br>MR4.. = payload | Message を送り, 選択された Receiver からの Reply を待つ. IPC Port Slot に WRITE を要求する. | PERMISSION_DENIED,<br>INVALID_ARGUMENT,<br>FATAL. |

| Method            | Message Register                       | Description                                                                                                  | Error                                             |
|-------------------|----------------------------------------|--------------------------------------------------------------------------------------------------------------|---------------------------------------------------|
| REPLY = 4         | MR2 = info<br>MR4.. =<br>payload       | Process が保持する Reply Authority へ Message を返す。IPC Port Slot Rights を要求しない。Reply Authority がない場合も、状態を変更せず成功を返す。 | INVALID_ARGUMENT,<br>FATAL.                       |
| Method            | Message Register                       | Description                                                                                                  | Error                                             |
| REPLY_RECEIVE = 5 | MR2 = info<br>MR4.. = reply<br>payload | Reply を完了してから同じ Port で次の Message を受信する。IPC Port Slot に READ を要求する。                                           | PERMISSION_DENIED,<br>INVALID_ARGUMENT,<br>FATAL. |
| Method            | Message Register                       | Description                                                                                                  | Error                                             |
| IDENTIFY = 6      | MR2 =<br>identifier                    | 呼出しに使用した IPC Port Slot の Identifier を変更する。IPC Port Slot に MODIFY を要求する。                                      | PERMISSION_DENIED,<br>INVALID_ARGUMENT,<br>FATAL. |

## ■ 9.4 SEND and RECEIVE

Receiver が待機している場合、SEND は Queue 先頭の Receiver へ Message を転送し、Receiver を Ready Queue へ戻す。Sender は実行を続ける。Sender が待機している場合、RECEIVE は Queue 先頭の Sender から Message を受け取り、Sender を Ready Queue へ戻す。

RECEIVE を実行する Process が Reply Authority を持っている場合、カーネルは古い Client との Reply Link を解除する。Reply せずに Client を切り離す場合は RECEIVE を、Reply を完了してから次の Message を待つ場合は REPLY\_RECEIVE を用いる。

Peer が存在せず、block = 0 の場合、操作は成功を返すが Message を転送しない。対象 Revision は、Non-blocking 操作でも Port State を READY\_TO\_SEND または READY\_TO\_RECEIVE へ変更する。Queue が空のまま State だけが変わるため、後から呼ばれた Peer 側の操作が FATAL を返し、State を WAIT へ戻す経路がある。対象 Revision では、Non-blocking IPC を Poll に使用してはならない。

## ■ 9.5 CALL and Reply Authority

Receiver が待機している場合、CALL は Caller を BLOCKED\_REPLY へ移す。カーネルは Caller に source\_reply\_target を、Receiver に destination\_reply\_target を設定する。Reply Authority を使えるのは Receiver だけである。Reply Authority は Process State であり、Capability Slot として Copy または Transfer できない。

REPLY は Reply Authority が指す Client へ Payload と Capability を転送し、Client を Ready Queue へ戻す。Server の Reply State は NONE となる。実装は Client へ直接切り替えず、Client を後続の Scheduling 対象とする。

REPLY\_RECEIVE は、Reply Payload を Client へ転送して Ready Queue へ戻した後、次の Sender から受信する。Server Loop では、最初に RECEIVE を呼び、以後は REPLY\_RECEIVE を繰り返せる。

Peer が存在せず、block = 0 の CALL は Reply 待ち状態へ入らず、成功を返す。Receiver が待機中の場合は、block 値に関係なく Caller が Reply まで Block される。

## ■ 9.6 IPC Fastpath

IPC Fastpath は、Server が REPLY\_RECEIVE を発行して BLOCKED\_RECEIVE となった後、Client の CALL によって Client から Server へ直接切り替える実行経路である。成立条件は、CALL の実行時点で、(1) 対象 Server が Receiver Queue で待機していること、(2) Server より高い Priority の実行可能 Process が Ready Queue に存在しないことである。Direct Process Switch と Reply/Receive 統合は、L4 系 Microkernel で用いられてきた IPC Optimization に由来する [8]。IPC Fastpath は独立した Operation Number を持たない。

Fastpath 成立時は、対象 Server による Request 処理を同じ Priority 以下の Ready Process より先に実行し、通常の schedule() 呼出しと Ready Queue の探索を省略する。短縮されるのは、Client の CALL から Server が Request 処理を開始するまでの Scheduling 経路である。

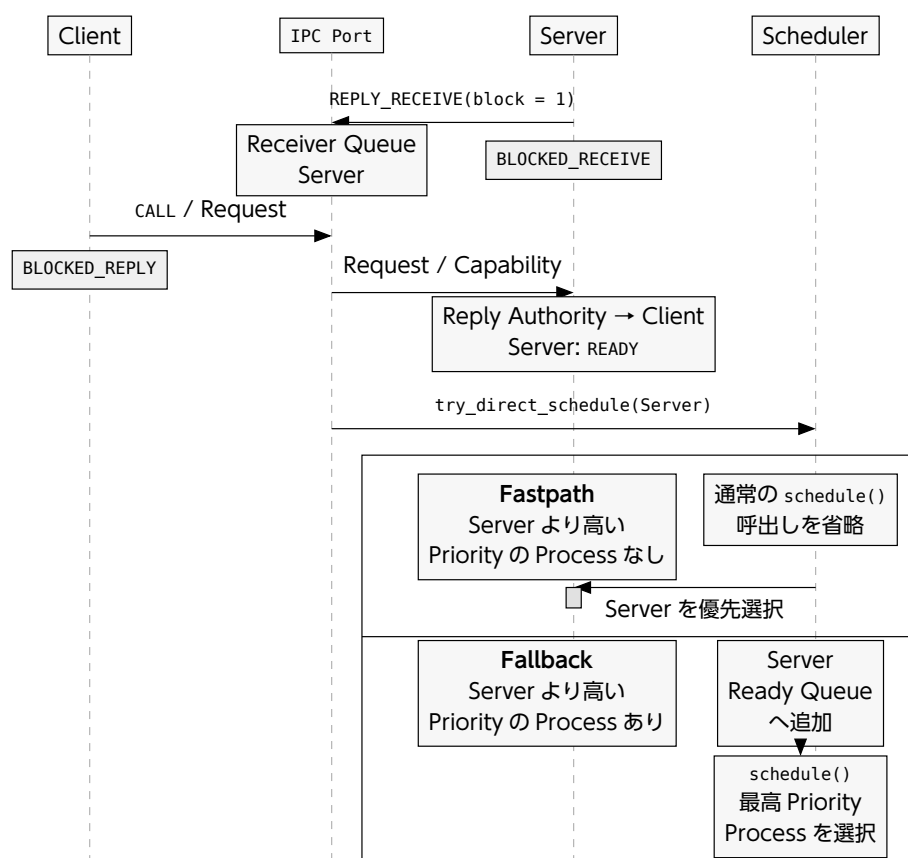


Figure 16: REPLY\_RECEIVE による Server 待機を前提とする CALL Fastpath

Server Loop は最初の Request を RECEIVE で受信し、Reply Authority を得た後は REPLY\_RECEIVE を繰り返す。REPLY\_RECEIVE は旧 Client へ Reply を転送した後、Sender と Pending Notification が存在せず、block = 1 なら Server を BLOCKED\_RECEIVE として Receiver Queue へ追加する。CALL + REPLY\_RECEIVE Fastpath では、Server の待機が Client の CALL より先に発生する。

後続の CALL は Receiver Queue から Server を取り出し、Client を BLOCKED\_REPLY へ設定して、Server との Reply Link を構成する。カーネルは Request の Virtual Message Register と Capability を転送し、転送完了後に Server を READY へ設定する。Server より高い Priority の実行可能 Process が存在しなければ、Direct Schedule によって Client から Server へ切り替える。高い Priority の Process が存在する場合は、通常の Scheduling が Ready Queue 内の最高 Priority Process を選択する。

REPLY\_RECEIVE の発行時点で Sender がすでに待機している場合、カーネルは旧 Client への Reply と Queue 先頭の Sender からの受信を同じ Kernel Call 内で行う。次 Sender が CALL を実行していた場合は、Server の Reply Authority を次 Client へ付け替える。Sender 待機時の Reply/Receive 統合経路では Server が実行を継続するため、旧 Client への Context Switch と Server の再選択は発生しない。

IPC Port に Sender が存在しない場合、REPLY\_RECEIVE は Bind 済み Notification を確認する。Pending Notification も存在せず、block = 1 なら Server は BLOCKED\_RECEIVE となり、Scheduler が次 Process を選択する。block = 0 なら Server は Block せずに戻る。IPC Fastpath は Message Copy と Capability Transfer を省略しない。message\_length = 0 かつ transfer\_count = 0 の場合だけ、Payload Copy と Capability Transfer を省略する。

IPC Operation は Timeout を受け取らない。BLOCKED\_REPLY の Client は Reply が完了するまで自動では Ready にならない。Server または Client を破棄する場合、Process Manager は Reply Link を解消してから PCB を Revoke する必要がある。

## ■ 9.7 Capability Transfer

Capability Transfer は Copy ではなく Move である。Sender は IPC Buffer の transfer\_source\_descriptors[i] に Source Descriptor を設定する。Receiver は transfer\_destination\_node に Destination Node Descriptor を、transfer\_destination\_index に開始 Index を設定する。転送数は Sender が message\_info.transfer\_count へ設定する。

Sender と Receiver の両方に、有効な process.buffer と buffer\_frame.type == FRAME が必要である。Destination Descriptor によって Receiver の Root Node から Destination Node Slot を、Source Descriptor によって Sender の Root Node から Source Slot を探索する。Destination Node Slot の Capability は NODE を指し、Capability の移動先となる Destination Slot は NONE でなければならない。

Transfer は Index 順に行う。途中で Error が起きても、移動済みの Capability は元に戻らない。Source Slot と Destination Node の Rights は検査しない。IPC Port への送信権限を持つ Code は、Sender の Root Capability Node から到達できる

Capability を移動できる。Transfer Metadata を作成する Code を信頼対象に含める必要がある。

**WARNING**

Destination Index と Node Slot の範囲は検査されない。開始 Index を  $i$ 、Node Slot 数を  $n$ 、転送数を  $c$  とすると、Receiver は  $i < n$  かつ  $c \leq n - i$  を Transfer 前に検証する必要がある。条件を満たさない Transfer は Kernel Memory を破壊し得る。

## ■ 9.8 IDENTIFY

IDENTIFY は MR2 を新しい Slot-local Identifier として保存する。IPC Port の共通 Dispatch は MR2 を先に message\_info として検査するため、Identifier の bit 13..14 は 0 でなければならない。任意の Word 値を Identifier として使用することはできない。

SEND または CALL を実行すると、Kernel は呼出しに使用された Slot の Identifier を Process State へ保持する。Receiver が既に待機している場合も、Sender が Queue で待機した後に受信される場合も、Kernel は保持した Identifier を Receiver の MR3 へ書く。Sender が MR3 へ置いた値は Message Payload として転送されないため、Receiver は MR3 を Message の送信に使用された Capability Slot と対応する値として扱える。

Capability の COPY と MINT は Identifier を複製する。IDENTIFY で一方の Slot を変更しても、別 Slot の Identifier は変化しない。Service Manager は、Client ごとに異なる Identifier を設定した IPC Port Capability を配布することで、Server 側の Client 識別、Session 選択、Request Routing、Accounting 等へ利用できる。識別に用いる Slot から MODIFY を除かなければ、Client は Identifier を変更できる。

## ■ 9.9 Concurrency and Revoke

IPC Port Queue と Reply State に対する共通 Lock は存在しない。同じ IPC Port を複数 Core から操作する場合は、ユーザ空間で直列化する必要がある。

IPC Port の revoke() は空実装であり、Wait Queue 内 Process を解除しない。IPC Port を破棄する前に、待機中 Sender、Receiver、Reply 待ち Client を解放する必要がある。PCB の SUSPEND と Revoke は、Process が所属する IPC Wait Queue から Process を外す。

# 10

## Notification Port

### ■ 10.1 Object Model

Notification Port は、Word 幅の Pending Flag と FIFO の Wait Queue を持つ Capability Object である。NOTIFY は、Capability Slot-local Identifier を Pending Flag へ Bitwise OR する。WAIT と POLL は Pending Flag 全体を返し、内部の Flag を 0 に戻す。

同じ Bit へ複数回通知しても、Pending Bit は一つだけ立つ。異なる Bit は一つの Word にまとめられる。Notification Port は、通知回数、通知順序、送信元 Process を記録しない。Identifier 0 では Pending Flag が変化しないため、通知に使う Identifier には 0 以外の Bit Mask を設定する必要がある。

### ■ 10.2 Operation Summary

| Method     | Message Register | Description                                                                                                             | Error                                        |
|------------|------------------|-------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|
| NOTIFY = 1 | 追加引数なし。          | Slot-local Identifier を Pending Flag へ OR する。Notification Port Slot に WRITE を要求する。Operation は呼出し元を Block しない。           | PERMISSION_DENIED, FATAL.                    |
| Method     | Message Register | Description                                                                                                             | Error                                        |
| WAIT = 2   | 追加引数なし。          | Pending Flag が 0 でない場合は MR2 へ返す。Pending Flag が 0 の場合は Process を BLOCKED_WAIT へ設定する。Notification Port Slot に READ を要求する。 | PERMISSION_DENIED, FATAL, ILLEGAL_OPERATION. |

| Method       | Message Register    | Description                                                                                            | Error                                              |
|--------------|---------------------|--------------------------------------------------------------------------------------------------------|----------------------------------------------------|
| POLL = 3     | 追加引数なし.             | Pending Flag を MR2 へ返す.<br>Pending Flag がない場合は MR2 = 0<br>を返す. Notification Port Slot に<br>READ を要求する. | PERMISSION_DENIED,<br>FATAL,<br>ILLEGAL_OPERATION. |
| Method       | Message Register    | Description                                                                                            | Error                                              |
| IDENTIFY = 4 | MR2 =<br>identifier | 呼出しに使用した Slot の Identifier<br>を変更する. Notification Port<br>Slot に MODIFY を要求する.                         | PERMISSION_DENIED,<br>FATAL.                       |

## 10.3 IDENTIFY

IDENTIFY は、MR2 を呼出しに使用した Notification Port Slot の Slot-local Identifier として保存する。IPC Port とは異なり、MR2 を `message_info` として先に検査しないため、Kernel は任意の Word 値を保存する。ただし、Identifier 0 を設定した Slot で NOTIFY しても Pending Flag は変化しない。

NOTIFY は Identifier を引数として受け取らない。Kernel は呼出しに使用された Slot から Identifier を読み、Notification Port の Pending Flag へ Bitwise OR する。WAIT と POLL では、Kernel が蓄積した Flag を MR2 へ書く。Notification Port を Process へ Bind した場合は、IPC 形式の Result として Flag を MR3 へ書く。したがって Receiver は、通知側が Message Register へ自己申告した値ではなく、Kernel が Capability Slot から配送した Flag を受け取る。

Capability の COPY と MINT は Identifier を複製する。同じ Notification Port Object を参照する Slot へ異なる Identifier を設定すれば、各 Identifier の bit を Event Source, IRQ, Device, Wakeup 理由へ割り当てられる。WAIT または POLL で得た Word を Bit Mask として検査することで、一つの Notification Port から複数の発生源を識別できる。同じ bit への複数回の通知は一つにまとめられるため、Identifier を Event 回数として使用することはできない。

Identifier を変更不能な割当てとして扱う場合は、通知側へ Capability を配布する前に Identifier を設定し、配布する Slot から MODIFY を除く必要がある。MODIFY を持つ通知側は、別の Identifier へ変更してから NOTIFY できる。

## 10.4 Delivery and Wakeup

Wait Queue が空の場合、NOTIFY は Pending Flag を残して成功を返す。Process が待っている場合は、Queue 先頭 Process の MR2 へ Pending Flag 全体を書き、内部の Flag を Clear して Process を Ready Queue へ戻す。対象 Process の Priority が実行中 Process より高ければ、カーネルは Scheduling を行う。

WAIT は、最初に Pending Flag を調べる。Flag が立っている場合は Block せずに値を返す。Flag が 0 の場合は、Process を Wait Queue の末尾へ追加する。複数の Process が待っていても、一回の NOTIFY が起こす Process は一つだけである。



POLL は Block しない。MR2 = 0 は通知がないことを表すため、Identifier 0 による通知と区別できない。Identifier 0 を通知に用いてはならない。

*b*: Slot-local Identifier

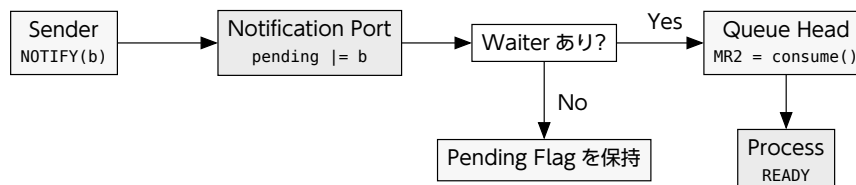


Figure 17: NOTIFY による Pending Flag の蓄積と Waiter の Wakeup

## 10.5 Binding to a Process

PCB の CONFIGURE Bit 3 は、Notification Port を Process へ Bind する。一つの Notification Port に Bind できる Process も一つだけである。同じ Process への再 Bind は成功する。別の Process へ Bind しようとする、Kernel 内部 Error が FATAL へ変換される。

Bind 済み Process が IPC Port で BLOCKED\_RECEIVE になっている間に通知が届くと、Process を IPC Queue から外す。カーネルは IPC 形式の message\_info を MR2 へ設定し、source = NOTIFICATION, message\_length = 0, transfer\_count = 0 とする。MR3 には Consume した Pending Flag を返す。

IPC Port の CALL 待機前と REPLY\_RECEIVE 待機前も、Bind 済み Notification を検査する。Pending Notification が存在する場合、IPC 待機へ入らず Notification 形式の Result を返す。

## 10.6 Interrupt Delivery

Interrupt Port は Notification Port Capability を IRQ Handler Slot へ Copy する。Hardware IRQ 到着時、Kernel は Handler Slot の Identifier を Pending Flag へ OR し、IRQ を Acknowledge してから Mask する。Driver は Notification を処理した後、Interrupt Port の ACK を実行して IRQ を再 Enable する。

同じ IRQ が Mask されるまでに発生した複数 Event は、Notification Bit へ統合され得る。Device Driver は Device Status Register を読んで全 Pending Event を処理する必要がある。Notification の受信回数を Event 数として扱うと、Event を取りこぼす可能性がある。

## 10.7 Revoke and Concurrency

Notification Port の revoke() は空実装であり、Wait Queue と Bind 済み Process を解除しない。PCB Revoke は PCB 側の Binding を解除する。Notification Port を破棄する前に、Waiter を Suspend し、PCB Binding と Interrupt Binding を解除する必要がある。

Pending Flag, Wait Queue, Bind 済み Process Pointer に対する共通 Lock は存在しない。複数 Core からの NOTIFY, WAIT, POLL, Binding 変更は、ユーザ空間で直列化する必要がある。

# 11

## Interrupt

### 11.1 Interrupt Object Model

Interrupt Region は、IRQ 番号を割り当てる権限を表す Capability Object である。Interrupt Region の MAKE\_PORT は、未使用の IRQ に対応する Interrupt Port を一つ作成する。Interrupt Port は IRQ 番号を持ち、Notification Port を配送先として Bind する。

Init は Interrupt Region Capability を Root Node Slot 8 に受け取る。Generic から Interrupt Region と Interrupt Port を作成する経路は存在しない。IRQ 番号の範囲はアーキテクチャごとの ABI が定める。

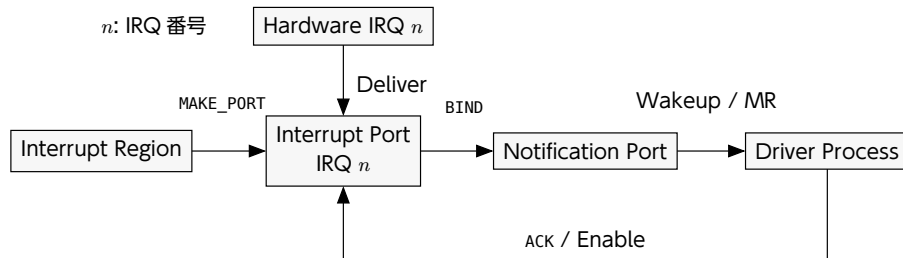


Figure 18: IRQ Capability の作成, Binding, 配送, ACK

### 11.2 Interrupt Region

| Method        | Message Register                                                                 | Description                                                                                                                    | Error                                                                    |
|---------------|----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| MAKE_PORT = 1 | MR2 = IRQ number<br>MR3 = destination node descriptor<br>MR4 = destination index | 未使用 IRQ を予約し、Destination Slot へ Interrupt Port を作成する。<br>MR3 によって探索した Slot の Type は NODE であり、MR4 で選択する子 Slot は NONE でなければならない。 | INVALID_DESCRIPTOR,<br>INVALID_ARGUMENT,<br>ILLEGAL_OPERATION,<br>FATAL. |

MR3 は、呼出し Process の Root Capability Node を起点として Destination Node Slot を探索するための Capability Descriptor である。MAKE\_PORT は、Interrupt

Region Slot と Destination Node の Rights を検査しない。Interrupt Region Capability を保持する Process は、受理される範囲内の任意の IRQ を割り当てられる。

同じ IRQ へ二つ目の Interrupt Port は作成できず、ILLEGAL\_OPERATION となる。Interrupt Port を Revoke すると、IRQ の used Flag が Clear される。Interrupt Region を Revoke すると、全 IRQ の used Flag と Handler Binding Slot が Clear される。

Destination Index の範囲は Node 実装で検査されない。ユーザ空間は、Destination Node の Radix から Index の上限を検証する必要がある。

## ■ 11.3 Interrupt Port

| Method                       | Message Register                   | Description                                                                       | Error                                         |
|------------------------------|------------------------------------|-----------------------------------------------------------------------------------|-----------------------------------------------|
| BIND_NOTIFICATION_PORT = 1   | MR2 = notification port descriptor | Notification Port Capability を IRQ Handler Slot へ Copy する。Handler 未 Bind 状態を要求する。 | INVALID_DESCRIPTOR, ILLEGAL_OPERATION, FATAL. |
| Method                       | Message Register                   | Description                                                                       | Error                                         |
| UNBIND_NOTIFICATION_PORT = 2 | 追加引数なし。                            | IRQ Handler Slot を削除する。Bind されていない場合も成功を返す。                                       | FATAL.                                        |
| Method                       | Message Register                   | Description                                                                       | Error                                         |
| ACK = 3                      | 追加引数なし。                            | IRQ を HAL 経由で Enable する。                                                          | FATAL.                                        |
| Method                       | Message Register                   | Description                                                                       | Error                                         |
| GET_IRQ_NUMBER = 4           | 追加引数なし。                            | MR2 へ IRQ 番号を返す。                                                                  | FATAL.                                        |

Interrupt Port に対する操作は、Interrupt Port Slot の Rights を検査しない。BIND\_NOTIFICATION\_PORT は、Source Notification Port の Type を検査する。IRQ 配送時の NOTIFY には、Copy された Notification Slot の WRITE Right が必要である。

Hardware IRQ が発生すると、カーネルは Bind 済みの Notification Port へ通知する。通知に成功した後、IRQ を Acknowledge して Disable する。Driver は Device Event の処理を終えてから Interrupt Port の ACK を呼び、IRQ を再 Enable する必要がある。

### WARNING

Driver が ACK を実行しない場合、対象 IRQ は Disable 状態に残る。Device Status を Clear する前に ACK を実行すると、同じ Level-triggered IRQ が再配送され続ける可能性がある。Driver は Device 固有の Status 確認と Clear を完了してから ACK を実行する必要がある。

Bind 済み Handler が存在しない IRQ はユーザ空間へ配送されない。Bind 済み Handler がない場合、カーネルは IRQ を Acknowledge または Disable しないため、Interrupt Controller の状態は HAL 実装に依存する。Driver を開始する前に Binding を完了する必要がある。

## ■ 11.4 Revoke and Concurrency

Interrupt Port の Revoke は IRQ 予約と Notification Binding を解除する。

IRQ Handler Table と Notification Binding に対する共通 Lock は存在しない。Interrupt Port の作成と Binding 変更は、Init または単一の Resource Manager で直列化する必要がある。



# 12

## I/O Port

### ■ 12.1 Object Model

I/O Port Capability は、HAL が提供する I/O Space への Read/Write 権限を表す Kernel Object である。I/O Address の意味と実際の Access 方法は HAL が定める。Port-mapped I/O を持つ Architecture だけに限定された Capability ではない。

Capability Slot は、許可する I/O Address の閉区間を data[0] の min と data[1] の max に持つ。Init は Root Node Slot 9 に min = 0, max = UINTMAX\_MAX の Capability を受け取る。Generic から作成する経路はない。Driver 用の範囲は MINT で作成する。

### ■ 12.2 Operation Summary

| Method   | Message Register                         | Description                                                                   | Error                     |
|----------|------------------------------------------|-------------------------------------------------------------------------------|---------------------------|
| READ = 1 | MR2 = source address<br>MR3 = byte width | I/O Address から値を読み、MR2 へ返す。Slot に READ を要求する。受理する Address と Width は HAL が定める。 | PERMISSION_DENIED, FATAL. |

| Method    | Message Register                                            | Description                                                          | Error                     |
|-----------|-------------------------------------------------------------|----------------------------------------------------------------------|---------------------------|
| WRITE = 2 | MR2 = destination address<br>MR3 = byte width<br>MR4 = data | I/O Address へ値を書き込む。Slot に WRITE を要求し、Address が Slot の範囲内であることを要求する。 | PERMISSION_DENIED, FATAL. |

| Method   | Message Register                                                                               | Description                                                              | Error                                             |
|----------|------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|---------------------------------------------------|
| MINT = 3 | MR2 = new min<br>MR3 = new max<br>MR4 = destination node descriptor<br>MR5 = destination index | 呼出しに使用した Capability の範囲に含まれる I/O Port Capability を Destination Slot へ作る。 | INVALID_ARGUMENT,<br>PERMISSION_DENIED,<br>FATAL. |

## 12.3 Address Range

WRITE は、指定した Address が  $\text{min} \leq \text{address} \leq \text{max}$  を満たす場合だけ HAL を呼ぶ。new\_min > new\_max は INVALID\_ARGUMENT となる。MINT で親の範囲外を指定すると PERMISSION\_DENIED となる。MR4 の Capability Descriptor によって、呼出し Process の Root Capability Node から Destination Node Slot を探索する。Destination Node Slot の Type は NODE であり、MR5 で選択する子 Slot は NONE でなければならない。

MINT は、Source Slot の COPY と MODIFY、Destination Node の Rights を検査しない。作成した Slot には ALL を設定する。I/O Port Capability を保持する Process は、自身が持つ範囲内で Rights ALL の子 Capability を作成できる。

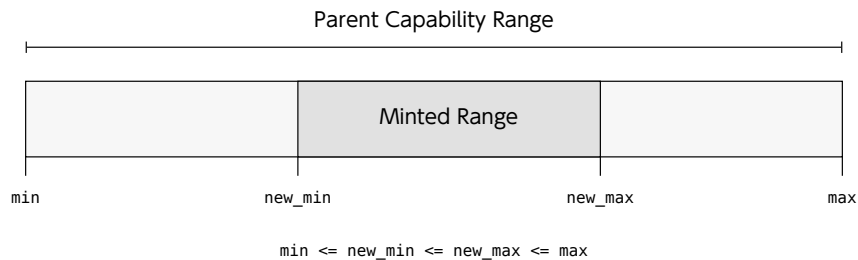


Figure 19: I/O Port Capability の閉区間と MINT による派生 Range

### WARNING

READ は対象 Revision で Address Range を検査しない。READ Right を持つ I/O Port Capability は、Slot-local Data の範囲外も HAL へ渡せる。I/O Access を委譲する場合は、HAL または上位 Service で Read 範囲を検証する必要がある。

## 12.4 HAL Boundary and Lifetime

READ と WRITE は、Address, Byte Width, Data を read\_io\_port() または write\_io\_port() へ渡す。受理する Address 空間, Width, Access 命令, Data の切捨て規則は Architecture ABI が定める。HAL Error は FATAL へ変換される。

I/O Port の revoke() は処理を行わない。派生 Slot の削除は Capability Dependency の処理に従う。Hardware 側の状態を初期化する処理は呼ばれない。



## Part IV System Construction

本 Part は、Boot 後の Init と User-level System を構成する方法を説明する。



# 13

## Boot and Init Protocol

### ■ 13.1 Scope

Init は、カーネルが起動する最初の User Process である。Bootloader は Kernel Image, Init Image, Memory Map, Architecture 情報を準備する。カーネルは `boot_info` を読み、Init の Capability Space, Address Space, Hardware Context を構成する。

対象 Revision の Kernel は、Init Image の ELF Header を解析しない。Bootloader が Image Format を解析し、物理配置と Entry 情報を `boot_info` へ設定する必要がある。Kernel は、`boot_info` の Field 間に矛盾がないかを検査しない。

### ■ 13.2 Boot Flow

Boot から Init 実行までの制御順序は次の通りである。

1. Bootloader が Kernel Image と Init Image を物理メモリへ配置する。
2. Bootloader が `boot_info` を構成し、Pointer を `kernel_entry()` へ渡す。
3. Kernel が CPU-local Data と HAL を初期化する。
4. Kernel が Interrupt Manager と Process Manager を初期化する。
5. Kernel が `create_init()` で Init Process を作成する。
6. Scheduler が Init を選択し、HAL が User Context へ Restore する。

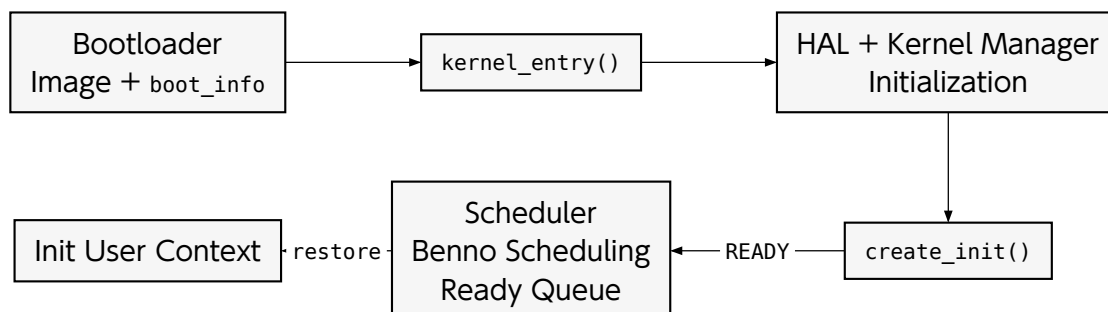


Figure 20: Bootloader から Init User Context までの制御順序

`create_init()` は、Create Phase と Configure Phase に分かれる。Create Phase では、PCB, Root Node, Page Table Node, Frame Node, Generic Node を Kernel 初期 Allocator から作成する。Configure Phase では、`init_info`, I/O Port, Address Space, Frame, Generic, Interrupt Region, Scheduling State を設定する。

Kernel 初期 Allocator は  $\text{PAGE\_SIZE} * 1024$  Byte の Linear Allocator であり、割り当てた領域を解放しない。Init 用の Node 配列と Page Table も同じ領域から割り当て。具体的な Byte 数はアーキテクチャごとの基本 Page Size によって決まる。

## ■ 13.3 Word 単位の Layout

Layout 表は、Structure 先頭からの Offset と Member Size を Word 単位で表す。32-bit Word は 4 Byte, 64-bit Word は 8 Byte である。1 Word より小さい Member と Padding に限り、Word 先頭からの Byte 位置と Byte 数を併記する。Offset と Size の各値には、Word, Byte, または Byte の単位記号 B を必ず付記する。したがって、表を読むために Word 幅から Offset を計算する必要はない。

以下の Layout は、Pointer が 1 Word, `memory_map_type` が 4 Byte, `bool` が 1 Byte である C++ ABI を対象とする。新しい Architecture では、これらの Size と Alignment を Build 時に検証する必要がある。

## ■ 13.4 boot\_info

`boot_info` は `__attribute__((packed))` で宣言される。`memory_info` の Size を 3 Words, `init_image_info` の Size を 5 Words として、Top-level Layout は次の順序となる。

| Field                             | Offset (32-bit) | Size (32-bit) | Offset (64-bit) | Size (64-bit) |
|-----------------------------------|-----------------|---------------|-----------------|---------------|
| <code>boot_memory_info</code>     | Word 0          | 3 Words       | Word 0          | 3 Words       |
| <code>boot_init_image_info</code> | Word 3          | 5 Words       | Word 3          | 5 Words       |
| <code>arch_info[128]</code>       | Word 8          | 128 Words     | Word 8          | 128 Words     |

`boot_memory_info` は Memory Size, Memory Map Entry 数, Memory Map Pointer を保持する。`boot_init_image_info` は Init Image の物理配置と User Address 情報を保持する。`arch_info` は Architecture 固有の Boot 情報を保持する。

`boot_info` 全体の Size は 136 Words である。`packed` 指定は Structure 自体の Alignment を 1 Byte まで下げるため、Bootloader は `boot_info` 先頭を Word 境界へ配置する必要がある。Kernel は `boot_info` Pointer の Alignment を検査しない。

### 13.4.1 memory\_info

boot\_memory\_info の型である memory\_info は、Pointer を Word 境界へ配置する Padding を持つ。

| Field            | Offset (32-bit) | Size (32-bit) | Offset (64-bit) | Size (64-bit) |
|------------------|-----------------|---------------|-----------------|---------------|
| memory_size      | Word 0          | 1 Word        | Word 0          | 1 Word        |
| memory_map_count | Word 1          | 2 Byte        | Word 1          | 2 Byte        |
| Padding          | 1 Word+2 B      | 2 Byte        | 1 Word+2 B      | 6 Byte        |
| memory_map       | Word 2          | 1 Word        | Word 2          | 1 Word        |

memory\_size は Kernel へ通知する Memory Size, memory\_map\_count は Memory Map Entry 数である。memory\_map は memory\_map\_entry Array 先頭を指す。

どちらの Word 幅でも、memory\_info 全体の Size は 3 Words である。memory\_map が指す各 memory\_map\_entry も Word 境界へ配置する。

### 13.4.2 memory\_map\_entry

| Field                  | Offset (32-bit) | Size (32-bit) | Offset (64-bit) | Size (64-bit) |
|------------------------|-----------------|---------------|-----------------|---------------|
| start_physical_address | Word 0          | 1 Word        | Word 0          | 1 Word        |
| page_count             | Word 1          | 1 Word        | Word 1          | 1 Word        |
| type                   | Word 2          | 1 Word        | Word 2          | 4 Byte        |
| Padding                | なし              | なし            | 2 Words+4 B     | 4 Byte        |

start\_physical\_address は Entry 先頭の Physical Address, page\_count は Entry が含む基本 Page 数である。type は FREE, DEVICE, RESERVED を表す。

どちらの Word 幅でも、memory\_map\_entry 全体の Size は 3 Words である。memory\_map\_type は明示的な Underlying Type を持たないため、新しい Toolchain では 4 Byte であることと数値表現を確認する必要がある。

### 13.4.3 init\_image\_info

boot\_init\_image\_info の型である init\_image\_info は、5 個の Word-sized Member から成る。

| Field                   | Offset (32-bit) | Size (32-bit) | Offset (64-bit) | Size (64-bit) |
|-------------------------|-----------------|---------------|-----------------|---------------|
| loaded_address          | Word 0          | 1 Word        | Word 0          | 1 Word        |
| init_image_size         | Word 1          | 1 Word        | Word 1          | 1 Word        |
| entry_point_address     | Word 2          | 1 Word        | Word 2          | 1 Word        |
| init_info_address       | Word 3          | 1 Word        | Word 3          | 1 Word        |
| init_ipc_buffer_address | Word 4          | 1 Word        | Word 4          | 1 Word        |

loaded\_address は Init Image 先頭の Physical Address, init\_image\_size は割り当てた初期 Frame 数, entry\_point\_address は Init の User Entry Point である。init\_info\_address と init\_ipc\_buffer\_address は、それぞれ Image 先頭から対象領域までの Offset である。

`loaded_address` は初期 Frame Size に合わせて配置する。初期 Frame Size を  $2^{\text{INITIAL\_FRAME\_SIZE\_BITS}}$  Byte とする。Kernel は Init Image の Frame  $i$  を、Physical Address `loaded_address + 2^{\text{INITIAL\_FRAME\_SIZE\_BITS}} * i` から Virtual Address  $2^{\text{INITIAL\_FRAME\_SIZE\_BITS}} * i$  へ Map する。Init Image の User Virtual Base は 0 である。`entry_point_address`, `init_info_address`, `init_ipc_buffer_address` は 0 起点 Mapping と一致する必要がある。Kernel は `loaded_address + init_info_address` へ起動情報を書き込み、`init_ipc_buffer_address` を Init 側の Virtual Address としても使用する。

**WARNING**

`init_image_size` は名称に反して Frame 数である。Byte 数を設定すると、Kernel は過大な Frame 数を Map し、意図しない物理メモリを Init へ公開する。Bootloader は `ceil(image_bytes / initial_frame_size)` を設定し、配置済み範囲と一致させる必要がある。

## ■ 13.5 init\_info

Kernel は、Init Image 内の予約領域へ `init_info` を書き込む。`init_info` 全体は一つの基本 Page へ収まる。Init Entry の引数 Register に `init_info*` を渡す規約はない。Init は、Link 時に確定した User Virtual Address から `init_info` を取得する。

### 13.5.1 generic\_descriptor

`generic_descriptor` は 2 Words である。`size_radix` と `is_device` は Word 1 の先頭に並び、残りは Padding となる。

| Field                   | Offset (32-bit) | Size (32-bit) | Offset (64-bit) | Size (64-bit) |
|-------------------------|-----------------|---------------|-----------------|---------------|
| <code>address</code>    | Word 0          | 1 Word        | Word 0          | 1 Word        |
| <code>size_radix</code> | Word 1          | 1 Byte        | Word 1          | 1 Byte        |
| <code>is_device</code>  | 1 Word+1 B      | 1 Byte        | 1 Word+1 B      | 1 Byte        |
| Padding                 | 1 Word+2 B      | 2 Byte        | 1 Word+2 B      | 6 Byte        |

### 13.5.2 init\_info Layout

| Field                                   | Offset (32-bit) | Size (32-bit) | Offset (64-bit) | Size (64-bit) |
|-----------------------------------------|-----------------|---------------|-----------------|---------------|
| <code>kernel_major_version</code>       | Word 0          | 1 Word        | Word 0          | 1 Word        |
| <code>kernel_minor_version</code>       | Word 1          | 1 Word        | Word 1          | 1 Word        |
| <code>kernel_patch_version</code>       | Word 2          | 1 Word        | Word 2          | 1 Word        |
| <code>kernel_pre_release[32]</code>     | Word 3          | 8 Words       | Word 3          | 4 Words       |
| <code>kernel_build_meta_data[32]</code> | Word 11         | 8 Words       | Word 7          | 4 Words       |
| <code>arch_info[128]</code>             | Word 19         | 128 Words     | Word 11         | 128 Words     |
| <code>ipc_buffer</code>                 | Word 147        | 1 Word        | Word 139        | 1 Word        |
| <code>generic_list[128]</code>          | Word 148        | 256 Words     | Word 140        | 256 Words     |
| <code>generic_list_count</code>         | Word 404        | 1 Word        | Word 396        | 1 Word        |

Version Field は Kernel Version を、kernel\_pre\_release と kernel\_build\_meta\_data は Version 文字列を保持する。arch\_info は boot\_info.arch\_info の Copy である。ipc\_buffer は init\_ipc\_buffer\_address と同じ User Virtual Address を保持する。generic\_list は Memory Map から生成した Descriptor であり、generic\_list\_count は有効な Element 数を表す。

init\_info 全体の Size は、32-bit Word では 405 Words、64-bit Word では 397 Words である。

Generic Descriptor が表す範囲は  $[\text{address}, \text{address} + 2^{\text{size\_radix}})$  である。generic\_list\_count は、有効な generic\_list Element 数を表す。実装上の Count 制約は「Generic Descriptor Generation」に記載する。

## 13.6 Initial Capability Space

Init Root Node は 256 Slot を持つ。Kernel が構成する Root Slot は次の通りである。

| Slot | Name                     | Initial Capability                                                  |
|------|--------------------------|---------------------------------------------------------------------|
| 0    | RESERVED                 | NONE.                                                               |
| 1    | PROCESS_CONTROL_BLOCK    | Init 自身の PCB Capability.                                            |
| 2    | PROCESS_ADDRESS_SPACE    | Init の Root Address Space Capability.                               |
| 3    | PROCESS_ROOT_NODE        | Init Root Node 自身への再帰参照.                                            |
| 4    | PROCESS_PAGE_TABLE_NODE  | 128 Slot の Node. Init 用 Page Table Capability を格納する.                |
| 5    | PROCESS_FRAME_NODE       | 32768 Slot の Node. Init Image Frame Capability を格納する.               |
| 6    | PROCESS_IPC_BUFFER_FRAME | Init IPC Buffer を含む Frame Capability.                               |
| 7    | GENERIC_NODE             | 128 Slot の Node. 初期 Generic Capability を格納する.                       |
| 8    | INTERRUPT_REGION         | 全 IRQ から Interrupt Port を作成できる Interrupt Region Capability.         |
| 9    | I/O_PORT                 | 全 I/O Address Range を持つ I/O Port Capability. Driver 用の範囲は MINT で作る. |

Root Slot 4, 5, 7 は Node Capability である。Root Slot 6 の Frame は Root Slot 5 配下にある同じ IPC Buffer Frame の Sibling である。PCB 内部の buffer\_frame も同じ Frame を参照する。

## 13.7 Address Space and Hardware Context

Kernel は新しい Address Space を作成し、Init Image の Frame を Virtual Address 0 から順に Map する。作成した中間 Page Table は Root Slot 4 配下へ、Init Image の Frame は Root Slot 5 配下へ格納する。

Kernel 共通の Boot 処理は Instruction Pointer を entry\_point\_address へ設定するが、Stack Pointer を明示的に設定しない。初期 Stack Pointer は HAL が作成する User Hardware Context に依存する。Init は、C または C++ の Prologue を実行する

前に、Assembly Entry Stub または HAL の初期 Context で有効な Stack Pointer を確立する必要がある。Stack 用 Memory は Init Image に含め、対応する Frame 数を `init_image_size` へ加える。

Init の Priority は 31, Quantum は 10 である。Kernel は Init を Ready Queue へ追加し、最初の User Process として実行する。CPU Affinity は Boot 経路で明示設定されない。

IPC Buffer Frame は、`loaded_address + init_ipc_buffer_address` と一致する Init Frame から検出される。Offset が Frame 先頭と一致しない場合、Kernel は PCB の `process.buffer` と Root Slot 6 を構成しない。Init Image は IPC Buffer を初期 Frame の境界へ配置する必要がある。

中間 Page Table を準備する Loop は、Virtual Address 0 から `init_image_size * 2^INITIAL_FRAME_SIZE_BITS` までを終端値を含めて走査する。終端値は最後の Image Frame の直後を指すため、Mapping される Frame 数とは別に、直後の Address に対応する中間 Page Table が追加で作成される場合がある。

## ■ 13.8 Generic Descriptor Generation

Kernel は、RESERVED 以外の Memory Map Entry を Generic へ変換する。FREE Entry からは Device Flag が 0 の Generic を、DEVICE Entry からは Device Generic を作る。2 の冪でない Entry は、先頭から格納できる最大の 2 の冪へ分割し、残りを最大 7 回まで追加分割する。

Memory Map Entry 先頭は `PAGE_SIZE` に合わせた Alignment が必要である。`memory_map_count > 128` は Boot Failure となる。対象 Revision は分割後の総 Descriptor 数を 128 以下へ制限しない。分割後 Descriptor 総数が 129 以上になる Memory Map は、`generic_list` 外への書き込みを起こし得る。Bootloader は分割後 Descriptor 総数を 128 以下へ制限する必要がある。

対象 Revision は、各 Memory Map Entry の最初の Descriptor を作った直後に `generic_list_count` を更新する。追加分割後には更新しないため、最後の非 Reserved Entry から作られた追加 Descriptor は Count に含まれない。Kernel は Count に含まれる Descriptor だけを Generic Node へ設定する。最後の非 Reserved Entry が 2 の冪 Size でない場合、追加 Descriptor が Generic Node へ反映されない。Bootloader は、最後の非 Reserved Entry を 2 の冪 Size にする必要がある。

## ■ 13.9 Init Startup Requirements

Init Entry Stub は、Kernel Call 開始前に次の状態を確立する必要がある。

1. C または C++ の Prologue が要求する Stack Pointer を、Init Image 内の Map 済み Stack 上端へ設定する。
2. Link 時に確定した Address から `init_info` を取得する。
3. `init_info.ipc_buffer` から IPC Buffer Pointer を取得する。
4. `generic_list_count` と Generic Node Slot を照合して Resource 台帳を作成する。



5. Root Slot 7 の Generic から、必要な Node, PCB, Address Space, Page Table, Frame, IPC Port, Notification Port を作成する。

Kernel Call の成否は、MR0 の Flag と MR1 の Capability Error で判定する。レジスタへ収まらない Message Register を使う操作は、Init IPC Buffer が構成済みであることを前提とする。

## ■ 13.10 Boot Failure Model

不正な boot\_info Pointer, 0 Address, Alignment の不一致, Page Table 作成の失敗, Init Allocator の不足, Image Frame Mapping の失敗, Generic Descriptor の不整合は、Init 作成の失敗や Kernel 停止につながる。Bootloader へ Error を返す経路はない。Bootloader は、Kernel へ制御を渡す前に全 Field と物理範囲を検証する必要がある。



# 14

## User-level System Architecture

### ■ 14.1 Scope

User-level Software は、A9N の User Mode で動作し、Kernel Call によって Kernel Object を操作する Software である。A9N Kernel は、Executable Loader, File System, POSIX Process API を提供しない。Boot 時に最初の User-level Software として Init Image を一つ起動し、後続の Process と System Service を構成する責務を Init へ委譲する。

本章は、アーキテクチャに依存しない User-level System の構成要素、責務、Authority, Resource Lifetime を説明する。Init と Service を実装する手順は「Building Init and Services」に、Kernel Call Entry と Register 配置は「x86\_64 ABI」に記載する。

### ■ 14.2 Development Model

Init Image は、Application の Entry Point だけでは成立しない。Language Runtime へ入る前の初期化、init\_info の取得、Kernel Call への変換、Capability 管理、Boot 可能な Image への Link が必要である。各層の責務は次の通りである。

| 層                   | 説明                                                                                 |
|---------------------|------------------------------------------------------------------------------------|
| Application         | Resource Policy, Driver, Memory Manager, Protocol 処理を実装する。                         |
| Runtime             | Stack, 静的領域, Panic 処理, 必要な Language 機能を初期化する。                                      |
| Kernel Call Adapter | Operation 固有の値を Virtual Message Register へ配置し、Kernel Call Entry を呼び出し、結果を型付きの値へ戻す。 |
| Image               | Entry Point, init_info 領域, IPC Buffer, Stack, Load 可能な Program 領域を保持する。            |
| Boot                | Kernel Image と Init Image を配置し、boot_info を構成して Kernel へ制御を渡す。                      |

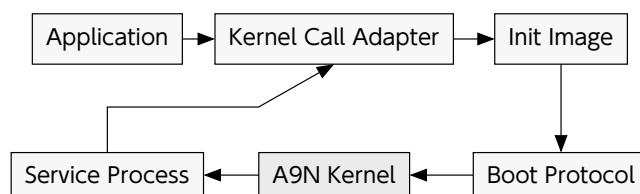


Figure 21: Source から Service Process までの開発境界

Kernel Call Adapter は、Architecture ABI だけを局所化する。Application は Hardware Register を直接扱わず、Capability Descriptor, Operation, Message Field を指定する。新しい Architecture へ移植する場合は、Kernel Call Entry と Virtual Message Register の保存先を差し替え、Object Operation の呼出し側を共有する。

## ■ 14.3 Implementation Units

User-level Software の Source は、Kernel Interface と Application Policy を分離する。最小構成は次の Unit から成る。

| Unit                 | 入力と出力                                                | 責務                                                            |
|----------------------|------------------------------------------------------|---------------------------------------------------------------|
| Entry                | <code>init_info</code> Pointer から Runtime Entry へ移る。 | Stack, 静的領域, IPC Buffer, Language Runtime を初期化する。             |
| Kernel Call Adapter  | 型付き引数から MR 列を作り, Result を返す。                         | CAPABILITY_CALL と YIELD の Architecture Entry を局所化する。          |
| Capability Inventory | Descriptor, Type, Rights, Owner を記録する。               | Capability Space の割当て, 委譲, 失効を追跡する。                           |
| Object Builder       | Generic と Destination Slot から Object 群を作る。           | Address Space, PCB, IPC Port 等の作成順と Rollback を管理する。           |
| Protocol             | Message Layout と Service Error を定義する。                | IPC Request, Reply, Notification, Capability Transfer を符号化する。 |
| Policy               | Kernel Object と Protocol を利用する。                      | Memory Manager, Driver, Process Manager, File System 等を実装する。  |

Entry と Kernel Call Adapter だけが Architecture 固有コードを含む。Capability Inventory, Object Builder, Protocol, Policy は、Architecture 固有の Address Width や Page Attribute を Parameter として受け取り、共通の Object Operation を利用する。

## ■ 14.4 Software Roles

User-level System は、Init, System Service, Client の 3 種類に分けて設計できる。A9N Kernel は Role を識別しない。各 Process が保持する Capability と、IPC Protocol が Role を決める。

| Role           | 保持する Resource                                                 | 責務                                                                 |
|----------------|---------------------------------------------------------------|--------------------------------------------------------------------|
| Init           | Initial Capability Space, Generic, Interrupt Region, I/O Port | Kernel から Resource を受け取り, Object を作成し, Capability を Service へ配布する. |
| System Service | Service 用 PCB, Address Space, Root Node, IPC Port             | Memory, Driver, Process, File System 等の Policy を実装し, IPC で機能を提供する. |
| Client         | Service IPC Port と Client 自身の Memory Resource                 | Request を構成し, Reply または Notification を処理する.                        |

Init は, 全 Resource を自身で使い続ける Process ではない. System 構成に必要な Capability を作成し, MINT で Rights を制限して Service へ配布する. Client には Service の IPC Port を渡し, Device Generic, Interrupt Region, PCB 等の管理 Capability を直接渡さない構成を基本とする.

## ■ 14.5 Resource Lifetime and Concurrency

Capability Object と Wait Queue には, User-level System 全体を覆う Transaction や共通 Lock が存在しない. 同じ Node, Generic, IPC Port, Notification Port, PCB を複数 Process または複数 Core から操作する場合, Resource Manager は Object 単位で直列化する. Operation の完了後に台帳を更新し, 台帳更新前の Object を別の Worker へ渡さない.

Service を停止する際の Resource 依存関係は, 受付停止, Waiter の解消, Process 停止, Mapping 解除, Capability 失効, Generic Revoke の順に解消する. 具体的な終了手順は「Building Init and Services」に記載する.

IPC Port の Revoke は Waiter を自動的に起こさず, Generic の Revoke は派生 Object Memory を Clear しない. 破棄順序は Kernel Call の成功だけでなく, User-level 台帳と各 Process の State を用いて確認する.



# 15

## Building Init and Services

### ■ 15.1 Scope and Preconditions

本章は、Init Image を Kernel Entry から起動し、Capability を管理し、最初の Service Process を構成する手順を示す。読者は「Capability」、「Kernel Call」、Part III の使用対象 Object、「Boot and Init Protocol」、「User-level System Architecture」を先に確認する必要がある。

手順は Architecture に依存しない Kernel Interface を用いる。Entry Stub、Register 配置、Linker Script、Bootloader の具体値は Architecture ABI が定める。x86\_64 では「x86\_64 ABI」を参照する。

### ■ 15.2 Init Image Requirements

「Boot and Init Protocol」は Kernel が Init Image へ要求する境界条件を定義する。Init Image の実装では、少なくとも次の条件を満たす必要がある。

1. Bootloader が読める Executable Image であり、Entry Point を持つ。
2. `init_info` を書き込む領域と IPC Buffer 領域を Image 内に確保する。
3. IPC Buffer 領域を基本 Page の境界へ配置する。
4. Language が生成する Function Prologue より前に、有効な Stack Pointer を設定する。
5. Link 時に確定した Address から `init_info` を取得する。
6. Entry Point から戻らず、待機時は YIELD、IPC 受信、Notification 待機のいずれかを用いる。
7. Kernel Call の返却値を読み、Capability Error を呼出し元へ返す。

`init_info` の Pointer を Entry Point 引数として受け取る共通規約はない。Init Image の Entry Stub が Linker Symbol を参照し、Language 側の Entry Point へ Pointer を渡す。Stack と `init_info` の具体的な Address、Alignment、Register は Architecture ABI に従う。

## ■ 15.3 Kernel Call Adapter

Capability Operation の共通呼出し手順は次のように表せる。enter\_kernel だけが Architecture に依存する。mr は Virtual Message Register を表す。

```
capability_call(descriptor, operation, input):
    mr[0] = descriptor
    mr[1] = operation
    mr[2..] = input
    enter_kernel(CAPABILITY_CALL, mr)

    if mr[0] == 0:
        return error(mr[1])
    return operation_specific_output(mr)
```

成功時に共通で定義される値は MR0 = 1 だけである。MR1 は成功時に初期化されず、呼出し前の Operation Number が残り得る。Adapter は、MR0 = 0 の場合だけ MR1 を capability\_error として読む。MR2.. は Operation が出力を定義する場合だけ読む。

不正な Descriptor, Operation, Rights, Argument は Kernel が Capability Error として返す。Adapter に事前検査は要求されない。型付き API で早期に Error を分類する場合は、次の値を Kernel Call 前に検査できる。

| 入力             | 事前に判定できる内容                                   | Kernel Result                            |
|----------------|----------------------------------------------|------------------------------------------|
| Descriptor     | Encoded Depth と呼出し側台帳の Slot 情報。              | INVALID_DESCRIPTOR または INVALID_DEPTH.    |
| Operation      | 台帳上の Capability Type と要求 Rights.             | ILLEGAL_OPERATION または PERMISSION_DENIED. |
| Message Layout | Message Length, Transfer Count, 最大 MR Index. | INVALID_ARGUMENT または Operation 固有 Error. |
| Address        | Page Size, Alignment, 呼出し側が管理する Mapping.     | INVALID_ARGUMENT または Operation 固有 Error. |

Kernel が保持する Capability Slot の状態は、User-level Software から直接参照できない。事前検査後に Capability State が変化する可能性もある。Kernel Call 後の成否は、事前検査の結果に関係なく、返却された MR0 で確定する。

YIELD は Capability Descriptor と Operation Number を取らない。呼出し層は Architecture 固有の Kernel Call Entry へ YIELD の Call Number を渡す。YIELD から復帰した時点では、別 Process が実行された可能性を考慮し、共有 Memory や User-level Resource の状態を再確認する。

## ■ 15.4 Capability Inventory and Error Handling

Kernel は、Process から到達可能な Capability を列挙する Operation を提供しない。User-level Resource Manager は、自身が作成、移動、受信した Capability を台帳で追跡する。Descriptor は Process の Capability Space 内でのみ意味を持つため、別 Process へ数値だけを渡しても同じ Capability を参照できない。



| Field           | 例                             | 更新時点                                           |
|-----------------|-------------------------------|------------------------------------------------|
| Descriptor      | 0x1000...                     | CONVERT, COPY, MINT, IPC 受信の成功後に登録する。          |
| Type and Rights | IPC_PORT, READ   WRITE        | 作成元 Operation と Rights 制限を記録し, DEMOTE 後に更新する。  |
| Owner           | Init, Process Manager, Driver | MOVE と Capability Transfer の成功後に所有者を変更する。      |
| Dependency      | Source Slot, Generic          | COPY と MINT の派生関係, Generic からの Object 作成を記録する。 |
| State           | Reserved, Active, Revoking    | 多段階の作成と破棄中に同じ Slot を再利用しないよう更新する。              |

COPY は Source と Destination の両方を台帳へ残す。MOVE と Capability Transfer は Source を削除し, Destination を登録する。MINT は Source Rights を超えない Rights を持つ派生 Entry を追加する。REMOVE と REVOKE は成功した Slot を台帳から削除する。複数 Slot を変更する Operation が途中で失敗し得る場合, 呼出し側は処理対象を Revoking 等の中間状態へ移し, 別の操作から隔離する。

Capability Error は, 少なくとも次の方針で上位処理へ伝播できる。

| Error                             | User-level 処理                                                             |
|-----------------------------------|---------------------------------------------------------------------------|
| ILLEGAL_OPERATION                 | Capability Type と Operation の組合せ, または Object State を確認する。同じ入力を実条件に再試行しない。 |
| PERMISSION_DENIED                 | 要求 Rights と使用した Slot を確認する。権限を追加せずに同じ Call を再試行しない。                       |
| INVALID_DESCRIPTOR, INVALID_DEPTH | Capability 台帳と Descriptor 生成処理を確認する。失効済み Entry を削除する。                     |
| INVALID_ARGUMENT                  | Operation 固有の Count, Index, Address, Bit Field を確認する。                     |
| FATAL                             | 対象 Process または Service を隔離し, Kernel Log と直前の State 遷移を保存する。               |
| DEBUG_UNIMPLEMENTED               | 対象 Operation を利用不可として扱い, 別の実装経路を選択する。                                     |

## ■ 15.5 Object Construction and Failure Recovery

Generic から複数 Object を作成して Process を起動する処理は, 単一の Kernel Transaction ではない。User-level Resource Manager は, 各 Operation の成功後に Rollback 情報を記録する。作成処理は, 次の段階へ分割できる。

1. Destination Node と未使用 Slot を予約する。
2. Generic の CONVERT で必要な Kernel Object を作成する。
3. Page Table と Frame を Address Space へ Map する。
4. Capability を Child Root Node へ配置し, Rights を制限する。
5. PCB を CONFIGURE し, Entry, Stack, Address Space, Root Node, IPC Buffer, Fault Resolver を設定する。
6. 構成済みの PCB だけを RESUME する。

失敗時は、PCB を RESUME する前なら、作成順の逆順で Mapping と Capability Slot を解除する。PCB を RESUME した後は、対象 Process を停止し、IPC Queue と Notification Queue から外してから Resource を破棄する。Generic から割り当てた個別 Object の Memory は REMOVE だけでは Generic へ戻らない。Generic の Watermark を戻す REVOKE は、全派生 Capability が失効したことを Resource Manager が確認した場合だけ実行する。

**CAUTION**

PCB の CONFIGURE, Capability Transfer, Node の REVOKE は、途中まで状態を変更した後に Error を返し得る。Error を受け取っただけで呼出し前の状態へ戻ったと判断してはならない。

## ■ 15.6 Bootstrap to a Service Process

Init は、受け取った Generic と Initial Capability Space から、後続 Process に必要な Object を構成する。Executable Loader と Resource Manager は User-level Software として実装する。最初の Service Process を起動する順序は次の通りである。

1. `init_info` の Version, `generic_list_count`, IPC Buffer Pointer を検査し、Root Slot 1 から 9 を Resource 台帳へ登録する。
2. Generic を CONVERT し、Child 用 Capability Node, Address Space, Page Table, Frame, PCB, IPC Port を作成する。
3. Child 用 Frame を Init Address Space の一時領域へ Map し、Executable の Program Segment と初期 Data を書き込む。
4. Child Address Space へ Page Table と Frame を Map し、Entry Point, Stack, IPC Buffer の User Virtual Address を確定する。
5. Child Root Node へ必要な Capability を COPY または MINT する。不要な Rights は MINT で除去する。
6. PCB の CONFIGURE で Root Node, Address Space, IPC Buffer Frame, Fault Resolver, Entry Point, Stack Pointer, Priority を設定する。
7. Server を RESUME し、Server が IPC Port で REPLY\_RECEIVE を発行して待機した後に Client を RESUME する。
8. Init Address Space の一時 Mapping と、配布を終えた一時 Capability を解除する。

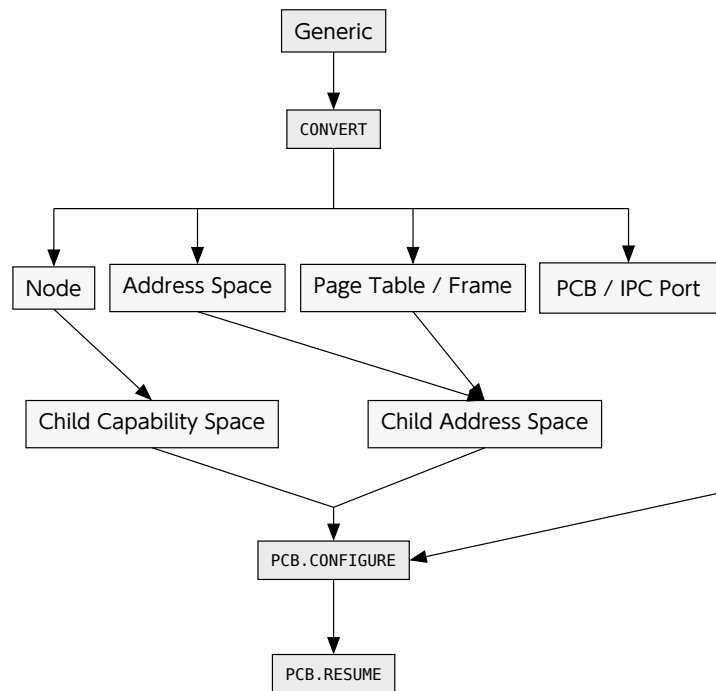


Figure 22: Generic から Service Process を起動するまでの依存関係

PCB を RESUME する時点で、Entry Point、Stack、Address Space、Root Node が相互に整合していなければならない。IPC Buffer を使う Process では、IPC Buffer Frame を Child Address Space へ Map してから PCB へ設定する。Fault Resolver を持たない Process が Fault を起こすと、復帰手段を持たず BLOCKED\_SUSPEND へ遷移する。開発中の Process にも Fault Resolver を設定する必要がある。

## 15.7 IPC Service Implementation

IPC Protocol は、Port Capability、Slot-local Identifier、`message_info`、Payload Layout、Capability Transfer Layout を一組として定義する。Operation Number だけでは Request 種別を表せないため、Service 固有の Method Number を MR4 以降の Payload へ配置する。Server は `message_length` を確認してから、各 Method が要求する Word を読む。

Service Manager は、Client へ IPC Port Capability を配布する前に、Client または Session ごとの Identifier を設定できる。Identifier を設定した Source Slot から、MODIFY を除いた Rights で MINT すれば、Client は Identifier を変更できない。Server は Kernel が MR3 へ配送した Identifier を Client、Session、Role、Request Route 等へ対応付ける。Identifier の値域と再利用規則は Service Protocol が定義する。

Client の CALL は、Request を送信した後に Reply まで Block する。Server Loop は、開始時から REPLY\_RECEIVE を利用できる。Reply Authority を持たない最初の REPLY\_RECEIVE では Reply 部分が状態を変更せず、Receive 部分が最初の Request を待つ。

```

server_loop(port):
    request = reply_receive(port, block = 1)
  
```

```
loop:
    reply = dispatch(request.identifier, request.info, mr[4..])
    mr[4..] = encode(reply)
    request = reply_receive(port, block = 1)
```

dispatch は、Source Identifier, Message Length, Method Number を組み合わせて Request を分類する。Unknown Method や短い Payload には、Service 固有の Error Reply を返す。Kernel が返す Capability Error と、Service Protocol が返す Application Error は別の型として扱う。

IPC Fastpath は、Server が REPLY\_RECEIVE を発行して BLOCKED\_RECEIVE となった後に、Client が CALL した場合に成立し得る。Server は Client の CALL より先に Receiver Queue で待機する。各 Request の処理後は Reply を構成してから次の REPLY\_RECEIVE へ入り、後続 Client を待つ。

Payload の Word 数が Hardware Register へ割り当てられた範囲を超える場合、残りの Virtual Message Register は IPC Buffer に置かれる。同じ Process 内の Kernel Call Adapter と IPC Protocol 実装は、一つの IPC Buffer を共有する。Nested Call や Callback を実装する場合は、外側の Request を別の User Memory へ退避してから Message Register を再利用する。

Capability Transfer を行う Sender は、Source Descriptor 列と transfer\_count を設定する。Receiver は、Destination Node Descriptor と開始 Index を設定してから受信する。Transfer は Move であるため、成功後の Sender は Source Capability を利用できない。Protocol は、移動する Capability Type, Rights, 個数, Destination Slot の所有者を Method ごとに定義する。

## ■ 15.8 Notification and Fault Resolver

Notification Port は、回数ではなく Pending Bit 集合を配送する。Driver Manager は、通知側へ Capability を配布する前に Slot-local Identifier を設定し、一つの Bit を一つの Event Source へ割り当てられる。Driver は、WAIT または POLL で得た Word を Bit ごとに処理する。同じ Bit へ複数回通知しても一つにまとめるため、Event 回数が必要な Protocol は、共有 Memory 上の Counter または Device State を Notification 受信後に読む。

Fault Resolver は、対象 Process の PCB へ設定した IPC Port で Fault Message を受け取る Service である。Resolver は Fault Type, Program Counter, Fault Address, Architecture Fault Code を読み、次のいずれかを選択する。

1. Missing Mapping を構成し、Fault を起こした Process へ Reply する。
2. User Context を規約内の値へ修正して Reply する。
3. 回復不能 Fault として対象 Process を停止し、Resource Manager へ通知する。

Resolver からの Reply は、Fault を起こした Process の Hardware Context を変更し得る。Resolver は一般 Client より強い Authority を持つため、専用 Process へ分離し、Resolver 用 IPC Port を Client へ配布しない。Resolver 自身の Address Space, Stack, IPC Buffer は、Resolver が処理する Process へ依存させない。

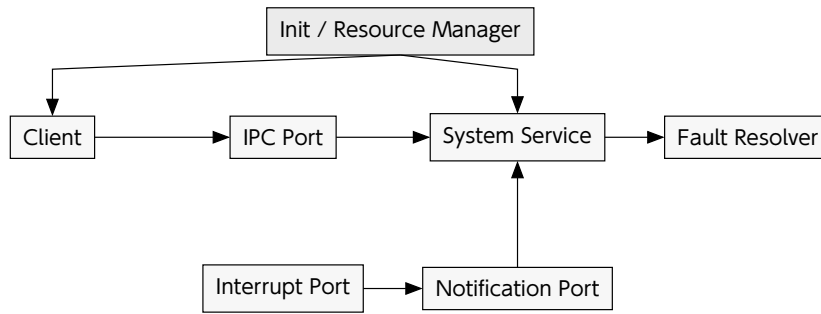


Figure 23: User-level Service を構成する Capability 経路

## 15.9 Service Shutdown

Capability Object と Wait Queue には、User-level System 全体を覆う Transaction や共通 Lock が存在しない。同じ Node, Generic, IPC Port, Notification Port, PCB を複数 Process または複数 Core から操作する場合、Resource Manager は Object 単位で直列化する。Operation の完了後に台帳を更新し、台帳更新前の Object を別の Worker へ渡さない。

Service を停止する場合は、次の順序で Resource を閉じる。

1. 新しい Client へ Service Capability を配布しない。
2. Client Request の受付を停止し、Reply 待ちと Wait Queue を解消する。
3. PCB を停止し、Interrupt Port と Notification Port の Binding を解除する。
4. Child Address Space から Frame と Page Table を Unmap する。
5. Child Root Node 内の Capability を REVOKE または REMOVE する。
6. 全派生 Capability の失効を確認した後に、親 Generic を Revoke する。

IPC Port の Revoke は Waiter を自動的に起こさず、Generic の Revoke は派生 Object Memory を Clear しない。破棄順序は Kernel Call の成功だけでなく、User-level 台帳と各 Process の State を用いて確認する。

## 15.10 Verification

Build 成功だけでは、User-level Software が A9N 上で動作したとは判定しない。検証は、失敗箇所を分離できる順序で進める。

| 段階              | 合格条件                          | 失敗時の確認箇所                                                    |
|-----------------|-------------------------------|-------------------------------------------------------------|
| Image           | Entry と予約領域を機械的に確認できる。        | Linker Script, Load Segment, Symbol Table, Alignment を確認する。 |
| Boot            | Loader が Kernel と Init を配置する。 | boot_info, Image Path, Frame 数, Entry Point を確認する。          |
| Init Entry      | User Entry の識別文字列を観測できる。      | Stack, init_info Address, Kernel Call Entry を確認する。          |
| Capability Call | 成功例と失敗例が期待した Result を返す。      | Descriptor, Rights, Operation Number, Message Layout を確認する。 |

| 段階                   | 合格条件                                                   | 失敗時の確認箇所                                                           |
|----------------------|--------------------------------------------------------|--------------------------------------------------------------------|
| Capability Inventory | Copy, Move, Mint, Transfer 後の Owner と Rights が台帳と一致する. | Source Slot, Destination Slot, Dependency, 途中失敗時の中間 State を確認する.   |
| Process              | Child が Entry へ到達し, Fault を配送できる.                      | Address Space, Frame, PCB, Fault Resolver, Scheduling State を確認する. |
| IPC                  | Server が待機後に Client Message を受信して Reply する.            | Message Info, Queue State, Reply Authority, Priority を確認する.        |
| Notification         | 複数 Bit と同一 Bit の再通知が定義した Event 処理になる.                  | Identifier, Pending Bit, Device State の読出しを確認する.                   |
| Teardown             | Waiter を残さず, 派生 Capability を失効して Resource を閉じる.        | 停止順, Unmap, Binding 解除, Revoke 対象を確認する.                            |

各段階では、成功経路と少なくとも一つの意図的な失敗経路を実行する。Capability Call の失敗経路では、存在しない Descriptor または不足した Rights を与え、`MR0 = 0` と期待する `capability_error` を確認する。IPC の失敗経路では、Message Length と Capability Transfer 数を上限で検査する。

`DEBUG Kernel Call` は、Init Entry までの起動確認に限って利用する。`DEBUG` は Deprecated であり、Application の出力 Interface として扱わない。System の出力は、権限を制限した I/O Service を構成し、Client から IPC で利用する。

# Part V Architecture-specific Interface

本 Part は、x86\_64 固有の ABI を定義し、新しい Architecture へ HAL を移植する手順を示す。





# 16

## x86\_64 ABI

### ■ 16.1 Scope

x86\_64 ABI は、A9N の共通インターフェースを x86\_64 Long Mode へ対応させる規約である。対象 Revision で実装されている Hardware Architecture は x86\_64 だけである。本章では、Register、Page Table、I/O Port の HAL 実装、Boot 情報、割り込み、VMX の順に固有の規約を示す。

| 項目                | 値                                                               |
|-------------------|-----------------------------------------------------------------|
| Word 幅            | 64 bit.                                                         |
| 基本 Page Size      | 4 KiB. PAGE_SIZE = 4096, INITIAL_FRAME_SIZE_BITS = 12.          |
| User Address      | $0 \leq \text{address} < 0x0000\_7fff\_ffff\_ffff$ . 上限値は含まれない. |
| Kernel Direct Map | 物理 Address へ KERNEL_VIRTUAL_BASE を加えた Virtual Address を用いる.     |
| IRQ 数             | IRQ_NUMBER_MAX = 256. IRQ 番号は 0 から 255 である.                     |
| 実行 Model          | Single-core. SMP Scheduling と Remote TLB Shootdown は実装されていない.   |

### ■ 16.2 Kernel Call Register

Kernel Call には `syscall` 命令を用いる。RAX に Call Number を置き、Virtual Message Register の MR0 から MR9 を General-purpose Register へ割り当てる。MR10 以降は、実行中 Process の `ipc_buffer.messages[index]` へ割り当てる。MESSAGE\_BUFFER\_SIZE\_MAX は 494 であり、messages の有効 Index は 0 から 493 である。x86\_64 HAL は MR Index の範囲を検査しない。ユーザ空間は MR<sub>i</sub> を使用する前に  $i < 494$  を満たすことを検証する必要がある。

| ABI 上の値     | Register                   |
|-------------|----------------------------|
| Kernel Call | RAX                        |
| MR0         | RDI                        |
| MR1         | RSI                        |
| MR2         | RDX                        |
| MR3         | R8                         |
| MR4         | R9                         |
| MR5         | R10                        |
| MR6         | R12                        |
| MR7         | R13                        |
| MR8         | R14                        |
| MR9         | R15                        |
| MR10..      | ipc_buffer.messages[index] |

syscall 命令は User RIP を RCX へ、User RFLAGS を R11 へ保存する。Kernel Call Entry は保存値を sysret に用いる。ユーザ空間は RCX と R11 が呼出し後も保存されると仮定してはならない。対象 Revision にはユーザ空間向けの C Function Signature がない。Wrapper は、C ABI の引数順ではなく、Message Register 番号に従って値を置く必要がある。

## ■ 16.3 Hardware Context

PCB の READ\_REGISTER と WRITE\_REGISTER が扱う Hardware Context は 23 Word である。Fault IPC の INVALID\_KERNEL\_CALL も、同じ並びを MR7 以降に格納する。

| Index | Register   |
|-------|------------|
| 0     | RAX        |
| 1     | RBX        |
| 2     | RCX        |
| 3     | RDX        |
| 4     | RDI        |
| 5     | RSI        |
| 6     | RBP        |
| 7     | R8         |
| 8     | R9         |
| 9     | R10        |
| 10    | R11        |
| 11    | R12        |
| 12    | R13        |
| 13    | R14        |
| 14    | R15        |
| 15    | RIP        |
| 16    | CS         |
| 17    | RFLAGS     |
| 18    | RSP        |
| 19    | SS         |
| 20    | GS_BASE    |
| 21    | FS_BASE    |
| 22    | ENTER_FROM |

PCB の `thread_local_base` は `GS_BASE` へ保存する。 `WRITE_REGISTER` は User Address, Segment Selector, `RFLAGS`, `ENTER_FROM` を検証しない。 Process Manager は, Privilege Level を変えない Field だけを書き換える必要がある。

### CAUTION

実行中 Process が自身の PCB へ `READ_REGISTER` を実行する場合, `register_count` は 8 未満にする必要がある。 `register_count >= 8` では, Index 0 の返却先 MR3 が Hardware Context の Index 7 (R8) を上書きしてから Index 7 を読むため, Index 7 以降は呼出し開始時点の Snapshot にならない。 23 Word の Context を確認する Process Manager は, 対象 Process を停止して別 Process から `READ_REGISTER` を実行する必要がある。

## 16.4 Page Table

x86\_64 の Address Space は, 4 階層の Page Table で構成される。 Address Space を作成すると, HAL は Kernel Root Page Table を新しい PML4 へ Copy する。 ユーザ空間の Mapping には Lower Half だけを使用する。

| Level | Depth | 範囲      | 役割                                |
|-------|-------|---------|-----------------------------------|
| PML4  | 4     | 512 GiB | Address Space の Root.             |
| PDPT  | 3     | 1 GiB   | 中間 Table, または 1 GiB Frame の Leaf. |
| PD    | 2     | 2 MiB   | 中間 Table, または 2 MiB Frame の Leaf. |

| Level | Depth | 範囲    | 役割                        |
|-------|-------|-------|---------------------------|
| PT    | 1     | 4 KiB | 4 KiB Frame の Leaf を格納する。 |

Frame Size Bits は 12, 21, 30 である。12 は常に利用できる。21 と 30 は、Boot 時に検出した CPU 機能に依存する。CAN\_MAP\_FRAME\_SIZE\_BITS は、指定した Size を利用できる場合に成功を返す。Page Table の Depth は 1 から 3 である。

| Attribute | 値   | Page Entry                                                                  |
|-----------|-----|-----------------------------------------------------------------------------|
| READ      | 0x1 | Present な User Mapping を作る。独立した Read-disable Bit がないため、実装は Bit 0 を個別に判定しない。 |
| WRITE     | 0x2 | rw を 1 にする。                                                                 |
| EXECUTE   | 0x4 | execute_disable を 0 にする。未指定の場合は NX を設定する。                                   |

4 KiB Frame は PT へ、2 MiB Frame は PD へ、1 GiB Frame は PDPT へ Map する。Frame の Physical Address は 12 bit 右 Shift して Page Entry へ格納する。Virtual Address と Physical Address の Alignment は Address Space Call で検査されないため、ユーザ空間が Frame Size に合わせる必要がある。

実行中 Process の Address Space を変更した場合、HAL は対象 Address へ `invlpg` を実行する。別の Address Space を変更した場合は即時に TLB を無効化しない。後続の Context Switch で CR3 が変わると、Address Space の切替えによって TLB が更新される。PCID は使用しない。Remote TLB Shutdown は実装されていない。

## ■ 16.5 I/O Port HAL

I/O Port Capability の Object Model と Capability Call は「I/O Port」に記載する。x86\_64 HAL は、I/O Address を 16 bit Port 番号として `in` 命令と `out` 命令へ渡す。Word 幅の Address は `uint16_t` へ変換されるため、上位 bit は切り捨てられる。

Byte Width 1, 2, 4 は、順に 8 bit, 16 bit, 32 bit の命令へ対応する。別の Width は `hal_error::ILLEGAL_ARGUMENT` となり、Capability Call では FATAL へ変換される。Write Data は命令幅へ切り詰められ、Read Data の上位 bit は 0 となる。

## ■ 16.6 Boot and Init

INITIAL\_FRAME\_SIZE\_BITS は 12 であり、Init Image を 4 KiB 単位で Map する。`init_image_size` には Byte 数ではなく 4 KiB Frame 数を設定する。Kernel は新しい PML4 を作成し、Init Image を Virtual Address 0 から連続して Map する。IPC Buffer も 4 KiB 境界へ配置する必要がある。Kernel 初期 Allocator の Size は `PAGE_SIZE * 1024`、すなわち 4 MiB である。

`boot_info.arch_info[0]` は、ACPI RSDP の Physical Address である。HAL は Kernel Direct Map Address へ変換して ACPI の初期化に渡す。0 は受理されない。`arch_info[1..127]` は使用しない。

Init の Stack Pointer は 0 のまま User Mode へ復帰する。Init Entry は、C または C++ の Prologue を実行する前に、Assembly Stub で Map 済み Stack の上端を RSP へ設定する必要がある。

## ■ 16.7 Interrupt

IRQ 番号は 0 から 255 である。Hardware IRQ を Notification Port へ配送した後、Kernel は IRQ を Acknowledge して Mask する。Driver は Device 固有の Status を処理してから Interrupt Port の ACK を呼び、IRQ を Unmask する必要がある。

Bind 済み Handler がない IRQ はユーザ空間へ配送されない。Bind 済み Handler がない場合、カーネルは IRQ を Acknowledge または Mask しない。Interrupt Controller に残る状態は、x86\_64 HAL の Controller 実装に依存する。

## ■ 16.8 Initial Capability Descriptors

Init Root Node の radix\_bits は 8, ignore\_bits は 0 である。Root Slot  $n$  を直接指定する Descriptor は、Depth 16, Encoded Depth 8 であり、次式となる。

$$d_{\text{root}(n)} = 0x0800\_0000\_0000\_0000 + n \times 2^{48}. \quad (8)$$

PCB を保持する Root Slot 1 は 0x0801\\_0000\\_0000\\_0000, Generic Node を保持する Root Slot 7 は 0x0807\\_0000\\_0000\\_0000 である。Root Slot 7 の Generic Node は radix\_bits = 7 である。Generic Node 内の Slot  $g$  を指定する Descriptor は、Depth 23, Encoded Depth 15 であり、次式となる。

$$d_{\text{generic}(g)} = 0x0f07\_0000\_0000\_0000 + g \times 2^{41}, \quad 0 \leq g < 128. \quad (9)$$

generic\_list\_count は、Kernel が Generic Node へ設定した Slot 数の確認に用いる。対象 Revision には「Generic Descriptor Generation」に記載した Count の不整合があるため、Descriptor を構成する前に Generic Node の有効範囲と Bootloader の Memory Map 制約を照合する必要がある。

## ■ 16.9 ABI Conformance Example

Repository には、x86\_64 ABI 境界を検査する自己完結型 Payload として doc/a9n-manual/examples/x86\_64-hello を収録する。検査対象は、Init Entry, Stack, init\_info Layout, Register-backed MR, IPC Buffer-backed MR, CAPABILITY\_CALL, Capability Error, DEBUG, YIELD である。

ABI Conformance Example は、Nun と a9n\_abi を使用する標準 User Payload とは目的が異なる。ABI Conformance Example を Application または System Init の雛形として使用してはならない。Build, ELF Layout 検査, 実行, 期待する Serial 出力は doc/a9n-manual/examples/x86\_64-hello/README.md に記載する。本章は、Example が検査する ABI 値と不変条件だけを定義する。



# 17

## HAL Porting Guide

### ■ 17.1 Scope and Starting State

HAL を移植するには、新しい Architecture 用の Boot, Context, Memory Mapping, Interrupt, Timer, I/O, Kernel Call Entry を実装し、共通の Kernel Interface へ接続する。対象 Revision で起動まで実装されている Architecture Directory は `src/hal/x86_64` だけである。

移植には、Freestanding C++20 Toolchain, Target 用 Assembler と Linker, Boot 環境, Architecture Manual, Interrupt Controller と Timer の仕様が必要となる。Kernel の初期化, Init の起動, Capability Call, Timer Preemption, IPC, Fault 配送が Target Hardware または Emulator 上で動作すれば、最小の Port が成立する。

### ■ 17.2 Source Layout

新しい Architecture 名を `{ARCH}` とする。Directory は次のように構成する。

```
src/hal/{ARCH}/
├─ CMakeLists.txt
├─ toolchain.cmake
├─ kernel.ld
├─ include/hal/arch/arch_types.hpp
├─ boot/
├─ arch/
├─ process/
├─ syscall/
├─ memory/
├─ interrupt/
├─ time/
├─ io/
└─ factory/
```

Directory 名は CMake の ARCH 値と一致させる。Root CMake が `src/hal/${ARCH}` の存在を確認し、`src/hal/CMakeLists.txt` が Subdirectory を追加する。Kernel Target には `src/hal/${ARCH}/include` を Include Path として渡す。

**CAUTION**

src/hal/CMakeLists.txt の一つの Include Path は `${arch}` という Lowercase 変数を参照する。標準入力変数は `${ARCH}` である。新しい Port で Architecture Header が見つからない場合、Lowercase 参照を修正する必要がある。

## 17.3 Architecture Constants

`include/hal/arch/arch_types.hpp` は Kernel Data Structure と ABI Size を決める。少なくとも次の定数を定義する必要がある。

| Constant                 | Description                                                              |
|--------------------------|--------------------------------------------------------------------------|
| BYTE_BITS                | 1 Byte の Bit 数。A9N 共通型は 8 を前提とする。                                        |
| PAGE_SIZE                | 基本 Page Size。Kernel 共通実装が仮定する値と一致させる。                                    |
| KERNEL_VIRTUAL_BASE      | Physical Memory の Kernel Direct Map Base。                                |
| USER_VIRTUAL_BASE        | User Virtual Address Base。                                               |
| HARDWARE_CONTEXT_SIZE    | PCB Hardware Context の Word 数。Register Read/Write ABI と Fault ABI へ露出する。 |
| FLOATING_CONTEXT_SIZE    | Floating-point Context の Word 数。                                         |
| VIRTUAL_CPU_CONTEXT_SIZE | Virtual CPU Context Buffer Size。Virtualization 未完成でも Build に必要である。       |
| VIRTUAL_CPU_STATE_COUNT  | Virtual CPU State Descriptor 数。                                          |
| IRQ_NUMBER_MAX           | Kernel IRQ Handler Table の Entry 数。                                      |
| INITIAL_FRAME_SIZE_BITS  | Init Image を Map する Frame Size Bits。Bootloader の Image Size 単位と一致させる。    |

Kernel 共通 Code には、Page Size、Word 幅、Descriptor の Depth 幅、Kernel Direct Map を固定値として扱う処理がある。異なる値を採用する Port では、HAL だけでなく Kernel 共通 ABI も変更する必要がある。対象 Revision が仮定する値は「x86\_64 ABI」に記載する。

## 17.4 Required HAL Interfaces

| Area              | Interface Header                             | Required responsibility                                                                    |
|-------------------|----------------------------------------------|--------------------------------------------------------------------------------------------|
| Architecture Init | <code>arch_initializer.hpp</code>            | <code>arch_info[]</code> を Decode し、CPU、Interrupt、Timer、Platform を初期化する。                   |
| CPU-local State   | <code>cpu.hpp</code> , <code>lock.hpp</code> | Core 番号、CPU-local Pointer、Kernel Lock を提供する。                                               |
| Process Context   | <code>process_manager.hpp</code>             | Context 初期化、Context Switch、Restore、Message Register、General Register、User Address 検査を実装する。 |
| Memory            | <code>memory_manager.hpp</code>              | Address Space 作成、Page Table と Frame の Map/Unmap、Depth 探索、Frame Size 検査を実装する。               |



| Area           | Interface Header | Required responsibility                                                                  |
|----------------|------------------|------------------------------------------------------------------------------------------|
| Interrupt      | interrupt.hpp    | Timer, Kernel Call, IRQ, Fault Handler 登録と IRQ Mask/Unmask/Ack を実装する。                    |
| Timer          | timer.hpp        | System Clock Frequency 設定を実装する。                                                          |
| Port I/O       | port_io.hpp      | Architecture が持つ I/O Space の Read/Write を実装する。MMIO Architecture は Compatibility 方針を定義する。 |
| Serial         | serial.hpp       | Early Boot Logger 用 Serial Driver を実装する。                                                 |
| Factory        | hal_factory.hpp  | Architecture 固有 HAL Object を生成する。                                                        |
| Virtualization | virtualize.hpp   | Build に必要な Stub または Hardware Virtualization 実装を提供する。                                     |

## ■ 17.5 Kernel Call Port

Kernel Call Entry は、Architecture の Privilege Transition 命令から `kernel_call_handler` を呼び、Kernel Call Number は Signed Word として渡す。Message Register は、`get_message_register()` と `configure_message_register()` から読み書きできるようにする。

Register へ収まらない Message Register は IPC Buffer に割り当てる。Register との対応、破壊される Register、保存される Register、Return 命令、Stack Alignment、Interrupt Masking を Architecture ABI に記載する必要がある。

不正 Kernel Call Number は `fault_dispatcher(INVALID_KERNEL_CALL, ...)` へ渡す。Fault Address、Program Counter、Architecture Fault Code の意味は Architecture Port が定義する。

Context Switch では、Hardware Context、Floating-point Context、Thread-local Base、Address Space を切り替える。同じ Address Space 間で Root Register の再設定を省略してもよいが、TLB の整合性を保つ必要がある。

## ■ 17.6 Memory Port

Memory HAL は、Address Space Root の検査、Page Table Depth、Frame Size、Mapping の重複、Unmapping、TLB Invalidation を実装する。Kernel は HAL Error を Capability Error へ変換するため、`memory_map_error` の意味を変えてはならない。

`make_address_space()` は、新しい User Address Space へ Kernel Mapping を組み込む。Kernel Mapping を User Mode から参照できないように Page Entry の Privilege を設定する。`is_valid_user_address()` の判定範囲は、User Mapping を作成できる範囲と一致させる。

SMP Port は Remote TLB Shutdown を追加する必要がある。対象 Revision の Kernel 共通 Code は Remote Shutdown を要求する Interface を持たないため、SMP 対応は Interface 拡張を伴う。

## ■ 17.7 Interrupt and Timer Port

Architecture Exception は A9N の `fault_type` へ変換する。Memory Fault には Fault Address と Instruction-fetch の区分を付ける。Invalid Instruction, Arithmetic Fault, Invalid Kernel Call, Architecture Fault は、Resolver IPC へ配送できる情報へ変換する。回復できない Exception は FATAL として扱う。

外部 IRQ は 0 から `IRQ_NUMBER_MAX - 1` までの Kernel IRQ 番号へ正規化する。Timer IRQ は Process Manager の `handle_timer()` へ接続する。外部 IRQ は Notification 後に Mask し、Interrupt Port の ACK で Unmask する。

## ■ 17.8 Boot Protocol Port

Boot Entry は `boot_info*` を Kernel へ渡す。Bootloader と HAL は、`arch_info[]` の Index ごとの意味を共有する必要がある。Init Image は `INITIAL_FRAME_SIZE_BITS` 単位で、User Address 0 から Map される。Bootloader が用いる Frame Size も同じ値にする。

新しい Architecture の Build は、Pointer の Size と Alignment が 1 Word であること、`memory_map_type` が 4 Byte、`bool` が 1 Byte であることを検査する必要がある。`offsetof` と `sizeof` による Build-time Assertion は、「Boot and Init Protocol」の Word 単位の Layout と `boot_info`, `memory_info`, `memory_map_entry`, `init_image_info`, `generic_descriptor`, `init_info` の実 Layout を照合する。Layout が一致しない場合、Bootloader と Kernel で同じ Structure を共有してはならない。

Kernel Linker Script は Kernel Virtual Address, Physical Load Address, Boot Section, Page Table, Stack, Entry Symbol を定義する。Boot Assembly は Kernel Direct Map と Kernel Stack を確立してから C++ Entry へ移る必要がある。Global Constructor が実行される保証はないため、Early Boot Object は Static Storage と Placement New を使用する。

## ■ 17.9 Build and Verification

Docker Build の実行例は次の通りである。{ARCH} は Architecture Directory 名へ置き換える。

```
ARCH={ARCH} BUILD_TYPE=Release docker compose run --rm a9n-build
```

Local CMake Build の実行例は次の通りである。

```
cmake -S . -B build-{ARCH} \
-DARCH={ARCH} \
-DCMAKE_TOOLCHAIN_FILE=./src/hal/{ARCH}/toolchain.cmake \
-DCMAKE_BUILD_TYPE=Debug
cmake --build build-{ARCH}
```

Build 成功時は `kernel.elf` と `kernel.map` が Build Directory に生成される。Build 成功だけでは Port 完了としない。Port は次の順で検証する。

1. Serial Logger が Kernel Entry と Boot 情報を出力する。
2. Architecture, Interrupt, Process Manager の初期化が完了する。
3. Init が User Mode で実行され、有効な Stack と `init_info` を取得する。
4. DEBUG Call と YIELD Call が Return する。
5. Capability Descriptor を構成し、Init Root Capability Node の Slot を探索できる。
6. Generic から Node, Frame, Address Space, PCB, IPC Port を作成できる。
7. Address Space へ Page Table と Frame を Map し、User Memory へ Access できる。
8. IPC Port の CALL/REPLY\_RECEIVE が Payload と Capability を転送する。
9. Notification と外部 IRQ が Driver へ配送され、ACK で再 Enable される。
10. User Fault が Resolver IPC へ配送され、Reply 後に Process が再開する。
11. Timer Tick が同 Priority Process を Round-robin で切り替える。

Test HAL の `src/hal/test/include/hal/arch/arch_types.hpp` は Host Unit Test 用であり、Hardware Port の代替ではない。Architecture Port は、Register ABI, Page Table, Interrupt Entry, Context Restore を Target Emulator または実機で検証する必要がある。



## Part VI Ecosystem

本 Part は、A9N Microkernel と組み合わせる Software Component の責務と選択基準を示す。



# 18

## Ecosystem

### ■ 18.1 Scope

A9N Ecosystem は、A9N Kernel を用いる System の Build, Boot, User-level Runtime, Kernel Call Adapter, 共有型を構成する Software 群である。

SPENCER, Nun, A9NLoader-rs, a9n\_abi, a9n\_types は、異なる実行段階と抽象度を担当する。各 Component の境界を保つことで、Boot 処理, ABI, Runtime, Application Policy を個別に更新できる。

SPENCER, Nun, A9NLoader-rs, a9n\_abi, a9n\_types の役割はアーキテクチャに依存しない観点から説明する。Kernel Call Register, Entry Stub, UEFI Image の配置を含む x86\_64 固有の構成は「x86\_64 ABI」に記載する。

### ■ 18.2 Component Relationship

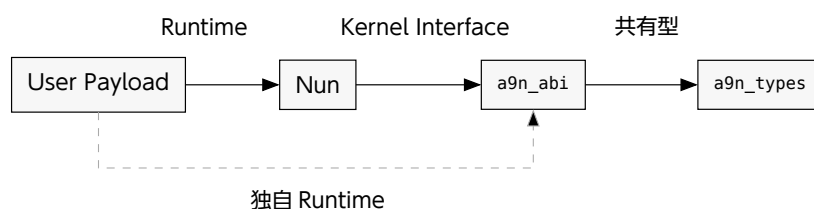


Figure 24: User-level Component の依存関係

User Payload は、Nun を Runtime として利用する構成と、a9n\_abi を直接利用して独自 Runtime を構成する方式を選択できる。両方式は a9n\_types が定義する共有値を用いる。SPENCER と A9NLoader-rs は Cargo Dependency の層に属さず、Build と Boot を担当する。

### ■ 18.3 a9n\_types

a9n\_types は Cargo Package 名、a9n\_types は Rust の crate 名である。a9n\_types は no\_std crate であり、Kernel と User-level Software の境界を越える共有値を定義する。Word, CapabilityDescriptor, CapabilityType, CapabilityRights, CapabilityError, MessageInfo, InitInfo, IpcBuffer が対象となる。

`a9n_types` は、Kernel Call Number, Capability Operation Number, Register 操作, Runtime 初期化を実装しない。ABI として共有する構造体と列挙型には明示的な Representation が必要であり、Field 順, Alignment, 数値を Kernel 側の定義と一致させる必要がある。

## ■ 18.4 a9n\_abi

`a9n_abi` は、User Mode から A9N Kernel を呼び出す低水準の `no_std` crate である。`a9n_types` へ依存して共有型を再公開し、Kernel Call Number, Capability Operation Number, Message Register Layout, Capability Call Wrapper を提供する。Architecture Module は、Trap 命令と Hardware Register への配置を実装する。API Documentation は [docs.rs](https://docs.rs/a9n_abi) の `a9n_abi` で公開されている。

独自 Runtime は、`a9n_abi` を直接利用して Entry Point, IPC Buffer 初期化, Panic 処理を実装できる。Application Policy や Resource 管理は `a9n_abi` の責務に含まれない。Architecture Module の Register 配置と Kernel Call Entry は、アーキテクチャごとの ABI に従う必要がある。

## ■ 18.5 Nun

Nun は、A9N 上で Rust 製 User-level Software を構築するための `no_std` Runtime である。Nun は `a9n_abi` へ依存し、`a9n_abi` が公開する型と Kernel Call Wrapper を再公開する。`entry!` Macro は Architecture Entry を生成し、`InitInfo` を User Entry へ渡す。初期化処理は Init の IPC Buffer を Thread-local Storage へ対応付ける。Debug 出力と Panic 処理も Nun が提供する。

Nun は、Executable Loader, Memory Manager, Driver, File System, Process Manager を提供しない。System Service と Resource Policy は、Nun を利用する User Payload が実装する。Entry, IPC Buffer, Debug 出力を独自実装する場合、Nun を介さずに `a9n_abi` を利用できる。

## ■ 18.6 A9NLoader-rs

`A9NLoader-rs` は、A9N Boot Protocol を実装する Rust 製 Bootloader である。Disk Image 内の `kernel.elf` と `init.elf` を読み込み、ELF Segment と Symbol を解析する。`A9NLoader-rs` は、Firmware の Memory Map と Hardware 情報を `boot_info` へ格納し、Firmware Service を終了した後に Kernel Entry へ制御を渡す。

`A9NLoader-rs` は Boot 完了後に常駐する System Service ではない。Kernel は `boot_info` から Init Image を構成し、Init へ `init_info` を渡す。Boot 情報の共通構造は「Boot and Init Protocol」に、特定 Architecture での Firmware と Entry 規約は「x86\_64 ABI」に記載する。



## ■ 18.7 SPENCER

SPENCER は、A9N-based System の Source 取得、Build、Disk Image 生成、実行を統合する ToolKit である。SPENCER の Git Submodule は、A9N、Nun、A9NLoader-rs を保持する。a9n\_abi と a9n\_types は Git Submodule ではなく、Nun または User Payload の Cargo Dependency として解決される。

SPENCER は A9N Kernel、A9NLoader-rs、User Payload を選択した構成で Build し、三つの Executable を Boot 可能な Disk Image へ格納する。Command、既定値、Artifact Path、起動確認は「Getting Started」を正本とする。SPENCER が指定する Submodule 構成と、User Payload の Cargo.lock が確定する Cargo Dependency を一つの組合せとして扱う必要がある。

## ■ 18.8 Layer Selection

共有構造体と列挙型だけを扱う Library は a9n\_types を用いる。Kernel Call Adapter を独自に構成する Runtime は a9n\_abi を用いる。標準の Rust Entry と IPC Buffer 初期化を必要とする User Payload は Nun を用いる。Kernel、Bootloader、User Payload を一括して Build・実行する開発環境は SPENCER を用いる。独自 Bootloader は、A9NLoader-rs と同じ A9N Boot Protocol を実装する必要がある。

### CAUTION

a9n\_types の共有値、a9n\_abi の Operation Number、A9N Kernel の Header、A9NLoader-rs の Boot 構造体は、同じ境界仕様を表す。一つの Component だけを更新すると、Compile 成功後の Kernel Call 失敗、Init 情報の誤読、Boot Failure が発生し得る。SPENCER の Submodule と Payload の Cargo.lock が固定する組合せを基準に Build と起動を検証する必要がある。



# Back Matter Glossary and References

本 Part は、用語集、謝辞、参考文献を収録する。



# Glossary

## ■ Architecture-independent Terms

### Access Control List

Object ごとに、呼出し主体の Identity と許可する Operation を対応付けた Entry を保持する Access Control Model である。ACL と略記する。

### A9N Microkernel

Capability に基づく権限管理を採用した第 3 世代の Microkernel である。Kernel Object の操作、Process の実行、仮想メモリ、IPC、Notification、割り込み配送の機構を提供する。

### A9N Ecosystem

A9N Kernel を用いる System の Build, Boot, User-level Runtime, Kernel Call Adapter, 共有型を構成する Software 群である。

### a9n\_abi

Kernel Call Number, Capability Operation Number, Message Register Layout, Architecture ごとの Kernel Call Wrapper を提供する `no_std` Rust crate である。

### a9n\_types

Capability, Message, Init, IPC Buffer に関する共有値を定義する、アーキテクチャ非依存の `no_std` Rust crate である。

### Base Library

Freestanding 環境で Kernel と User-level System が使用する基盤 Library である。Container, Result, Option, libc 互換処理を提供する。

### Capability

Kernel Object に対する操作権限を表す、カーネル管理の参照である。Object Type, Rights, Slot-local Data, Dependency 情報とともに Capability Slot へ格納する。

### Capability Call

Capability Descriptor によって対象 Capability Slot を探索し、指定した Kernel Object Operation を実行する Kernel Call である。

### Capability Descriptor

Root Capability Node を起点として Capability Slot を探索するための 1 Word Address である。上位 8 bit に Encoded Depth を、残りの bit に Descriptor Payload を格納する。

**Capability Space**

Root Capability Node から到達できる Capability Node と Capability Slot の階層である。

**Capability Slot**

Capability を格納するカーネル管理領域である。Object への参照, Type, Rights, 3 Word の Slot-local Data, Dependency 情報を保持する。

**Depth**

Capability Descriptor 先頭から, Slot 探索を終了する Node 境界までの bit 数である。8 bit の Encoded Depth Field と, 探索経路上で消費する Ignore Bits および Radix Bits の総和で表す。

**Direct Schedule**

IPC の通信相手を通常の Scheduling を介さずに選択する経路である。Ready Queue に通信相手より高い Priority の Process が存在する場合は, 通常の Scheduling へ戻る。

**Descriptor Payload**

Capability Descriptor の下位  $W - 8$  bit である。各 Capability Node の Node Index を上位側から順に配置する。

**Encoded Depth**

Capability Descriptor の上位 8 bit へ格納する値である。Depth を  $D$  とすると, Encoded Depth は  $D - 8$  となる。

**HAL**

Hardware Abstraction Layer の略称である。CPU 初期化, Context Switch, Memory Mapping, 割り込み制御, Kernel Call Entry のアーキテクチャ依存処理を共通 Kernel Interface へ接続する。

**Ignore Bits**

Capability Node の探索時に読み飛ばす Descriptor bit 数である。

**Kernel Call**

ユーザ空間からカーネル機能呼び出す共通インターフェースである。通常利用する Call は CAPABILITY\_CALL と YIELD の 2 つである。Deprecated な DEBUG も実装に残る。

**Kernel Call Adapter**

Operation 固有の値を Virtual Message Register へ配置し, Architecture ごとの Kernel Call Entry を呼び出し, 結果を User-level Software の値へ戻す層である。

**Kernel Object**

Capability を介して操作するカーネル管理 Object である。ユーザ空間へ Object の Memory Address を公開しない。

**Policy/Mechanism Separation**

機構と方針の分離。Resource の操作と保護に必要な Mechanism を Kernel へ置き, Resource の配分や Service の構成を決める Policy を User-level Software へ委ねる設計原則である。

**Nun**

A9N 上で Rust 製 User-level Software を構築するための `no_std` Runtime である。Entry 生成, IPC Buffer 初期化, Debug 出力, Panic 処理を提供する。

**Virtual Message Register**

Kernel Call の引数と結果を運ぶ Process ごとの論理 Word 列である。Message Register または MR と略記する。HAL が各 Index を Hardware Context 内の Register または IPC Buffer へ割り当てる。

**Node Index**

Capability Node 内の Capability Slot を選択する Index である。Descriptor から Radix Bits 分を取り出して算出する。

**Object-Capability Model**

Object を指定する参照と Authority を Capability として一体化し、呼出し主体から到達可能な Capability を操作可否の根拠とする Access Control Model である。

**Radix Bits**

Capability Node の Slot 数と Node Index の幅を表す bit 数である。Node は  $2^{\text{radix\_bits}}$  個の Slot を持つ。

**Reply Authority**

IPC の CALL に対して Reply できる一時的な権限である。Process State として保持し、Capability Slot として Copy または Transfer できない。

**Rights**

Capability Slot に設定する操作権限の bit 集合である。READ, WRITE, COPY, MODIFY を定義するが、具体的な検査内容は Operation ごとに異なる。

**Slot-local Data**

Capability Slot ごとに保持する 3 Word の補助情報である。IPC Port と Notification Port の Identifier, I/O Port Capability の Address Range に使用する。

**Slot-local Identifier**

IPC Port または Notification Port を参照する Capability Slot の `data[0]` に保持する 1 Word 値である。Kernel は、IPC では Receiver の MR3 へ、Notification では Pending Flag を介して MR2 または MR3 へ配送する。

**User-level System**

Init を起点としてユーザ空間に構成する OS 部分である。Capability の配布, Memory Manager, Driver, File System, System Service の Policy を実装する。

**User-level Software**

A9N の User Mode で動作し、Kernel Call によって Kernel Object を操作する Software である。Init と、Init が構成する Service Process を含む。

**Word**

A9N の Kernel Interface が整数, Address, Capability Descriptor, Message Register の基準に用いる `a9n::word` である。32-bit Word は 4 Byte, 64-bit Word は 8 Byte である。

## ■ Kernel Objects and Runtime

### Address Space

Root Page Table を表す Capability Object である。Page Table と Frame の Mapping および Unmapping を実行する。

### A9NLoader-rs

A9N Boot Protocol に従って Kernel Image と Init Image を配置し、`boot_info` を Kernel へ渡す UEFI Bootloader である。

### Capability Node

$2^{\text{radix\_bits}}$  個の Capability Slot を持つ Radix Tree Node である。別の Capability Node を Slot へ格納して Capability Space を階層化できる。

### Fault Resolver

Process Fault の配送先として PCB へ設定する IPC Port である。Resolver は Fault Message を受信し、必要に応じて Reply する。

### Frame

物理メモリ範囲を表す Capability Object である。Address Space Operation の引数として Virtual Address へ Map する。

### Generic

Kernel Object の作成に使用する物理メモリ領域を表す Capability Object である。Base Address, Size Bits, Device Flag, Watermark を保持する。

### Init

カーネルが最初に起動する User Process である。起動時に Initial Capability Space, Address Space, Hardware 情報を受け取る。

### Init Image

Bootloader が配置し、Kernel が最初の User Process として Map する Executable Image である。Entry Point, `init_info` 領域, IPC Buffer, Stack を含む。

### Interrupt Port

一つの IRQ 番号を保持し、割り込みを Bind 済み Notification Port へ配送する Capability Object である。

### Interrupt Region

IRQ 番号を割り当てる権限を表す Capability Object である。未使用 IRQ に対応する Interrupt Port を作成する。

### I/O Port Capability

HAL が提供する I/O Space への Read/Write 権限を表す Kernel Object である。I/O Address の意味と Access 方法は HAL が定める。

### IPC Buffer

Message Register の退避, IPC Payload, Capability Transfer の Metadata に用いる共有データ構造である。



**IPC Fastpath**

Server が `REPLY_RECEIVE` を発行して Receiver Queue で待機した後、Server より高い Priority の実行可能 Process が存在しない場合に、Client の `CALL` から Server へ Direct Schedule する IPC 経路である。

**IPC Port**

Message と Capability を Process 間で転送する Capability Object である。Sender Queue または Receiver Queue を FIFO で保持する。

**Notification Port**

Word 幅の Pending Flag と FIFO の Wait Queue を持つ Capability Object である。通知値を Bitwise OR で蓄積する。

**Page Table**

Address Translation の中間 Table を表す Capability Object である。Mapping 操作は Address Space を介して実行する。

**Process**

Scheduler が実行、Block、再開するユーザ空間の実行単位である。実行文脈と Resource 参照は Process Control Block が保持する。

**Process Control Block**

Process の実行文脈と状態を保持する Capability Object である。PCB と略記する。Priority, Quantum, Address Space, Root Capability Node, IPC Buffer, Notification Port, Fault Resolver を管理する。

**Root Capability Node**

Process が Capability Descriptor による Slot 探索を開始する Capability Node である。PCB 内部の Root Slot から参照する。

**SPENCER**

A9N Kernel, A9NLoader-rs, User Payload を Build し、Boot 可能な Disk Image を生成する ToolKit である。

**Benno Scheduling**

実行可能な Process だけを Priority ごとの FIFO Ready Queue で管理する Scheduling 手法である。A9N の Scheduler は、固定 Priority で Queue を選択し、同じ Priority の Process を Round-robin で選択する。

**Virtualization Capability**

Virtual Machine 構成用として宣言された Capability 群である。対象 Revision では Experimental Stub であり、Guest を実行する経路は完成していない。

**Virtual CPU**

Virtual Machine の CPU 状態を保持する目的で宣言された Capability Object である。対象 Revision では Capability Call を利用できない。

**Virtual Address Space**

Guest Physical Address Space を表す目的で Type 値が予約された Capability Object である。対象 Revision は作成経路と Component 実装を持たない。

### **Virtual Page Table**

Guest 用 Page Table を表す目的で Type 値が予約された Capability Object である。対象 Revision は作成経路と Component 実装を持たない。

## **■ Architecture-dependent Terms**

### **ABI**

共通 Kernel Interface を特定の Hardware Architecture へ対応させる規約である。Register 配置, Context Layout, Page Table, Kernel Call Entry を定める。

### **Hardware Context**

Process の実行再開に必要な CPU Register 状態である。Word 数, Index, 書換え可能な Field はアーキテクチャごとの ABI が定める。

### **Kernel Call Entry**

ユーザ空間から Privilege を遷移し, Kernel Call Handler へ制御を渡すアーキテクチャ依存経路である。

### **x86\_64 ABI**

A9N の共通インターフェースを x86\_64 Long Mode へ対応させる規約である。対象 Revision で実装されている Hardware Architecture は x86\_64 だけである。

# Acknowledgement

A9N Microkernel の開発は、一般社団法人未踏による 2023 年度未踏ジュニア [9] と、独立行政法人情報処理推進機構（IPA）による 2024 年度未踏 IT 人材発掘・育成事業 [10] の支援を受けた。

石井翔氏には、未踏ジュニアの Mentor として助言をいただいた。

怒田晟也氏には、実装上の助言をいただいた。

曾川景介氏には、未踏 IT 人材発掘・育成事業の Project Manager（PM）として助言をいただいた。

また、A9N Microkernel の設計と実装では、MINIX, seL4, Fiasco.OC を参考にした。



## References

- [1] Rekka 'horizon' IGUMI, "horizon2038/A9N: Capability-based 3rd-generation Microkernel/Microhypervisor with High-speed IPC Mechanism.." [Online]. Available: <https://github.com/horizon2038/A9N>
- [2] R. Levin, E. Cohen, W. Corwin, F. Pollack, and W. Wulf, "Policy/mechanism separation in Hydra," in Proceedings of the Fifth ACM Symposium on Operating Systems Principles, in SOSP '75. Austin, Texas, USA: Association for Computing Machinery, 1975, pp. 132–140. doi: [10.1145/800213.806531](https://doi.org/10.1145/800213.806531).
- [3] J. B. Dennis and E. C. Van Horn, "Programming Semantics for Multiprogrammed Computations," Communications of the ACM, vol. 9, no. 3, pp. 143–155, Mar. 1966, doi: [10.1145/365230.365252](https://doi.org/10.1145/365230.365252).
- [4] N. Hardy, "KeyKOS architecture," SIGOPS Oper. Syst. Rev., vol. 19, no. 4, pp. 8–25, Oct. 1985, doi: [10.1145/858336.858337](https://doi.org/10.1145/858336.858337).
- [5] J. S. Shapiro, J. M. Smith, and D. J. Farber, "EROS: a fast capability system," SIGOPS Oper. Syst. Rev., vol. 33, no. 5, pp. 170–185, Dec. 1999, doi: [10.1145/319344.319163](https://doi.org/10.1145/319344.319163).
- [6] seL4 Foundation, "seL4 Reference Manual." July 2026. [Online]. Available: <https://sel4.systems/Info/Docs/seL4-manual-16.0.0.pdf>
- [7] J. H. Saltzer, "Protection and Control of Information Sharing in Multics," ACM SIGOPS Operating Systems Review, vol. 7, no. 4, p. 119, 1973, doi: [10.1145/957195.808059](https://doi.org/10.1145/957195.808059).
- [8] K. Elphinstone and G. Heiser, "From L3 to seL4: What Have We Learnt in 20 Years of L4 Microkernels?," in Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles, in SOSP '13. Farmington, Pennsylvania: Association for Computing Machinery, 2013, pp. 133–150. doi: [10.1145/2517349.2522720](https://doi.org/10.1145/2517349.2522720).
- [9] 未踏ジュニア実行委員会, "A9N: HAL を用いて移植容易性を実現するマイクロカーネル - 未踏ジュニア." [Online]. Available: <https://jr.mitou.org/projects/2023/a9n>
- [10] 情報処理推進機構, "未踏 IT 人材発掘・育成事業：2024 年度採択プロジェクト概要（伊組 PJ） | デジタル人材の育成 | IPA 独立行政法人 情報処理推進機構." [Online]. Available: <https://www.ipa.go.jp/jinzai/mitou/it/2024/gaiyou-sg-2.html>